

- Programlanabilir makineler geliştirildikten sonra, programlar ve programlama dilleri ortaya çıkmaya başladı. •

Devreleri yeniden kablolanmanın erken uygulamaları kanıtlandı çok hantal, bilgisayarı kontrol etmek için bilgisayar dilleri ortaya çıkmaya başladı. • Birincisi, makine dili, ikili

kodlar kullanılarak birer ve sıfırlardan oluşturuldu. – bilgisayar bellek sisteminde program adı verilen talimat grupları olarak depolanır

- Bilim camiası ağırlıklı olarak C/C++ kullanmaktadır.
- Gömülü sistemlerin son araştırması Geliştiriciler C'nin %60 tarafından kullanıldığını gösterdi. – %30'u montaj dilini kullandı – geri kalan BASIC ve JAVA kullanıldı
- Bu diller programcıya programlama ortamı ve bilgisayar sistemi üzerinde neredeyse tam kontrol sağlar. – özellikle C/C++

Mikroişlemci Çağı

- 1971, Intel 4004 ve 4040
 - 4 bitlik mikroişlemciler – 4096 (4 bitlik) bellek konumuna hitap eder – 45 talimat – çoğu hesap makinesi ve mikrodalga fırınlar gibi düşük seviyeli uygulamalar hala 4 bitlik mikroişlemciler kullanır
- 1972, Intel 8008
 - 8 bit mikroişlemci – 16K bellek konumuna hitap ediyor – 48 talimat – Saniyede 50.000 talimat

Pentium Mikroişlemci

- 1993, Intel Pentium (P5 veya 80586)
 - Saatli çalıştırma ile çalışan giriş versiyonları 60 ve 66 MHz frekans ve 110 MIP hızı
 - 120, 133 ve 140 MHz'de çalıştırılan hız aşırı hız sürümleri 233 MHz saat frekansları
 - 16 KB önbellek
 - Tam kare video, 30 Hz veya daha yüksek tarama hızlarında görüntülenir (bu, ticari televizyona benzerdir)

- Bir makineyi programlamak için yeniden kablolanmaktan daha verimlidir. – yine de bir program geliştirmek zaman alıcıdır. gereken program kodlarının sayısına göre
- Matematikçi John von Neumann, ilk Modern insanın talimatları kabul edip hafızada saklayacak bir sistem geliştirmesi.
- Bilgisayarlara genellikle von Neumann **denir** **Onun şerefine** makineler .
 - Babbage'ın da bu kavramı von Neumann'dan çok önce geliştirdiğini hatırlayalım.

- Assembly dili hala önemli bir rol oynuyor. – neredeyse sadece Assembly diliyle yazılmış birçok video oyunu montaj dilinde
- Assembly ayrıca C/C++ ile serpiştirilmiştir makine kontrol fonksiyonlarını verimli bir şekilde gerçekleştirir. – Pentium ve Core2 mikroişlemcilerinde bulunan bazı yeni paralel talimatlar yalnızca montaj dilinde programlanabilir

Mikroişlemci Çağı

- 1973, Intel 8080 (TTL ile uyumlu) – 64K bellek konumlarını ele aldı – saniyede 500.000 talimat (talimat başına 2 s)
- 1974, Motorola 6800
- 1974, İlk kişisel bilgisayar
 - (MITS Altair 8800) -> Gates&Allen'in BASIC yorumlayıcısı
- 1977, Intel 8085 (son 8 bit p Intel tarafından)
 - Saniyede 769.230 talimat (1,3 s/ins.) – dahili saat üretici ve sistem denetleyicisi

Sonraki Mikroişlemciler

- Pentium Pro
- Pentium II
- Pentium III
- Pentium 4
- Çekirdek2
- Dört Çekirdek
- 64-bit mikroişlemciler
- ...

- Daha sonra, ikili kodu girmeyi basitleştirmek için montaj dili kullanıldı. • Montajcı, programcının hafıza kodlarına göre
 - örneğin toplama için ADD
- İkili sayı yerine. – 0100 0111 gibi

- Assembly dili programlamaya yardımcı bir dildir.

Hatırlatma

- Bir bit , bir veya sıfır değerine sahip ikili bir rakamdır
- 4 bit genişliğindeki bellek konumuna genellikle nibble denir
- 8 bit genişliğindeki bellek konumuna bayt denir
- Bir kelime 2 bayttan (veya 16 bitten) oluşan bir koleksiyondur
- 1 KB = 1024 Bayt = 210 Bayt (1 KB = 1024 bayt)
- 1 MB = 1024 KB = 220 Bayt
- 1 GB = 230 Bayt, 1 TB = 240 Bayt
- **B bayt için kullanılır ve b bit için kullanılır**
 - 8 Mbps = Saniyede 8 Megabit (ADSL hızı)

Modern Mikroişlemciler

- 1978, Intel 8086 ve 8088
 - **16 bit** mikroişlemciler – talimat başına 400 ns
 - Saniyede 2,5 milyon talimat
 - 1M bellek konumuna hitap edildi – çarpma ve bölme talimatları eklendi
 - 20.000'den fazla talimat varyasyonu
 - bu tür mikroişlemcilere CISC (karmaşık) adı verilir (komut seti bilgisayarları)
 - Alternatif RISC'dir (azaltılmış komut setli bilgisayarlar)

Bazı Tanımlar

- Programlamanın ilk zamanlarından itibaren ek diller ortaya çıkmıştır.
 - FLOWMATIC-FORTRAN-ALGOL-COBOL-RPG
- Bazı yaygın modern programlama dilleri BASIC, PASCAL, C/C++, C#, Java ve ADA'dır.

– BASIC ve PASCAL dilleri her ikisi de öğretim dili olarak tasarlanmıştı, ancak sınıf dışına çıktılar.



Mikroişlemci Çağı

- Dünyanın ilk mikroişlemcisi Intel 4004.
- 4 bitlik bir mikroişlemci programlanabilir çip üzerindeki denetleyici.
- 4096, 4 bit genişliğindeki bellek konumlarını ele aldı.
- 4004 talimat seti 45 içeriyordu talimatlar.

Modern Mikroişlemciler

- 1983, Intel 80286
 - 16M bellek konumlarına hitap edildi – saat hızı artırıldı (8 MHz) – bazı talimatların yürütülmesi 250 ns sürdü
- 1986, Intel 80386
 - 32-bit mikroişlemci – adresli 4GB bellek
- 1989, Intel 80486
 - 80386 ile aynıdır ancak talimatlarının yarısı iki saat döngüsü yerine tek bir saat döngüsünde yürütülür
 - 8 KB önbellek

Entegre Devre

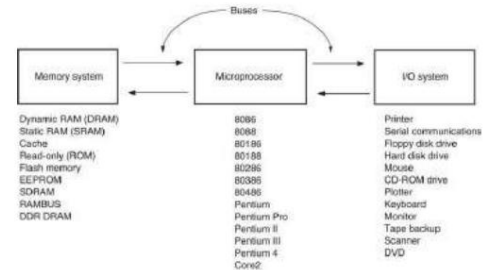
- diğer adıyla IC, mikro devre, mikroçip, silikon çip, veya çip
- Entegre Devre , yarı iletken malzemeden yapılmış ince bir alt tabakanın yüzeyinde üretilen minyatürleştirilmiş bir elektronik devredir (çoğunlukla yarı iletken aygıtlardan ve pasif bileşenlerden oluşur).



Mikroişlemci

- Mikroişlemci, merkezi işlem biriminin (CPU) işlevlerini tek bir yarı iletken entegre devre (IC) üzerinde birleştiren programlanabilir bir dijital elektronik bileşendir.
- 8 bit, 16 bit, 32 bit ve 64 bit mikroişlemci: tek bir işlemde işlenen bit sayısını ifade eder. Programları yürütmek için harici belleğe ihtiyaç duyar. G/Ç aygıtlarına doğrudan arayüz oluşturamaz, çevre birimi yongalarına ihtiyaç vardır.

Şekil 1-6 Mikroişlemci tabanlı bir bilgisayar sisteminin blok diyagramı.



TPA

- Geçici program alanı (TPA), DOS (disk işletim sistemi) işletim sistemini; bilgisayar sistemini kontrol eden diğer programları tutar. – TPA bir DOS kavramıdır ve aslında Windows'da uygulanabilir – ayrıca şu anda etkin veya etkin olmayan DOS uygulama programlarını da depolar – TPA'nın uzunluğu 640K bayttır

Windows Sistemleri

- Modern bilgisayarlar farklı bir bellek kullanır Windows ile DOS bellek haritalarından daha iyi bir haritalama.
- Şekil 1-10'daki Windows bellek haritası iki ana alandan oluşmaktadır; TPA ve sistem alanı.
- DOS ile arasındaki fark bellek haritası bunların boyutları ve konumlarıdır alanlar.

Mikrobilgisayar ve Mikrodenetleyici

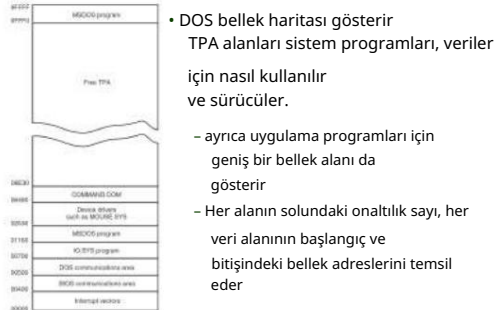
- Mikrobilgisayar , CPU olarak bir mikroişlemci kullanan bir bilgisayardır.
- Mikrodenetleyici (veya MCU), bir çip üzerinde bilgisayardır. Genel amaçlı mikroişlemcilerin (PC'lerde kullanılan tür) aksine, kendi kendine yeterliliği ve maliyet etkinliğini vurgulayan bir mikroişlemci türüdür .
- Mikrodenetleyici ile mikroişlemci arasındaki tek fark, mikroişlemcinin ALU, Kontrol Ünitesi ve kayıtlar olmak üzere üç bölümden oluşması, **mikrodenetleyicinin ise ROM, RAM, çevre birimleri** (zamanlayıcı, G/Ç portları vb.) gibi ek elemanlara sahip olmasıdır.

Bellek ve G/Ç Sistemi

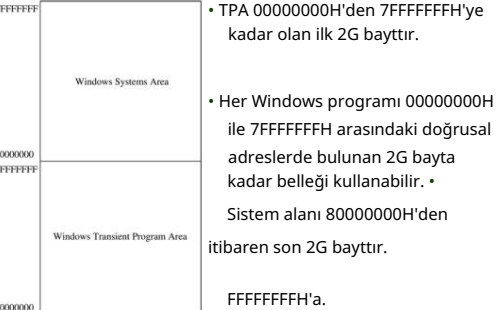
- Tüm Intel tabanlı kişisel bilgisayarların bellek yapısı benzerdir.
- Şekil 1-7, kişisel bilgisayar sisteminin bellek haritasını göstermektedir. • Bu harita, herhangi bir IBM kişisel bilgisayarı için geçerlidir.

– ayrıca mevcut IBM uyumlu klonlar da var

Şekil 1-8 Kişisel bir bilgisayardaki TPA'nın bellek haritası. (Bu haritanın sistemler arasında farklılık gösterebileceğini unutmayın.)



Şekil 1-10 Windows XP tarafından kullanılan bellek haritası.

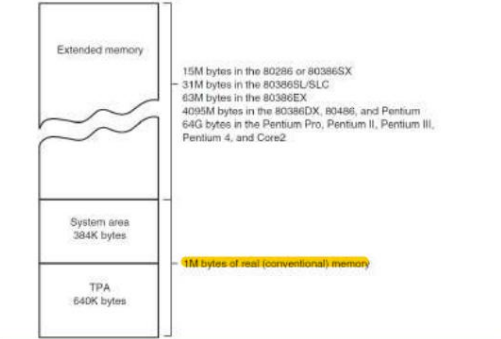


MİKROİŞLEMCI TABANLI

KİŞİSEL BİLGİSAYAR SİSTEMİ

- Bilgisayarlar son zamanlarda pek çok değişikliğe uğradı.
- Bir zamanlar büyük alanları dolduran makineler, küçük masaüstü bilgisayar sistemlerine dönüştü mikroişlemcinin.

Şekil 1-7 Kişisel bilgisayarın bellek haritası.



Sistem Alanı

- TPA'dan daha küçük; ama aynı derecede önemli.
- Sistem alanı salt okunur (ROM) veya flaş bellekteki programları ve veri depolama için okuma/yazma (RAM) bellek alanlarını içerir . • Şekil 1-9 tipik bir bilgisayarın sistem alanını gösterir.
- kişisel bilgisayar sistemi. • TPA haritasında olduğu gibi, bu harita da Çeşitli alanların onaltılık bellek adreslerini içerir.

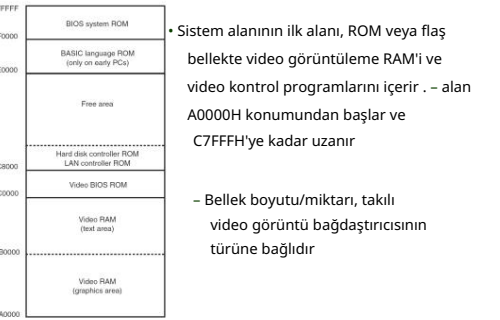
G/Ç Alanı

- G/Ç aygıtları mikroişlemcinin dış dünya ile iletişimi kurmasını sağlar.
- Bir bilgisayar sistemindeki G/Ç (giriş/çıkış) alanı, G/Ç portu 0000H'den başlayarak FFFFH portuna kadar uzanır. – G/Ç bağlantı noktası adresi bir bellek adresine benzer – bellek yerine bir G/Ç cihazına adres verir
- Şekil 1-11 birçok kişisel bilgisayar sisteminde bulunan G/Ç haritasını göstermektedir.

- Şekil 1-6 kişisel bilgisayarın blok diyagramını göstermektedir.
- İlk büyük bilgisayarlardan en son sistemlere kadar her türlü bilgisayar sistemine uygulanabilir .
- Otobüslerle birbirine bağlanan üç bloktan oluşan diyagram. – bir otobüs, ortak bağlantıların kümesidir aynı tür bilgiyi taşıyan

- Ana bellek sistemi üç bölüme ayrılmıştır: – TPA (geçici program alanı) – sistem alanı – XMS (genişletilmiş bellek sistemi)
- Mevcut mikroişlemci türü, genişletilmiş bellek sisteminin var olup olmadığını belirler. • İlk 1M bayt bellek genellikle gerçek veya geleneksel bellek sistemi olarak adlandırılır.
- Intel mikroişlemcileri, işlev görecektir şekilde tasarlanmıştır bu alanda gerçek mod operasyonu kullanılıyor

Şekil 1-9 Tipik bir kişisel bilgisayarın sistem alanı.



Şekil 1-11 Tipik bir kişisel bilgisayardaki bazı G/Ç konumları.

- Çoğu G/Ç'ye erişim Aygıtlara yapılan çağrılar her zaman Windows, DOS veya BIOS işlev çağrıları aracılığıyla yapılmalıdır.
- Gösterilen harita Sistemdeki G/Ç alanını göstermek için bir kılavuz olarak sağlanmıştır .

- sistem aygıtları için ayrılmış olarak kabul edilir
- Genişleme için mevcut alan 0400H I/O portundan FFFFH'ye kadar uzanır . • Genel olarak, 0000H
- 00FFH ana kart bileşenlerini adresler; 0100H - 03FFH ise tak-çalıştır kartlarda veya ana kartta bulunan cihazları yönetir.

- 0000 ile 03FFH arasındaki G/Ç adreslerinin sınırlandırılması IBM'in PC standardı için belirlediği orijinal standartlardan kaynaklanmaktadır.

Bölüm Hedefleri

(devamı)

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

- İkili, ondalık ve onaltılık sayılar.
- Sayısal ve sayısal değerleri ayırt edin ve temsil edin tamsayı, kayan nokta, BCD ve ASCII verileri gibi alfabetik bilgiler .

Otobüsler

- Bir bilgisayar sistemindeki bileşenleri birbirine bağlayan ortak bir kablo grubu .
- Mikroişlemci, bellek ve G/Ç arasında adres, veri ve kontrol bilgilerini aktarın.
- Bu transfer için üç otobüs mevcuttur bilgi: adres, veri ve kontrol.
- Şekil 1-12 bu otobüslerin nasıl çalıştığını göstermektedir çeşitli sistem bileşenlerini birbirine bağlamak.

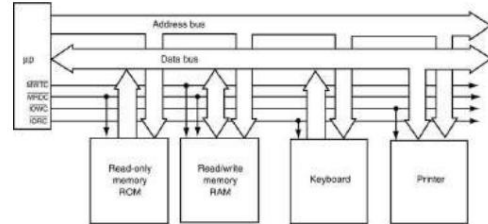
- Kontrol veri yolu hatları seçilir ve bellek veya G/Ç'nin okuma veya yazma işlemi gerçekleştirmesi sağlanır.
 - Çoğu bilgisayar sisteminde dört tane vardır kontrol veri yolu bağlantıları:
 - MRDC (bellek okuma kontrolü)
 - MWTC (bellek yazma denetimi)
 - IORC (G/Ç okuma kontrolü)
 - IOWC (G/Ç yazma denetimi). • üst
- çubuk, denetim sinyalinin etkin- düşük olduğunu gösterir; (kontrol satırında mantıksal sıfır görüldüğünde etkindir)



Mikroişlemci

- CPU (Merkezi İşlem Birimi) olarak adlandırılır.
- Bir bilgisayar sistemindeki kontrol elemanı. • Veri yolu adı verilen bağlantılar aracılığıyla belleği ve G/Ç'yi kontrol eder .
- veri yolları bir G/Ç veya bellek aygıtını seçer, G/Ç aygıtları veya bellek ile mikroişlemci arasında veri aktarımı yapar, G/Ç ve bellek sistemlerini kontrol eder
- Bellek ve G/Ç, mikroişlemci tarafından yürütülen, bellekte saklanan talimatlar aracılığıyla kontrol edilir .

Şekil 1-12 Adres, veri ve kontrol veri yolu yapısını gösteren bir bilgisayar sisteminin blok diyagramı .



- Mikroişlemci, adres veri yolu üzerinden belleğe bir adres göndererek bellek konumunu okur.
- Daha sonra, bir bellek okuma kontrol sinyali gönderir belleğin veri okumasını sağlar.
- Bellekten okunan veriler veri yolu aracılığıyla mikroişlemciye iletilir .
- Bir bellek yazma, G/Ç yazma veya G/Ç okuma gerçekleştiğinde aynı sıra izlenir.

CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

Mikroişlemci ve Bilgisayara Giriş

-2-

- Mikroişlemci üç ana görevi yerine getirir: – Kendisi ile bellek veya G/Ç sistemleri arasında veri aktarımı – Basit aritmetik ve mantık işlemleri – Basit kararlar yoluyla program akışı
- Mikroişlemcinin gücü, bellek sisteminde depolanan bir programdan veya yazılımdan (talimat grubu) saniyede milyarlarca milyonlarca talimatı yürütme yeteneğidir. – depolanan programlar mikroişlemciyi ve bilgisayar sistemini çok güçlü cihazlar haline getirir

- Adres veri yolu, bellekten bir bellek konumu veya G/Ç aygıtlarından bir G/Ç konumu ister. – G/Ç adreslenmişse, adres veri yolu 0000H'den FFFFH'ye kadar 16 bitlik bir G/Ç adresi içerir.
- eğer bellek adreslenirse, veri yolu , türüne göre genişliği değişen bir bellek adresi içerir mikroişlemci (8086 için 20 bit).
- Pentium'a yönelik 64-bit uzantılar 40 adres pini sağlayarak 1T bayta kadar belleğe erişime olanak tanır.

1–3 SAYI SİSTEMLERİ

- Bir mikroişlemcinin kullanımı, sayı sistemleri hakkında çalışma bilgisi gerektirir. – ikili, ondalık ve onaltılık
- Bu bölüm, bu numaralandırma sistemlerine ilişkin bir arka plan sağlar .
- Dönüşümler anlatılmaktadır.
 - ondalık ve ikili – ondalık ve onaltılık
 - ikili ve onaltılık

giriş

- Veriler hafızada nasıl saklanır?
- Sayısal veri gösterimleri
- Alfanoerik gösterimler

- Bir diğer güçlü özellik ise sayısal gerçeklere dayalı basit kararlar alma yeteneğidir . – bir mikroişlemci bir sayının sıfır, pozitif vb. olup olmadığına karar verebilir.

- Bu kararlar mikroişlemcinin program akışını değiştirmesine olanak tanır, böylece programlar bu basit kararları düşünüyormuş gibi görünür.

Basit aritmetik ve mantık işlemleri		8086'dan Core2 mikroişlemcilere kadar bulunan kararlar	
Operation	Comment	Decision	Comment
Addition		Zero	Test a number for zero or not-zero
Subtraction		Sign	Test a number for positive or negative
Multiplication		Carry	Test for a carry after addition or a borrow after subtraction
Division		Parity	Test a number for an even or an odd number of ones
AND	Logical multiplication	Overflow	Test for an overflow that indicates an invalid result after a signed addition or a signed subtraction
OR	Logical addition		
XOR	Logical inversion		
NOT	Arithmetic inversion		
SHL			
SHR			
Rotate			

- Veri yolu, mikroişlemci ile belleği ve G/Ç adres alanı arasında bilgi aktarımını sağlar.
- Veri transferleri çeşitli Intel mikroişlemcilerinde 8 bit genişlikten 64 bit genişliğe kadar değişir .
 - 8088, 8 bit veri aktarımı sağlayan 8 bitlik bir veri yoluna sahiptir bir seferde veri
 - 8086, 80286, 80386SL, 80386SX ve 80386EX 16 bit veri aktarır
 - 80386DX, 80486SX ve 80486DX, 32 bit
 - Pentium'dan Core2 mikroişlemcilere kadar 64 bit veri aktarımı

Rakamlar

- Sayıları tabanlar arasında dönüştürmeden önce sayı sisteminin rakamlarının anlaşılması gerekir.
- Herhangi bir numaralandırma sistemindeki ilk rakam her zaman sıfır.
- Ondalık (taban 10) sayı 0'dan 9'a kadar 10 rakamdan oluşur.
- 8 tabanında (sekizli) bir sayı; 8 basamaklı: 0'dan 7'ye.
- 2 tabanlı (ikili) bir sayı; 2 basamaklı: 0 ve 1.

- Taban 10'u aşarsa, ek rakamlar A ile başlayan alfabe harflerini kullanır. – 12 tabanlı bir sayı 11 rakamdan oluşur: 0'dan 9'a kadar, ardından 10 için A ve 11 için B gelir.
- 10 tabanlı bir sayının 10 basamak içermediğini unutmayın . – 8 tabanlı bir sayı 8 basamak içermez .
- Bilgisayarlarda kullanılan yaygın sistemler ondalık, ikili ve onaltılık (taban 16). – uzun yıllar önce sekizli sayılar popülerdi

- Ondalık sistemde, ondalık noktasının sağındaki konumlar negatif kuvvetlere sahiptir. – ondalık noktasının sağındaki ilk rakamın değeri 10^{-1} 'dir, veya 0.1.
- İkili sistemde, ikili noktanın sağındaki ilk rakamın değeri 2^{-1} 'dir, veya 0.5.
- Ondalık sayılar için geçerli olan ilkeler genel olarak diğer sistemlerdeki sayılar için de geçerlidir.
- İkili bir sayıyı ondalık sayıya dönüştürmek için, her basamağın ağırlıklarını toplayarak ondalık eşdeğerini oluşturun.

Tam Sayı Dönüşümü Ondalık

- Ondalık bir tam sayıyı başka bir tam sayıya dönüştürmek için sayı sisteminde, sayının tabanına bölünür ve kalanlar sonucun anlamlı basamakları olarak kaydedilir.
- Bu dönüşüm için bir algoritma:
 - ondalık sayıyı tabana bölün (sayı tabanı) – kalanı kaydet (ilk kalan en önemsiz basamaktır) – bölüm sıfır olana kadar 1. ve 2. adımları tekrarlayın

Ondalık Kesirden Dönüştürme

- Dönüşüm , sayının taban ile çarpılmasıyla gerçekleştirilir .
- Sonucun tam sayı kısmı, sonucun anlamlı bir basamağı olarak kaydedilir. – kesirli kalan tekrar taban ile çarpılır – kesirli kalan sıfır olduğunda, çarpma biter
- Bazı sayılar hiç bitmez (tekrarlanır). – sıfır asla bir kalan değildir

Pozisyonel Notasyon

- Rakamlar anlaşıldıktan sonra, konumsal gösterim kullanılarak daha büyük sayılar oluşturulur. – Birler basamağının solundaki konum onlar basamağıdır. konum
 - onların solunda yüzler basamağı vardır, vb.
- Bir örnek ondalık sayı 132'dir.
 - bu sayı 1 yüz, 3 on ve 2 birlik içerir
- Pozisyonların üstel kuvvetleri, diğer sistemlerdeki sayıları anlamak için kritik öneme sahiptir.

Ondalık Sayılara Dönüştürme Örnekleri

Power	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}
Weight	4	2	1	.5	.25	.125	.0625
Number	1	1	0	1	0	1	1
Numeric Value	$4 + 2 + 0 + .5 + 0 + .125 + .0625 = 6.625$						

Power	6^1	6^0	6^{-1}
Weight	6	1	.167
Number	2	5	2
Numeric Value	$12 + 5 + .333 = 17.333$		

Power	8^1	8^0	8^{-1}
Weight	64	8	.125
Number	1	2	5
Numeric Value	$64 + 16 + 5 + .875 = 85.875$		

Power	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}
Weight	16	8	4	2	1	.5	.25
Number	1	1	0	1	1	0	1
Numeric Value	$16 + 8 + 0 + 2 + 1 + 0 + .25 + .0625 = 27.4375$						

Power	16^1	16^0	16^{-1}
Weight	16	1	.0625
Number	6	A	C
Numeric Value	$96 + 10 + .75 = 106.75$		

- 10 ondalık sayıyı ikili sayıya dönüştürmek için 2'ye bölün. – sonuç 5'tir ve kalanı 0'dır • İlk kalan sonucun birim pozisyonudur. – bu örnekte 0 • Sonra 5'i 2'ye bölün; sonuç 2'dir ve kalanı 1'dir . – 1, ikinin (21) pozisyonunun değeridir
- Bölüm sıfır olana kadar bölme işlemine devam edilir. • Sonuç aşağıdan yukarıya doğru 10102 olarak yazılır.

- Ondalık kesirden dönüştürme algoritması :
 - ondalık kesri taban ile çarpın (sayı tabanı).
 - sonucun tam sayı kısmını kaydet (sıfır olsa bile) bir rakam olarak; ilk sonuç , taban noktasının hemen sağına yazılır – adım 2'nin kesirli kısmını kullanarak adım 2'nin kesirli kısmı sıfır olana kadar 1. ve 2. adımları tekrarlayın
- Aynı teknik, ondalık kesri herhangi bir sayı tabanına dönüştürür.

- Her pozisyonun üstel değeri: – birim pozisyonunun ağırlığı 100'dür , veya 1
 - onlar 101 ağırlığını konumlandırır , veya 10
 - Yüzler pozisyonunun ağırlığı 102'dir , veya 100
- Sayı tabanı noktasının solundaki konum her zaman sistemdeki birim konumudur. – yalnızca ondalık sistemde ondalık nokta olarak adlandırılır
 - ikili noktanın solundaki konum her zaman 20 , veya 1
 - sekizli noktanın solundaki konum 80'dir , veya 1
- Herhangi bir sayının sıfırıncı kuvveti her zaman bir (1) veya birim pozisyonudur.

Ondalık Sayıya Dönüştürme

- Herhangi bir sayı tabanını ondalık tabana dönüştürmek için sayının her bir basamağının ağırlıklarını veya değerlerini belirleyin .
- Ondalık eşdeğerini oluşturmak için ağırlıkları toplayın .

- 10 ondalık sayıyı 8 tabanına dönüştürmek için 8'e bölün. – 10 ondalık sayı 12 sekizlik sayıdır.
- Ondalıktan onaltılığa geçmek için 16'ya bölün.
 - kalanlar 0 ile 15 arasında bir değere sahip olacaktır – 10 ile 15 arasındaki herhangi bir kalan , onaltılık sayı için A'dan F'ye kadar olan harflere dönüştürülür
 - ondalık sayı 109, 6DH'ye dönüşür

Ondalık Kesirlerden Dönüştürme Örnekleri

.125	
x	2
0.25	digit is 0
.25	
x	2
0.5	digit is 0
.5	
x	2
1.0	digit is 1 result = 0.001 ₂

.125	
x	8
1.0	digit is 1 result = 0.1 ₈

.040975	
x	16
0.75	digit is 0
.75	
x	16
12.0	digit is 12(C) result = 0.0C ₁₆

- Birimler pozisyonunun solundaki pozisyon her zaman sayı tabanının birinci kuvvetine yükseltilir. – ondalık sistemde bu 101'dir , veya 10
 - ikili sistem, 21'dir , veya 2
 - 11 ondalık sayı, 11 ikili sayı sisteminden farklı bir değere sahiptir . • 11 ondalık sayı, 11 ikili sayı sisteminden farklı bir değere sahiptir. – 1 onluk sayı ve 1 birlikten oluşan ondalık sayı; 11 birimlik bir değer
 - ikili sayı 11, 1 iki artı 1 birimden oluşur : 3 ondalık birimlik bir değer
 - 11 sekizli sayının değeri 9 ondalık birimdir

Ondalıktan Dönüşüm

- Ondalık sayı sisteminden diğer sayı sistemlerine dönüşümlerin gerçekleştirilmesi daha zordur.
- Bir sayının tam kısmını ondalığa dönüştürmek için sayının tabanı 1'e bölünür.
- Kesirli kısmı dönüştürmek için sayının tabanı ile çarpın.

Ondalık Örneklerden Tam Sayı Dönüşümü

2) 10	remainder = 0	
2) 5	remainder = 1	
2) 2	remainder = 0	
2) 1	remainder = 1	result = 1010
0		

8) 10	remainder = 2	
8) 2	remainder = 1	result = 12
0		

16) 109	remainder = 13 (D)	
16) 9	remainder = 9	result = 6D
0		

İkili Kodlu Onaltılık

- İkili kodlanmış onaltılık (BCH), her rakamı 4 bitlik ikili bir sayı ile gösterilen onaltılık bir sayıdır.
- BCH kodu, bir BCH kodunun ikili bir sürümüne izin verir BCH ve heksadesimal arasında kolayca dönüştürülebilen bir biçimde yazılması gereken onaltılık sayı .
- Her rakam arasında boşluk olacak şekilde rakamların BCH koduna dönüştürülmesiyle temsil edilen onaltılık sayı sistemi .

Hexadecimal Digit	BCH Code
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

2AC = 0010 1010 1100

1000 0011 1101 . 1110 = 83D.E

ASCII Kodları

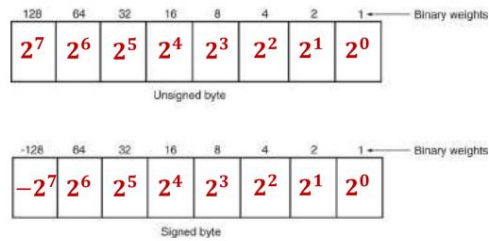
TABLE 1-8 ASCII code.

Second															
First	X0	X1	X2	X3	X4	X5	X6	X7	X8	X9	XA	XB	XC	XD	XE
0X	NUL	SOH	STX	ETX	EOT	ENO	ACK	BEL	BS	HT	LF	VT	FF	CR	SO
1X	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EMS	SUB	ESC	FS	GS	RS
2X	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.
3X	@	1	2	3	4	5	6	7	8	9	:	;	<	=	>
4X	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N
5X	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^
6X	-	a	b	c	d	e	f	g	h	i	j	k	l	m	n
7X	p	q	r	s	t	u	v	w	x	y	z	{		}	~

BCD (İkili Kodlu Ondalık) Veri

- BCD rakamının aralığı 00002'den başlar 10012'ye veya 0–9 ondalık sayıya kadar, iki biçimde saklanır:
- Paketlenmiş biçimde saklanır: – bayt başına iki basamak olarak saklanan paketlenmiş BCD verileri; – BCD'de toplama ve çıkarma işlemlerinde kullanılır mikroİşlemcinin talimat seti
- Paketlenmemiş biçimde depolanır: – paketlenmemiş BCD verileri bayt başına bir basamak olarak depolanır – bir tuş takımından veya klavyeden döndürüldü

Şekil 1-14 Her ikili bit konumunun ağırlıklarını gösteren imzasız ve imzalı baytlar.



Tamamlayıcılar

- Bazen veriler negatif sayıları temsil edecek şekilde tamamlayıcı biçimde saklanır.
 - Negatif verileri temsil etmek için kullanılan iki sistem: – kök – radix 1 tamamlayıcı (en erken - 1'in tamamlayıcısı)
- 1111 1111
- 0100 1100
1011 0011 (one's complement)
1011 0011 (two's complement)
- radix 1 tamamlayıcısı kök tamamlayıcısı

- PC'de, genişletilmiş ASCII karakter kümesi en soldaki bite 1 yerleştirilerek seçilir.
- Genişletilmiş ASCII karakter deposu: – bazı yabancı harfler ve noktalama işaretleri – Yunan ve matematiksel karakterler – kutu çizimi ve diğer özel karakterler
- ASCII kontrol karakterleri bir bilgisayar sisteminde kontrol işlevlerini gerçekleştirir. – ekranı temizleme, geri alma, satır besleme, vb.
- Klavyeden kontrol kodlarını girin. – Bir harf yazarken Kontrol tuşunu basılı tutun

Paketlenmiş ve Paketlenmemiş BCD

Decimal	Packed		Unpacked			
12	0001	0010	0000	0001	0000	0010
823	0000	0110	0010	0011	0000	0110
910	0000	1001	0001	0000	0000	1001

- İmzalanmamış tamsayılar 00H ile FFH (0–255) arasındadır
- -128'den 0'a ve + 127'ye kadar imzalı tam sayılar.
- Bu şekilde gösterilen negatif işaretli sayılar, **ikinin tamamlayıcısı** formunda saklanır .
- Her bit pozisyonunun ağırlıklarını kullanarak imzalanmış bir sayıyı değerlendirmek, bir sayının değerini bulmak için iki tamamlayıcının kullanılması eyleminden çok daha kolaydır. – özellikle programcılar için tasarlanmış hesap makineleri dünyasında doğrudur

1–4 BİLGİSAYAR VERİ FORMATLARI

- Başarılı programlama, veri formatlarının kesin olarak anlaşılmasını gerektirir .
- Veriler genellikle ASCII, Unicode, BCD, işaretli ve işaretersiz tam sayılar ve kayan nokta sayıları (reel sayılar) şeklinde görünür.
- Diğer formlar da mevcuttur ancak yaygın olarak bulunmazlar.

Genişletilmiş ASCII Kodları

TABLE 1-9 Extended ASCII code, as printed by the IBM ProPrinter.

First	Second															
	X0	X1	X2	X3	X4	X5	X6	X7	X8	X9	XA	XB	XC	XD	XE	XF
0X																
1X																
8X	Ç	ü	ë	â	ä	à	â	ç	ê	ë	è	é	î	í	î	Ë
9X	É	æ	Æ	ô	ö	û	ü	ÿ	Ö	Ü	¿	¼	½	¾	ƒ	„
AX	á	í	ó	ú	ñ	Ñ	ª	º	¸	¸	½	¼	¾	ƒ	„	„
BX																
CX																
DX																
EX	α	β	Γ	π	Σ	σ	μ	γ	Φ	Θ	Ω	δ	∞	φ	ε	η
FX	=	±	±	≤	≤	≤	÷	÷	÷	÷	÷	÷	÷	÷	÷	÷

- BCD verisi gerektiren uygulamalar, noktadan noktaya satış terminalleri. – ayrıca minimum miktarda işlem yapan cihazlar basit aritmetik • Bir sistem karmaşık aritmetik gerektiriyorsa, BCD verileri nadiren kullanılır. – karmaşık BCD aritmetiğini gerçekleştirmenin basit ve etkili bir yöntemi yoktur

Decimal	Packed		Unpacked							
12	0001	0010	0000	0001	0000	0010				
823	0000	0110	0010	0011	0000	0110	0000	0010	0000	0011
910	0000	1001	0001	0000	0000	1001	0000	0001	0000	0000

(+/-) Dönüşüm uygulaması

- Bir sayının iki ile tümlenmesi durumunda işareti negatiften pozitive veya pozitiften negatife değişir .
- Örneğin, 0000 1000 sayısı +8'dir. Negatif değer (-8), +8'in iki ile tamamlanmasıyla bulunur .
- 1. Bir'in sayıya tamamlayıcısı Bir sayının her bitini sıfırdan bire yirden sıfıra ters çevirin.
- 2. Bir'in tamamlayıcısına bir eklemek.

+ 8 = 00001000
11110111 (one's complement)
+ 1
- 8 = 11111000 (two's complement)

ASCII ve Amerikalı Verileri

- ASCII (Amerikan Standart Kodu) Bilgi Değişimi) verileri bilgisayar belleğindeki alfanümerik karakterleri temsil eder.
- Standart ASCII kodu 7 bitlik bir koddur. – Pariteyi tutmak için kullanılan sekizinci ve en önemli bit • Bir yazıcıyla kullanıldığında, alfanümerik yazdırma için en önemli bitler 0; grafikler için 1'dir.

- Birçok Windows tabanlı uygulama, Alfanümerik verileri depolamak için Unicode sistemi. – her karakteri 16 bitlik veri olarak depolar
- 0000H–00FFH kodları aynıdır standart ASCII kodu.
- Kalan kodlar, 0100H–FFFFH, tümünü saklayın birçok karakter setinden özel karakterler.
- Windows yazılımlarının dünyanın birçok ülkesinde kullanılabilemesini sağlar .
- Unicode hakkında daha fazla bilgi için şu adresi ziyaret edin: <http://www.unicode.org>

Bayt Boyutlu Veri

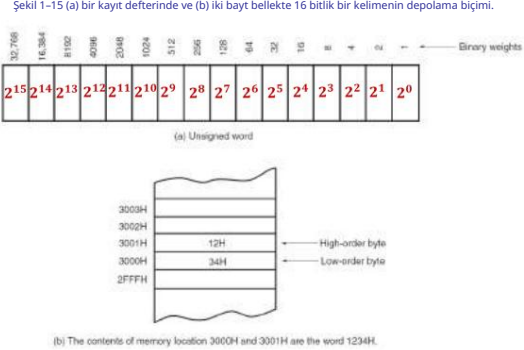
- İmzasız ve imzalı tam sayılar olarak saklanır .
- Bu formlardaki fark, en soldaki bit konumunun ağırlığıdır . – işaretsiz tamsayı için 128 değeri – işaretli tamsayı için eksi 128 • İşaretli tamsayı biçiminde, en soldaki bit sayının işaret bitini temsil eder. – ayrıca eksi 128 ağırlığı

(+/-) Daha basit dönüşüm uygulaması

- İkili sayı tamamlama tekniği için bir diğer ve muhtemelen daha basit teknik, en sağdaki rakamdan başlamakır.
- Öncelikle sayıyı sağdan sola doğru yazmaya başlayın.
- Sayıyı görüldüğü gibi yazın.
- ilk olan.
- Birincisini yaz, – sonra solundaki tüm bitleri ters çevir.

+8 = 00001000
1000 (write number to first 1)
1111 (invert the remaining bits)
-8 = 11111000

- Bir kelime (16 bit), iki baytlık bir kelimeden oluşur veri.
- En önemsiz bayt her zaman en düşük numaralı bellek konumunda saklanır.
- En önemli bayt en yüksekte saklanır. • Bir sayıyı saklamanın bu yöntemine küçük uçlu format denir .



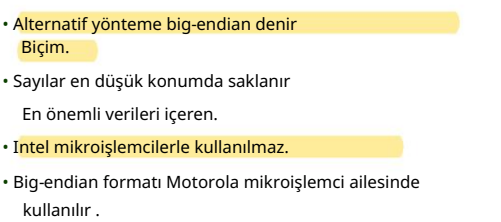
Gerçek Sayılar

- Birçok üst düzey dil Intel mikroişlemcilerini kullandığından , gerçek sayılarla sıklıkla karşılaşılır.

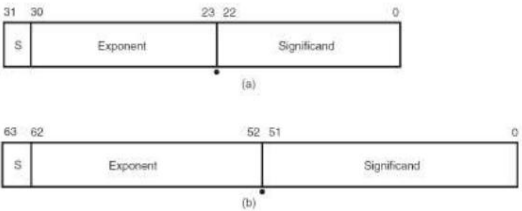
- Gerçek veya kayan noktalı sayılar iki kısımdan oluşur : - bir mantis,
- anlamlı sayı veya kesir - bir üs.

- 4 baytlık sayılara tek hassasiyetli sayılar denir .

- 8 baytlık biçime çift hassasiyet denir .



Şekil 1-17 (a) 7FH'lik bir önyargı kullanan tek hassasiyetli ve (b) 3FFH'lik bir önyargı kullanan çift hassasiyetli kayan nokta sayıları .



CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

Mikroişlemci ve Mimarlık

PROGRAMLAMA YAPISI

- En düşük seviyeli programlama dili
Bilgisayarların anlayabileceği tek dil olan Makine dili .

- Bilgisayarlar tarafından kolayca anlaşılabilmesine rağmen, Makine dillerinin insanlar tarafından kullanılması neredeyse imkansızdır çünkü tamamen sayılardan oluşurlar (örneğin 1'ler ve 0'lar).

Çift Kelime Boyutlu Veri

32 bitlik bir sayı olduğu için hafızanın bir parçasıdır. - çarpma işleminden sonra bir ürün olarak görünür - ayrıca bölme işleminden önce bir temettü olarak görünür

- Derleyici yönergesini kullanarak tanımlayın define
çift sözcük(ler) veya DD. – DD
yerine DWORD yönergesini de kullanın

Derleyici gerçekleri tanımlamak için kullanılabilir

tek ve çift hassasiyetli formlardaki sayılar: - tek hassasiyetli 32 bit

için DD yönergesini kullanın

sayılar

- 64 bitlik **çift hassasiyetli gerçek sayıları** tanımlamak için dördütlü kelimeyi veya DQ'yu tanımlayın - **İsteğe**

bağlı yönergeler REAL4, REAL8 ve REAL10'dur.

- tek, çift ve genişletilmiş tanımlamalar için hassas gerçek sayılar

giriş

Bu bölüm, mikroişlemciyi önce dahili programlama modeline ve ardından bellek alanının nasıl adreslendiğine bakarak programlanabilir bir cihaz olarak sunar. • Intel mikroişlemcilerinin mimarisi ve aile

Üyelerinin bellek sistemine adresleme yolları sunulur.

- Bu güçlü mikroişlemci ailesi için gerçek, korumalı ve düz çalışma modları için adresleme modları açıklanmaktadır.

PROGRAMLAMA YAPISI

- Bu nedenle programcılar ya yüksek bir
 - Bir montaj dili, aynı programlama dilini veya bir montaj dilini içerir .

Talimatlar makine dili olarak kullanılır, ancak talimatlar ve değişkenler sadece sayılardan oluşmak yerine isimlere sahiptir.

Machine Translated by Google

- Her CPU'nun kendine özgü benzersiz bir makine dili vardır . Programların farklı bilgisayar türlerinde çalışabilmesi için yeniden yazılması veya yeniden derlenmesi gerekir .
- Yüksek seviyeli diller ise platformdan bağımsızdır. Farklı bilgisayar tiplerinde çalışabilirler.

YAZILIM GELİŞTİRME AKIŞI

- Bağlayıcı , tarafından oluşturulan nesne dosyalarını birleştirir derleyiciyi tek bir yürütülebilir nesne modülüne dönüştürür. Yürütülebilir modülü oluştururken, semboller bellek konumlarına bağlar ve bu sembolere olan tüm referansları çözer. Ayrıca, önceki bir bağlayıcı çalışması tarafından oluşturulan kitaplık üyelerini ve çıktı modüllerini kabul eder.
- Çapraz referans listeleyicisi nesne dosyalarını kullanarak değişkenleri ve sembolleri, tanımlarını ve bellek konumlarını gösteren bir çapraz referans listesi oluşturur ve bağlantılı kaynak dosyalarındaki referansları.

Intel 8086'nın İçinde

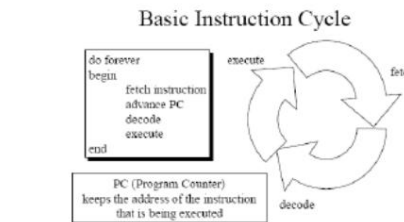
The diagram illustrates the internal architecture of the Intel 8086, which is divided into two main functional blocks: the **Base Unit** and the **Bus Interface Unit (BIU)**. The Base Unit contains the **Execution Unit** (including the ALU, registers, and control logic) and the **Instruction Decoder**. The BIU handles the external bus, including address and data buses, and manages the flow of instructions and data between the internal units and the external system.

- 8086 dahili olarak bölünmüştür iki ayrı fonksiyonel birime ayrılmıştır.

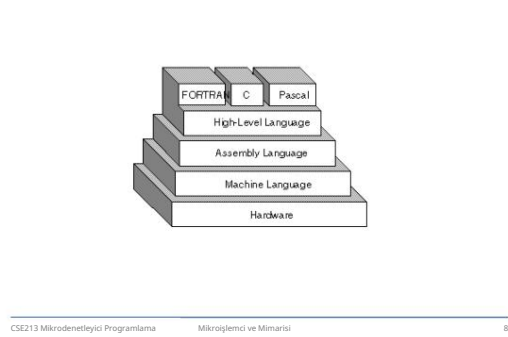
- Veriyolu Arayüz Birimi (BIU)
- Yürütme Birimi (AB).

- İşin iki işlemci arasında bölünmesi ve eş zamanlı olarak işlemesi (paralel işleme) kavramı, yürütmeyi hızlandırır.

Intel 8086'nın İçinde



PROGRAMLAMA YAPISI



Intel 8086'nın İçinde

- 8086 , 16 bitlik bir mikroişlemci çipidir 1978 yılında Intel tarafından tasarlanan ve x86 mimarisinin ortaya çıkmasına neden olan mimarı.
- 1979'da piyasaya sürülen Intel 8088, esasen aynı çipti, ancak harici bir 8 bit veri yoluna sahipti (bu sayede daha ucuz ve daha az sayıda destekleyen mantık çiplerinin kullanılmasına olanak sağlıyordu) ve orijinal IBM PC'de kullanılan işlemci olarak dikkat çekiyordu.

Intel 8086'nın İçinde

BUS ARAYÜZ ÜNİTESİ

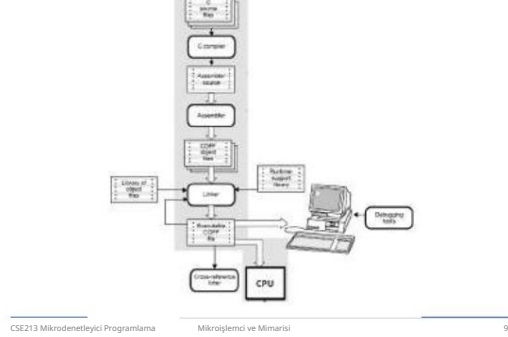
- BIU adresi gönderir, alır bellekten talimat, portlardan ve bellekten veri okur ve portlara ve belleğe veri yazar. Başka bir deyişle BIU, yürütme birimi için veri yollarındaki tüm veri ve adres transferlerini yönetir.

DAHİLİ MİKROİŞLEMÇİ

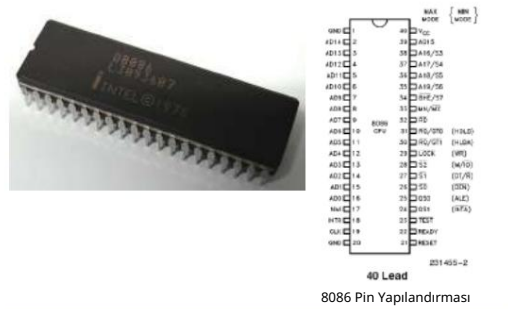
MİMARLIK

- Bir program yazılmadan veya bir talimat araştırılmadan önce, mikroişlemcinin dahili yapılandırması bilinmelidir. • Çok çekirdekli bir mikroişlemcide her çekirdek aynı programlama modelini içerir. • Her çekirdek aynı anda ayrı bir görevi veya iş parçacığını çalıştırır .

YAZILIM GELİŞTİRME AKIŞI



Intel 8086'nın İçinde



Intel 8086'nın İçinde

TALİMAT KUYRUĞU

- Programın yürütülmesini hızlandırmak için BIU, bellekten 6'ya kadar talimat baytı öncesini alır. Ön getirme talimatı baytları Talimat Kuyruğunda tutulur. BIU, EU bir talimatı kod çözerken veya yürütürken talimat baytlarını getirebilir , bu da veri yollarının kullanımını gerektirmez. • EU bir sonraki talimatı için hazır olduğunda, talimatı BIU'daki kuyruktan okur. Bu ön getirme ve Kuyruk şeması işlemeyi büyük ölçüde hızlandırır. Mevcut talimat yürütülürken bir sonraki talimatı getirmeye Boru Hattı denir.

Programlama Modeli

- 8086'dan Core2'ye kadar program görünür olarak kabul edildi.
 - programlama sırasında belirli kayıtlar kullanılır ve talimatlar tarafından belirtilir
- Diğer kayıtların program tarafından görünmez olduğu düşünülür .
 - uygulama programlaması sırasında doğrudan adreslenemez

YAZILIM GELİŞTİRME AKIŞI

- Aşağıdaki listede yazılım geliştirmede kullanılan örnek araçlar açıklanmaktadır.
- Derleyici (örneğin C, C++, C#) kaynak kodunu derleme dili kaynak koduna çevirir.
- Montajcı montaj dilini çevirir kaynak dosyaları makine diline (nesne dosyaları) dönüştürülür. Kaynak dosyaları talimatlara karşılık gelen numaralar içerir.

Intel 8086'nın İçinde

- 8086'nın tüm dahili kayıtları ve dahili ve harici veri yolları 16 bit genişliğindedir ve bu da gerçek bir "16 bit mikroişlemci" oluşturur.
- 8086'nın ilk versiyonunun saat hızı 5 MHz'di. Daha sonra 8 ve 10 MHz versiyonları da üretildi.
- 20 bitlik harici adres veri yolu 1 MB (parçalı) fiziksel adres alanı sağlar.
- 16 bitlik G/Ç adresleri 64 KB ayrı G/Ç sağlar uzay.

Intel 8086'nın İçinde

İCRA BİRİMİ

- 8086'nın AB'si BIU'ya şunu söylüyor:
 - Talimatların veya verilerin nereden alınacağını, – Talimatları çözer ve talimatları yürütür.
- AB, ikili sayıları toplayabilen, çıkarabilen, VE, VEYA, XOR işlemlerini yapabilen, artırabilen, azaltabilen, tamamlayabilen veya kaydırabilen 16 bitlik bir ALU içerir.

Şekil 2-1 64 bit uzantılar da dahil olmak üzere Core2 mikroşlemci üzerinden 8086'nın programlama modeli .



- RAX - 64 bitlik bir kayıt (RAX), 32 bitlik bir kayıt (akümülatör) (EAX), (8086 için) 16 bitlik bir kayıt (AX) veya iki 8 bitlik kayıttan biri (AH ve AL).

- Akümülatör çarpma, bölme ve bazı ayarlama komutları gibi komutlar için kullanılır.

- Intel, 4P (peta) baytlık belleği adreslemek için adres veri yolunu 52 bite çıkarmayı planlıyor.

- RFLAGS, aşağıdakilerin durumunu gösterir: mikroişlemci ve onun çalışmasını kontrol eder.

- Şekil 2-2, mikroişlemcinin tüm sürümlerinin bayrak kayıtlarını göstermektedir . • Bayraklar, 8086/8088'den Core2'ye kadar yukarı uyumludur.

- En sağdaki beş ve taşıma bayrağı çoğu aritmetik ve mantıksal işlem tarafından değiştirilir . – veri aktarımları bunları etkilemese de

- I (kesme), INTR (kesme isteği) giriş pininin çalışmasını kontrol eder.

- D (yön), DI ve/veya SI kayıtları için artış veya azalış modunu seçer .

- O (taşma), imzalı sayılar olduğunda meydana gelir eklenir veya çıkarılır.
– taşıma sonucu aşıldığını gösterir makinenin kapasitesi

GERÇEK MOD BELLEK ADRESLEME

- 80286 ve üzeri gerçek veya korumalı mod.

- Gerçek mod çalışması, adreslemeyi sağlar yalnızca ilk 1M bellek alanı baytı—Pentium 4 veya Core2 mikroişlemcilerinde bile. – ilk 1M bellek baytı gerçek bellek, geleneksel bellek veya DOS bellek sistemi olarak adlandırılır

- RBX , RBX, EBX, BX, BH, BL olarak adreslenebilir.

- BX kaydı (taban indeksi) bazen mikroişlemcinin tüm sürümlerinde bellek sistemindeki bir konunun ofset adresini tutar

- RCX, RCX, ECX, CX, CH veya CL olarak.

- çeşitli talimatların sayımını da tutan genel amaçlı bir (sayım) kayıt defteri

- RDX, RDX, EDX, DX, DH veya DL gibi – bir (veri)

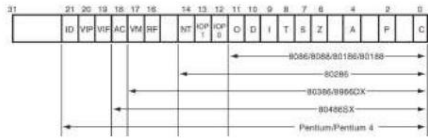
- genel amaçlı kayıt – bir çarpma işleminin

- sonucunun bir kısmını tutar

- veya bölünmeden önce temettütünün bir kısmı

CSE213 Mikrodeneleyici Programlama Mikroişlemci ve Mimari 24

Şekil 2-2 Tüm 8086 ve Pentium mikroişlemci ailesi için EFLAG ve FLAG kayıt sayıları .



- Bayraklar hiçbir veri transferi veya program kontrol işlemi için asla değişmez .

- Bazı bayraklar aynı zamanda mikroişlemcide bulunan özellikleri kontrol etmek için de kullanılır.

Segment Kayıtları

- Mikroişlemcideki diğer kayıtlarla birleştirildiğinde bellek adresleri oluşturur .

- Çeşitli dört veya altı segment kaydı mikroişlemcinin sürümleri (8086'da dört)

- Bir segment kaydı gerçek modda korumalı moddakinden farklı şekilde çalışır.

- Aşağıda her segment kaydının sistemdeki işleviyle birlikte listesi yer almaktadır.

Segmentler ve Ofsetler

- Tüm gerçek mod bellek adresleri bir segment adresi ve bir ofset adresinden oluşmalıdır . – segment adresi başlangıç adresini tanımlar herhangi bir 64K baytlık bellek segmentinin – ofset adresi, içindeki herhangi bir konumu seçer 64K bayt bellek segmenti

- Şekil 2-3, segment artı ofset adresleme şeması bir bellek konumu seçer.

- RBP, RBP, EBP veya BP olarak. – bir

- bellek (temel işaretçi) konumuna işaret eder bellek veri aktarımları için

- RDI , RDI, EDI veya DI olarak adreslenebilir.

- genellikle dize yönergeleri için dize hedef verilerini (hedef dizini) adresler

- RSI , RSI, ESI veya SI olarak kullanılır.

- (kaynak dizini) kaydı , dize talimatları için kaynak dize verilerini adresler – RDI gibi, RSI da genel

- amaçlı bir kayıt işlevi görür

Her Bayrak bitinin listesi

Fonksiyonun kısa bir açıklamasıyla.

- C (taşıma), toplamadan sonra taşımayı tutar veya çıkarma işleminden sonra ödünç almak.

- ayrıca hata koşullarını da gösterir

- P (parite), çift veya tek olarak ifade edilen bir sayıdaki birlerin sayısıdır . Tek parite için mantıksal 0; çift parite için mantıksal 1. – eğer bir sayı üç ikili bir bit içeriyorsa, tek pariteye sahiptir (örneğin, 1110 1010 P=0) – eğer bir sayı hiç bir bit içermiyorsa, çift pariteye sahiptir

parite (örneğin, 0000 0101 P=1)

- CS (kod) bölümü kodu (programları) tutar

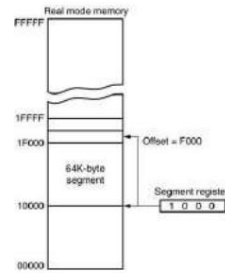
- ve mikroişlemci tarafından kullanılan prosedürler).

- DS (data), bir program tarafından kullanılan verilerin çoğunu içerir .

- Verilere bir ofset adresi veya ofseti tutan diğer kayıtların içerikleri adres

- ES (ekstra) Bazı talimatların hedef verileri tutmak için kullandığı ek bir veri segmenti .

Şekil 2-3 Bir segment adresi ve bir ofset kullanılarak gerçek mod bellek adresleme şeması .



- bu, şurada başlayan bir bellek segmentini gösterir: 10000H, 1FFFFH konumunda sona eriyor • 64K bayt uzunluğunda

- ayrıca bir ofset adresi, bir yer değiştirme olarak adlandırılır , F000H konumu seçer 1F000H bellekte

Özel Amaçlı Kayıtlar

- RIP, RSP ve RFLAGS'ı ekleyin

- segment kayıtları CS, DS, ES, SS, FS'yi içerir ve GS

- RIP, bir bölümdeki bir sonraki talimatı ele alır

- belleğin. –

- (talimat işaretçisi) bir kod parçası olarak tanımlanır

- RSP, yığın adı verilen bir bellek alanını adresler .

- (yığın işaretçisi) bu sayede veriyi depolar işaretçi

- Bir (yardımcı taşıma), toplamadan sonra taşımayı (yarım taşıma) veya çıkarmadan sonra ödünç almayı tutar

- Sonucun 3 ve 4 bit konumları arasında. • Z (sıfır), bir

- aritmetiğin sonucunun

- veya mantık işlemi sıfırdır.

- S (işaret) bayrağı, bir aritmetik veya mantık talimatı yürütüldükten sonra sonucun aritmetik işaretini tutar.

- T (tuzak) Tuzak bayrağı, yonga üzerindeki hata ayıklama özelliği aracılığıyla tuzağa düşürmeyi etkinleştirir.

- SS (yığın), yığın için kullanılan bellek alanını tanımlar.

- yığın giriş noktası, yığın segmenti ve yığın işaretçi kayıtları tarafından belirlenir – BP kaydı ayrıca yığın

- indeki verileri de adresler yığın segmenti

- Başlangıç adresi bilindiğinde, bitiş adresi FFFFH eklenerek bulunur. – çünkü belleğin gerçek mod segmenti 64K'dır uzunluk olarak

- Ofset adresi her zaman eklenir Verilerin bulunacağı segment başlangıç adresi.

- Segment ve ofset adresi bazen 1000:2000 olarak yazılır. – 1000H'lik bir segment adresi; 2000H'lik bir ofset – 12000H (1000H x 10H + 2000H) gerçek verim sağlar adres

Varsayılan Segment ve Ofset Kayıtları

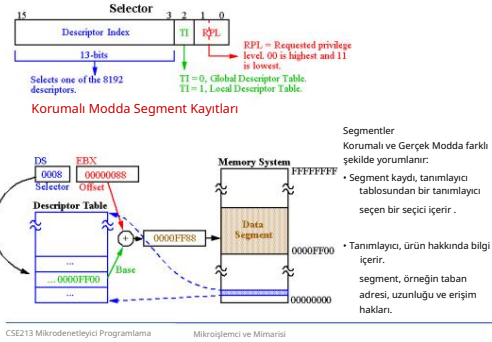
- Mikroişlemci, bellek adreslendiğinde segmentlere uygulanan kurallara sahiptir. – bunlar segment ve ofset kayıt kombinasyonunu tanımlar
- Kod segmenti kaydı, kod segmentinin başlangıcını tanımlar .
- Talimat işaretçisi, kod parçası içindeki bir sonraki talimatı bulur.

Segment ve Ofset Adresleme

Plan Yer Değiştirmeye İzin Veriyor

- Segment artı ofset adresleme, DOS programlarının bellekte yeniden konumlandırılmasına olanak tanır.
- **Taşınabilir bir program** , hafızanın herhangi bir alanına yerleştirilip, hiçbir **değişiklik** yapılmadan çalıştırılabilir.
- **Taşınabilir veriler** , belleğin herhangi bir alanına yerleştirilebilen ve programda herhangi bir **değişiklik** yapılmadan **kullanılabilen** verilerdir.

Korumalı Mod Bellek Adresleme



- Varsayılan kombinasyonlardan bir diğeri de yığın.
- yığın verilerine yığın aracılığıyla başvurulur yığın işaretçisi (SP/ESP) veya işaretçi tarafından adreslenen bellek konumundaki segment (BP/EBP)

Segment	Offset	Special Purpose
CS	IP	Instruction address
SS	SP or BP	Stack address
DS	BX, DI, SI, an 8- or 16-bit number	Data address
ES	DI for string instructions	String destination address

Varsayılan 16 bit segment ve ofset kombinasyonları.

- Çünkü hafıza bir Bir segment ofset adresi ile bellek segmenti, ofset adreslerinden hiçbirini değiştirmeden bellek sistemindeki herhangi bir yere taşınabilir.

- Programın adreslenmesi için yalnızca segment kaydının içeriği değiştirilmelidir yeni hafıza alanında.
- Windows programları , belleğin ilk 2 GB'ının kod ve veriler için kullanılabilir olduğu varsayılarak yazılır .

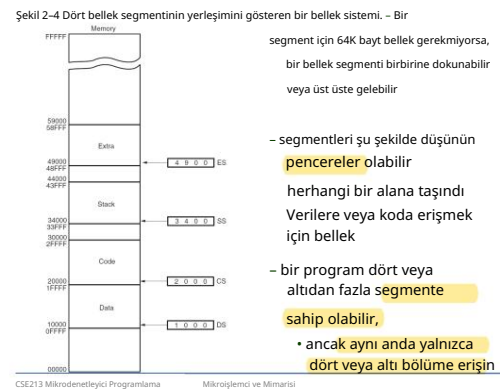
Düz Mod Belleği

- Düz modlu bellek sistemi, segmentasyon yoktur. – bellekteki bir konumu adreslemek için bir segment kaydı kullanmaz
- İlk bayt adresi 00 0000 0000H konumundadır; son konum ise FF FFFF FFFFH konumundadır. – adres 40 bittir
- Segment kaydı hala yazılımın ayrıcalık düzeyini seçmektedir.

CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

Adresleme Modları



KORUMALI MODA GİRİŞİ

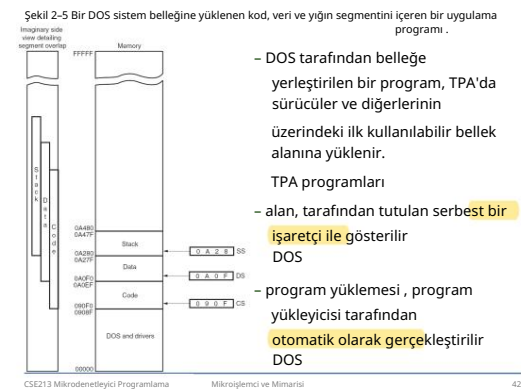
BELLEK ADRESLEME

- Belleğin ilk 1M baytının içinde ve üstünde bulunan verilere ve programlara erişime izin verir .
- **Korumalı mod**, Windows'un çalıştığı moddur.
- Bir segment adresi yerine, segment kaydı bir tanımlayıcı tablosundan bir tanımlayıcı seçen bir seçici içerir . • Tanımlayıcı , belleği tanımlar
- segmentin yeri, uzunluğu ve erişim hakları.

- İşlemci 64 bit modunda çalışıyorsa gerçek mod sistemi kullanılamaz .
- 64 bit modunda koruma ve sayfalamaya izin verilir .
- 64-bit modunda korumalı mod işleminde CS kaydı hala kullanılmaktadır.
- Günümüzde çoğu program IA32 uyumlu modda çalıştırılmaktadır. –
- Mevcut yazılım düzgün çalışmaktadır, ancak bellek dolduğunda bu durum birkaç yıl içinde değişecektir. daha büyük ve çoğu insanın 64 bit bilgisayarı var

giriş

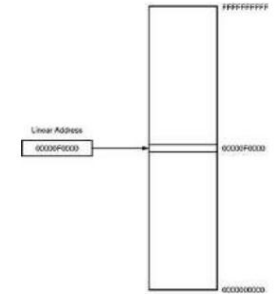
- Verimli yazılım geliştirme Mikroşlemci, her talimatın kullandığı adresleme modlarına tam olarak aşina olmayı gerektirir .
- Bu bölümde, PUSH ve POP yönergelerinin ve diğer yığın işlemlerinin yığın belleğinde depolanması için yığın belleğinin çalışması açıklanmaktadır. anlaşılacak.



Seçiciler ve Tanımlayıcılar

- Tanımlayıcı segmentte yer almaktadır kayıt & bellek segmentinin konumunu, uzunluğunu ve erişim haklarını açıklar . – 8192 tanımlayıcıdan birini seçer
- iki tanımlayıcı tablosunun
- Korunan modda, bu segment numarası sistemdeki herhangi bir bellek konumunu adresleyebilir kod bölümü için.
- Dolaylı olarak, kayıt hala bir bellek segmentini seçer, ancak gerçek modda olduğu gibi doğrudan seçmez.

Şekil 2-15 64 bit düz mod bellek modeli.



Bölüm Hedefleri

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

- Her veri adresleme işleminin nasıl çalıştığını açıklayın mod.
- Derleme dili ifadelerinin oluşturmak için veri adresleme modlarını kullanın.
- Her program belleği adresleme modunun çalışmasını açıklayın.
- Montaj ve makine dili ifadelerini oluşturmak için program belleği adresleme modlarını kullanın.

Bölüm Hedefleri

(devamı)

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

• Uygun adresleme modunu seçin

verilen bir görevi yerine getirmek.

• Verileri yığına yerleştiren veya yığından veri kaldıran olay dizisini tanımlayın.

• Bir veri yapısının belleğe nasıl yerleştirildiğini ve yazılımla nasıl kullanıldığını açıklayın.

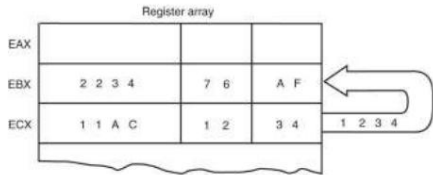
• Şekil 3-2, MOV kullanılarak veri adresleme modlarının tüm olası varyasyonlarını göstermektedir.

• Bu veri adresleme modları Intel mikroişlemcisinin tüm sürümlerinde bulunur . – yalnızca 80386'dan Core2'ye kadar bulunan ölçeklenmiş dizin adresleme modu hariç • RIP görelî adresleme modu

gösterilmemiştir.

– yalnızca Pentium 4 ve Core2'de mevcuttur
64 bit modunda

Şekil 3-3 BX kaydının değişmesinden hemen önceki noktada MOV BX, CX talimatının yürütülmesinin etkisi . EBX kaydının yalnızca en sağdaki 16 bitinin değiştiğine dikkat edin.



• Sembolik montaj dilinde, # sembolü

(sayı veya diyez işareti) bazı derleyicilerde anlık verilerden önce gelir.

– MOV AX,#3456H talimatı bir örnektir • Çoğu derleyici #

sembolünü kullanmaz,

ancak MOV AX,3456H talimatında olduğu gibi anında verileri temsil eder. – bazı Hewlett-Packard'larla kullanılan eski bir derleyici

Packard mantık geliştirme, diğerleri gibi bunu da yapabilir - bu metinde, # anlık veriler için kullanılmaz

Adresleme Modu Türleri

1. Veri Adresleme Modları

2. Program Bellek Adresleme Modları

3. Yığın Bellek Adresleme Modları

Şekil 3-2 8086-Core2 veri adresleme modları.

Type	Instruction	Source	Address Generation	Destination
Register	MOV AX,BX	Register BX		Register AX
Immediate	MOV CH,3AH	Data 3AH		Register CH
Direct	MOV [1234H],AX	Register AX	$DS \times 16H + DISP$ $10000H + 1234H$	Memory Address 1234H
Register indirect	MOV [BX],CL	Register CL	$DS \times 16H + BX$ $10000H + 0000H$	Memory Address 10000H
Base-plus-index	MOV [BX+SI],BP	Register BP	$DS \times 16H + BX + SI$ $10000H + 0000H + 0000H$	Memory Address 10000H
Register relative	MOV CL,[BX+4]	Memory Address 10004H	$DS \times 16H + BX + 4$ $10000H + 0000H + 4$	Register CL
Base relative-plus-index	MOV ARRAY[BX+SI],DX	Register DX	$DS \times 16H + ARRAY + BX + SI$ $10000H + 1000H + 0000H + 0000H$	Memory Address 10000H
Scaled index	MOV [EBX*2+ESI],AX	Register AX	$DS \times 16H + EBX \times 2 + ESI$ $10000H + 00003000H + 00000400H$	Memory Address 10000H

Note: EBX = 00003000H, ESI = 00000200H, ARRAY = 1000H, and DS = 1000H

• Şekil 3-3, MOV BX, CX komutunun çalışmasını göstermektedir.

• Kaynak kayıt içeriği değişmez. – hedef kayıt içeriği değişir . • CMP ve

TEST talimatları haricindeki tüm talimatlar için hedef kayıt

veya hedef bellek konumunun içeriği değişir.

• MOV BX, CX komutu EBX kayıt biriminin en soldaki 16 bitini etkilemez .

– MOV BL, CL komutu BX kayıtcısının en soldaki 8 bitini etkilemez.

• Sembolik derleyici anlık verileri birçok şekilde tasvir eder. • H harfi

onaltılık verileri ekler.

• Onaltılık veriler bir harfle başlıyorsa, derleyici verilerin 0 ile başlamasını gerektirir. – Onaltılık F2'yi temsil etmek için 0F2H kullanılır montaj dilinde

• Ondalık veriler olduğu gibi gösterilir ve özel kodlar veya ayarlamalar gerektirmez. – bir örnek, 100 ondalık basamağıdır.

MOV AL,100 talimatı

3-1 VERİ ADRESLEME MODLARI

• MOV talimatı yaygın ve esnek bir talimat. – veri

adresleme modlarının açıklanması için bir temel sağlar

• Şekil 3-1, MOV

talimatını gösterir ve veri akışının yönünü tanımlar.

• Kaynak sağda, hedef soldadır,

opcode MOV'un yanında. – bir opcode

veya işlem kodu,

mikroişlemci hangi işlemi gerçekleştirecek

Kayıt Adresleme

• Veri adreslemenin en yaygın biçimidir. – bir kez kayıt adları öğrenildiğinde, uygulanması en kolay olanıdır.

• Mikroişlemci bu 8 bitlik işlemcileri içerir

kayıt adreslemesiyle kullanılan kayıt adları: AH, AL, BH, BL, CH, CL, DH ve DL. • 16 bitlik kayıt adları: AX, BX,

CX, DX, SP, BP, SI ve DI.



Hemen Adresleme

• "Hemen" terimi , verilerin bellekteki onaltılık işlem kodunu hemen takip ettiğini ifade eder.

– anlık veriler sürekli verilerdir

– bir kayıt defterinden veya bellekten aktarılan veriler konum değişken verilerdir

• Anında adresleme, bir bayt veya veri sözcüğü üzerinde çalışır.

• Şekil 3-4 bir MOV'un çalışmasını göstermektedir EAX,13456H talimatı.



• ASCII verileri tırnak işaretleri altına alınır, ASCII kodlu bir karakter veya karakterler doğrudan biçimde gösterilebilir. – ASCII için tırnak işareti (") kullanmaya dikkat edin

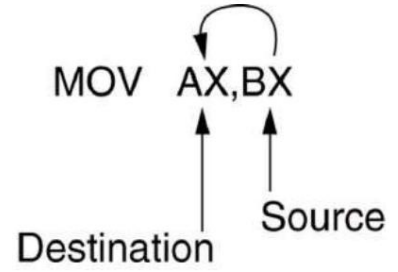
veri ve tek tırnak işareti (') değil

• İkili veriler, ikili sayının ardından B harfi geldiğinde gösterilir.

– bazı derleyicilerde Y harfi

– ondalık sayıyı temsil etmek için 10, 00001010B kullanılır montaj dilinde

Şekil 3-1 Veri akışının kaynağını, hedefini ve yönünü gösteren MOV talimatı.



• 80386 ve üzeri sürümlerde, genişletilmiş 32 bit kayıt adları şunlardır: EAX, EBX, ECX, EDX, ESP, EBP, EDI ve ESI. • 64 bit mod kayıt

adları şunlardır: RAX, RBX, RCX, RDX, RSP, RBP, RDI, RSI ve R8'den R15'e.

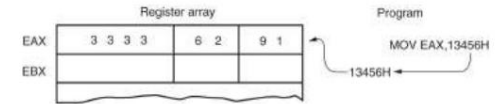
• Aynı boyuttaki kayıtları kullanma talimatları için önemlidir.

– 8 bitlik bir kaydı asla 16 bitlik bir kayıtlı, 8 veya

32 bitlik bir kayıtlı 16 bitlik kayıt

– bu mikroişlemci tarafından izin verilmez ve birleştirildiğinde hataya neden olur

Şekil 3-4 MOV EAX,13456H talimatının çalışması. Bu talimat, anlık verileri (13456H) EAX'a kopyalar.



• MOV talimatında gösterildiği gibi

Şekil 3-3, kaynak veriler hedef verilerin üzerine yazılır.

• Bir assembly dili programındaki her ifade dört bölümden veya alandan oluşur.

• En soldaki alana etiket adı verilir .

– temsil ettiği bellek konumu için sembolik bir ad depolamak için kullanılır

• Tüm etiketler bir harfle veya aşağıdaki özel karakterlerden biriyle başlamalıdır: @, \$, -, veya ?. – Bir etiket 1 ila 35 karakter arasında herhangi bir uzunlukta olabilir • Etiket, bir programda verileri depolamak ve diğer amaçlar için bir bellek konumunun adını tanımlamak amacıyla görünür.

Machine Translated by Google

Sağdaki bir sonraki alan opcode alanıdır. – **talimatı** veya opcode'u tutmak **için tasarlanmıştır** – veri taşıma talimatının MOV kısmı bir opcode örneğidir • Opcode alanının sağında operand

alanı bulunur. – opcode tarafından kullanılan **bilgileri içerir**

– MOV AL,BL talimatı MOV opcode'una sahiptir ve **AL ve BL işlenenleri**

• Yorum alanı , son alan, talimat(lar) hakkında bir yorum içerir. – yorumlar her zaman noktalı virgülle (;) başlar.

Örnek Kod

```
name "add-sub"
org 100h

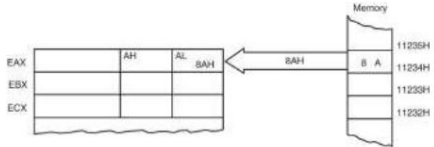
mov al, 5          ; bin-00000101h
mov bl, 10         ; hex-0ah or bin-00001010h
; 5 + 10 = 15 (decimal) or hex-0fh or bin-00001111h
add bl, al
; 15 - 1 = 14 (decimal) or hex-0eh or bin-00001110h
sub bl, 1

; print result in binary:
mov cx, 8
print: mov ah, 2    ; print function.
        mov dl, 'a' ; test first bit.
        test bl, 10000000h ; test first bit.
        js zero
        mov dl, 'i'
        int 21h
        shl bl, 1
        loop print
; print binary suffix:
mov dl, 'h'
int 21h

; wait for any key press:
mov ah, 0
int 16h
ret
```

CSE213 Mikrodeneleyici Programlama Adresleme Modları 20

Şekil 3-5 DS=1000H olduğunda MOV AL[1234H] konumunun çalışması.



• Bu talimat, 11234H bellek konumunun bir kopyasını AL'ye aktarır. – etkin adres, eklenerek oluşturulur.

1234H (ofset adresi) ve 1000H (1000H kez veri segmenti adresi) 10H) gerçek modda çalışan bir sistemde

CSE213 Mikrodeneleyici Programlama Adresleme Modları 24

• Veri segmenti varsayılan olarak kullanılır dolaylı adreslemeyi veya belleği adreslemek için BX, DI veya SI kullanan herhangi bir diğer modu kaydedin.

• BP kaydı belleğe adres veriyorsa, yığın segmenti varsayılan olarak kullanılır. – bu ayarlar varsayılan olarak kabul edilir bu dört endeks ve taban kaydı

• 80386 ve üzeri için EBP, varsayılan olarak yığın segmentindeki belleği adresler.

• EAX, EBX, ECX, EDX, EDI ve ESI varsayılan olarak veri segmentindeki belleği adresler.

CSE213 Mikrodeneleyici Programlama Adresleme Modları 28

Taban-Artı-Endeks Adresleme

• Bellek verilerini dolaylı olarak adreslediği için dolaylı adreslemeye benzer . • Temel kayıt genellikle

bir bellek dizisinin başlangıç konumunu tutar. – dizin kaydı göreceli konumu tutar

dizideki bir öğenin

– BP bellek verilerini adreslediğinde, hem yığın segment kaydı hem de BP, etkili adres

Base-plus-index MOV [BX+SI],BP Register BX DS = 1000H + BX + SI 10000H + 0000H + 0000H Memory address 10000H

CSE213 Mikrodeneleyici Programlama Adresleme Modları 32

Yerinden Edilmenin Ele Alınması

• Doğrudan adreslemeye neredeyse aynıdır, ancak talimat 3 yerine 4 bayt genişliğindedir. • 80386'dan Pentium 4'e

kadar, 32 bitlik bir kayıt ve 32 bitlik bir yer değiştirme belirtilirse bu talimat 7 bayta kadar geniş olabilir. • Bu tür doğrudan veri adreslemesi çok daha fazladır.

daha esnektr çünkü çoğu talimat bunu kullanır.

CSE213 Mikrodeneleyici Programlama Adresleme Modları 25

• Bazı durumlarda, dolaylı adresleme, verilerin boyutunun özel birleştirici yönergesi BYTE PTR, WORD PTR, DWORD PTR veya QWORD PTR ile belirtilmesini gerektirir. – bu yönergeler, bellek işaretçisi (PTR) tarafından adreslenen bellek verilerinin boyutunu belirtir .

• Yönergeler, talimatlarla birlikte gelir. bir bellek konumunu bir adres aracılığıyla ele almak anında veri içeren işaretçi veya dizin kaydı.

• SIMD talimatlarında, sekizli OWORD PTR, 128 bit genişliğinde bir sayıyı temsil eder.

CSE213 Mikrodeneleyici Programlama Adresleme Modları 29

Base-Plus-Index ile Veri Bulma Adresleme

• Şekil 3–8, mikroişlemci gerçek modda çalışırken MOV DX,[BX + DI] talimatı tarafından verilerin nasıl adreslendiğini gösterir . • Intel derleyicisi, bu adresleme modunun [BX + DI] yerine [BX][DI] olarak görünmesini gerektirir.

• MOV DX,[BX + DI] talimatı MOV'dur Intel ASM derleyicisi için yazılmış bir program için DX,[BX][DI].

CSE213 Mikrodeneleyici Programlama Adresleme Modları 33

Doğrudan Veri Adresleme

• Tipik bir programdaki birçok talimata uygulanır .

• Doğrudan veri adreslemenin iki temel biçimi: – MOV'a uygulanan doğrudan adresleme bir bellek konumu ile AL, AX veya EAX arasında – neredeyse her şeye uygulanan yer değiştirme adreslemesi talimat setindeki herhangi bir talimat

• Adres , varsayılan veri segmenti adresine veya alternatif bir segment adresine yer değiştirme eklenerek oluşturulur .

CSE213 Mikrodeneleyici Programlama Adresleme Modları 22

Dolaylı Adreslemeyi Kaydet

• Verilerin herhangi bir bellekte adreslenmesine izin verir Aşağıdaki kayıtların herhangi birinde tutulan bir ofset adresi aracılığıyla konum : BP, BX, DI ve SI. • Ayrıca, 80386 ve üzeri kayıtlara izin verir ESP hariç herhangi bir genişletilmiş kayıtlı dolaylı adresleme. • 64 bit

modunda, segment kayıtları bir konumu adreslemede hiçbir amaca hizmet etmez düz modelde.

Register indirect MOV [BX],CL Register CL DS = 100H + BX 10000H + 0000H Memory address 10000H

CSE213 Mikrodeneleyici Programlama Adresleme Modları 26

• Dolaylı adresleme, bir programın genellikle bellekte bulunan tablo verilerine başvurmasına olanak tanır.

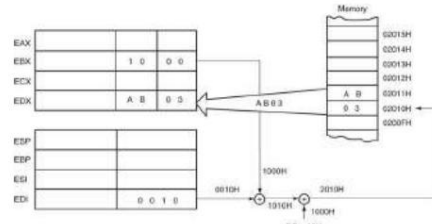
• Şekil 3–7 tabloyu ve BX'i göstermektedir Tablodaki her bir lokasyonu sıralı olarak adreslemek için kullanılan kayıt.

• Bu görevi gerçekleştirmek için tablonun başlangıç konumunu BX kaydına yükleyin MOV anında talimatı ile.

• Tablonun başlangıç adresini başlattıktan sonra , kayıt dolaylı adreslemesini kullanın 50 örneği sırayla saklayın.

CSE213 Mikrodeneleyici Programlama Adresleme Modları 30

Şekil 3-8 MOV DX,[BX + DI] talimatı için taban artı dizin adresleme modunun nasıl çalıştığını gösteren bir örnek . DS=0100H, BX=100H ve DI=0010H olduğu için bellek adresi 02010H'ye erişildiğine dikkat edin .



CSE213 Mikrodeneleyici Programlama Adresleme Modları 34

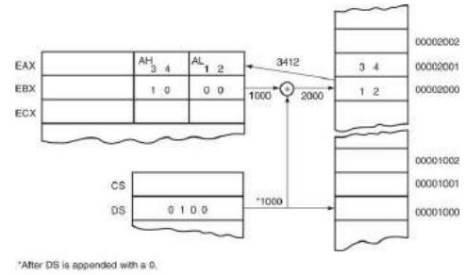
Doğrudan Hitap

• MOV talimatıyla doğrudan adresleme veri segmentinde bulunan bir bellek konumu ile AL (8 bit), AX (16 bit) veya EAX (32 bit) kaydı arasında veri aktarımı yapar . – genellikle 3 bayt uzunluğunda bir talimattır • MOV AL,DATA, AL'yi verilerden yükler segment bellek konumu VERİ (1234H). – DATA sembolik bir bellek konumudur, oysa 1234H gerçek onaltılık konumdur

Direct MOV [1234H],AX Register AX DS = 100H + DISP 10000H + 1234H Memory address 11234H

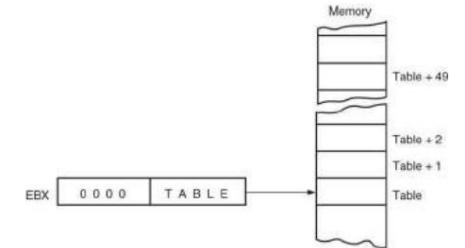
CSE213 Mikrodeneleyici Programlama Adresleme Modları 23

Şekil 3-6 BX = 1000H ve DS = 0100H olduğunda MOV AX,[BX] konumunun çalışması. Bu talimatın, hafızanın içeriği AX'e aktarıldıktan sonra gösterildiğine dikkat edin.



CSE213 Mikrodeneleyici Programlama Adresleme Modları 27

Şekil 3-7 Kayıt BX aracılığıyla dolaylı olarak adreslenen 50 bayt içeren bir dizi (TABLO) .



CSE213 Mikrodeneleyici Programlama Adresleme Modları 31

Base-Plus Kullanarak Dizi Verilerini Bulma Dizin Adresleme

• Başlıca kullanım, bir öğeyi ele almaktır bellek dizisi.

• Bunu başarmak için BX kaydını yükleyin (taban) dizinin başlangıç adresi ve erişilecek eleman numarası olan DI kayıt defteri (indeks).

• Şekil 3–9, bir veri dizisindeki bir öğeye erişmek için BX ve DI kullanımını göstermektedir.

CSE213 Mikrodeneleyici Programlama Adresleme Modları 35

Göreceli Adreslemeyi Kaydet

- Taban artı dizin adresleme ve yer değiştirme adreslemeye benzer . – belleğin bir bölümündeki veriler, aşağıdaki şekilde adreslenir:
Yer değiştirmenin bir taban veya dizin kaydının (BP, BX, DI veya SI) içeriğine eklenmesi • Şekil 3-10, MOV AX,[BX+1000H] talimatının çalışmasını gösterir.

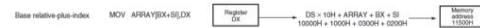
- Gerçek mod segmenti 64K bayt uzunluğundadır.



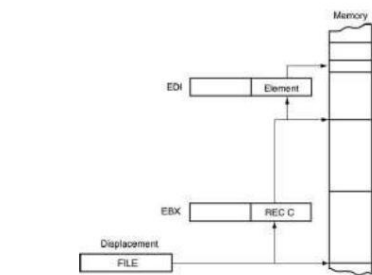
Şekil 3-11 ARRAY ögesini adreslemek için kullanılan kayıt bağlı adreslemesi. Yer değiştirme ARRAY'in başlangıcını adresler ve DI bir öğeye erişir.

Temel Göreceli-Artı-Endeks Adresleme

- Taban artı dizin adreslemesine benzer.
 - bir yer değiştirme ekler –
 - bir taban kaydı ve bir dizin kaydı kullanır bellek adresini oluştur
- Bu tür adresleme modu genellikle iki boyutlu bir bellek verisi dizisini adresler.



Şekil 3-13 Birden fazla kaydı içeren bir DOSYAYA (REC) erişmek için kullanılan temel göreceli artı dizin adreslemesi .



Şekil 3-14 Bir JMP [1000H] talimatının 5 baytlık makine dli sürümü.



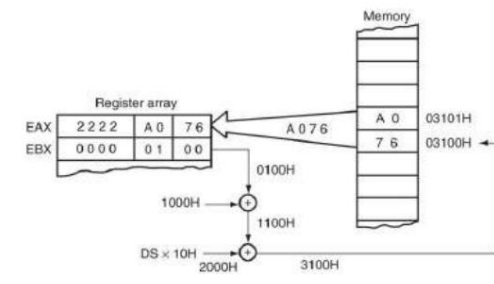
Dizileri Taban Göreceli Olarak Adresleme Artı Endeksi

- Birçok kayıttan oluşan bir dosyanın mevcut olduğunu varsayalım bellek, her kayıt birçok öğeye sahiptir. – yer değiştirme dosyayı adresler, temel kayıt bir kaydı adresler, dizin kaydı bir kaydın bir ögesini adresler
- Şekil 3-13 bu oldukça karmaşık adresleme biçimini göstermektedir.

Doğrudan Program Belleği Adresleme

- Erken dönemdeki tüm atlamalar ve çağrılar için kullanılır mikroişlemci; BASIC gibi üst seviye dillerde de kullanılır .
 - GOTO ve GOSUB talimatları
- Mikroişlemci bu formu kullanır, ancak bağıl ve dolaylı program belleği adreslemesi kadar sık kullanmaz.
- Doğrudan program belleğine yönelik talimatlar adresleme, adresi opcode ile birlikte depolar.

Şekil 3-10 BX=0100H olduğunda MOV AX, [BX+1000H] komutunun çalışması ve DS=0200H.



Verilerin Taban Göreliliği ile Adreslenmesi Artı Endeksi

- En az kullanılan adresleme modu. • Şekil 3-12, mikroişlemci tarafından yürütülen talimat MOV AX,[BX + SI + 100H] ise verilerin nasıl referans alındığını gösterir. – 100H'lik yer değiştirme, veri segmenti içinde offset adresini oluşturmak için BX ve SI'ya eklenir. • Bu adresleme modu, programlamada sık kullanım için çok karmaşıktır.

Veri Yapıları

- Bilgilerin bir bellek dizisinde nasıl depolandığını belirtmek için kullanılır . – veriler için bir şablon
- Bir yapının başlangıcı, STRUC assembly dili direktifi ve ENDS ifadesi ile sonlandırılır.

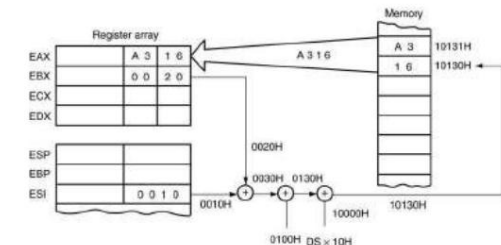
- Bu JMP talimatı CS'yi 1000H ile yükler ve bir sonraki talimat için 10000H bellek konumuna atlamak için 0000H ile IP.
 - segmentler arası atlama , tüm bellek sistemi içindeki herhangi bir bellek konumuna yapılan bir atlamadır
- Genellikle uzak atlama olarak adlandırılır çünkü bir sonraki talimat için herhangi bir bellek konumuna atlayabilir. – gerçek modda, ilk 1M bayt içindeki herhangi bir konum – Korumalı mod işleminde, uzak atlama 4G bayt adresindeki herhangi bir konuma atlayabilir. 80386 - Core2 mikroişlemcilerindeki aralık

Dizi Verilerini Kayıt ile Adresleme Arkaba

- Dizi verilerine kayıt göreceli adresleme ile hitap etmek mümkündür . – taban artı dizin adreslemesi gibi • Şekil 3-11'de, kayıt göreceli adresleme taban artı dizin adreslemesi için kullanılan örnekle gösterilmiştir . – bu, yer değiştirme DİZİNİN nasıl eklendiğini gösterir

DI kaydını indekslemek için bir referans oluşturmak bir dizi elemanı

Şekil 3-12 MOV AX,[BX+SI+100H] talimatı kullanılarak taban bağıl artı dizin adreslemesinin bir örneği . Not: DS=1000H



3-2 PROGRAM BELLEK ADRESLEMESİ MODLAR

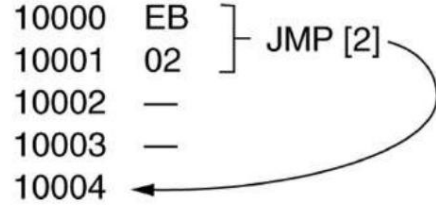
- JMP (atlama) ve CALL ile kullanılır talimatlar.
- Üç farklı biçimden oluşur: – doğrudan, bağıl ve dolaylı

- Doğrudan kullanan tek diğer talimat Program adreslemesi segmentler arası veya uzak ÇAĞRI talimatıdır.
- Genellikle etiket adı verilen bir bellek adresinin adı , gerçek sayısal adres yerine çağrılan veya atlanan konumu ifade eder.
- CALL veya JMP ile bir etiket kullanırken Talimat, çoğu derleyicinin program adreslemesinin en iyi biçimini seçmesidir .

Bağılı Program Bellek Adreslemesi

- Tüm erken mikroişlemcilerde mevcut değildir, ancak bu mikroişlemci ailesinde mevcuttur.
- Göreceli terimi "göreceli" anlamına gelir talimat işaretçisi (IP)".
- JMP talimatı, talimat işaretçisine eklenen 1 baytlık veya 2 baytlık bir yer değiştirmeye sahip 1 baytlık bir talimattır .
 - Şekil 3-15'te bir örnek gösterilmiştir.

Şekil 3-15 Bir JMP [2] talimatı. Bu talimat, takip eden 2 baytlık belleği atlar JMP talimatı.



Dolaylı Program Bellek Adreslemesi

- Mikroişlemci, JMP ve CALL talimatları için çeşitli program dolaylı bellek adresleme biçimlerine izin verir.
- 80386 ve üzeri sürümlerde, genişletilmiş bir kayıt, bağlı bir JMP veya CALL'ın adresini veya dolaylı adresini tutmak için kullanılabilir. – örneğin, JMP EAX,

EAX kaydına göre konum adresi

- Veriler PUSH ile yığına yerleştirilir talimat; POP talimatıyla kaldırıldı .
- Yığın belleği iki kayıt tarafından tutulur: – yığın işaretçisi (SP veya ESP) – yığın segment kaydı (SS)
- Yığına bir veri sözcüğü eklendiğinde, yüksek dereceli 8 bit SP – 1 tarafından adreslenen yere yerleştirilir. – düşük dereceli 8 bit SP – 2 tarafından adreslenen yere yerleştirilir.

- Eğer bağlı bir kayıt adresi tutuyorsa, atlama dolaylı bir atlama olarak kabul edilir. • Örneğin, JMP [BX], BX'te bulunan ofset adresindeki veri segmentindeki bellek konumunu ifade eder.

– bu ofset adresinde, segment içi atlamada ofset adresi olarak kullanılan 16 bitlik bir sayı bulunur – bu tür atlamalara bazen dolaylı - dolaylı veya çift-dolaylı atlama denir

- Şekil 3-16, TABLE bellek konumundan başlayarak depolanan bir atlama tablosunu göstermektedir.

- SP 2 azaltılır, böylece bir sonraki sözcük bir sonraki kullanılabilir yığın konumuna kaydedilir.
 - SP/ESP kaydı her zaman yığın segmentinde bulunan bir bellek alanına işaret eder. • Veriler yığından çıkarıldığında, düşük sıralı 8 bitler SP tarafından adreslenen konumdan kaldırılır.

- Daha sonra, yüksek mertebeli 8 bit kaldırılır; • Ve, SP kaydı 2 artırılır

CSE213 Mikrodenetleyici Programlama Adresleme Modları 52

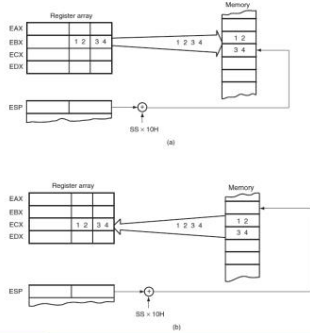
Assembly Language	Operation
JMP AX	Jumps to the current code segment location addressed by the contents of AX.
JMP CX	Jumps to the current code segment location addressed by the contents of CX.
JMP NEAR PTR[EX]	Jumps to the current code segment location addressed by the contents of the data segment location addressed by EX.
JMP NEAR PTR[DI+2]	Jumps to the current code segment location addressed by the contents of the data segment memory location addressed by DI plus 2.
JMP TABLE[BX]	Jumps to the current code segment location addressed by the contents of the data segment memory location address by TABLE plus BX.
JMP ECX	Jumps to the current code segment location addressed by the contents of ECX.
JMP RDI	Jumps to the linear address contained in the RDI register (64-bit mode).

Şekil 3-16 Çeşitli programların adreslerini depolayan bir atlama tablosu. TABLO'dan seçilen kesin adres, atlama talimatıyla depolanan bir dizin tarafından belirlenir.

TABLE	DW	LOC0
	DW	LOC1
	DW	LOC2
	DW	LOC3

CSE213 Mikrodenetleyici Programlama Adresleme Modları 56

Şekil 3-17 PUSH ve POP talimatları: (a) PUSH BX, BX'in içeriklerini yığına yerleştirir ; (b) POP CX, verileri yığından kaldırır ve bunları CX'e yerleştirir. Her iki talimat da yürütmeden sonra gösterilir.



CSE213 Mikrodenetleyici Programlama Adresleme Modları 60

CSE213 Mikrodenetleyici Programlama Adresleme Modları 53

3-3 YIĞIN BELLEK ADRESLEME MODLAR

- Yığın, tüm mikroişlemcilerde önemli bir rol oynar.
 - verileri geçici olarak tutar ve prosedürler tarafından kullanılan dönüş adreslerini depolar • Yığın belleği LIFO (son giren ilk çıkar) belleğidir
 - verilerin yığında saklanma ve yığından kaldırılma şeklini açıklar

CSE213 Mikrodenetleyici Programlama Adresleme Modları 57

- PUSH ve POP'un depolama veya alma işlevlerini yerine getirdiğini unutmayın .
 - 8086 - 80286 aralığındaki veri sözcükleri (asla bayt değil) .
- 80386 ve üzeri, kelimelerin veya çift kelimelerin yığına aktarılmasına ve yığından alınmasına izin verir.
- Veriler herhangi bir 16 bitlik kayıttan veya segment kayıttan yığına itilebilir. – 80386 ve üzeri sürümlerde, herhangi bir 32 bitlik genişletilmiş kayıttan Kayıt • Veriler , CS hariç herhangi bir kayıt veya segment kaydına yığından çıkarılabilir .

CSE213 Mikrodenetleyici Programlama Adresleme Modları 61

giriş

- Bu bölüm ortak verilere odaklanmaktadır hareket talimatları.

CSE213 Mikrodenetleyici Programlama Adresleme Modları 58

- PUSHA ve POPA talimatları, yığındaki segment kayıtları hariç hepsini iter veya çıkarır. • Erken 8086/8088 işlemcilerde mevcut değildir. • 80386 ve üzeri, genişletilmiş kayıtların itilmesine veya çıkarılmasına izin verir.

- Pentium ve Core2 için 64 bit modu bir PUSHA veya POPA talimatı içerir

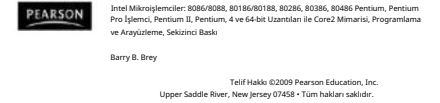
CSE213 Mikrodenetleyici Programlama Adresleme Modları 62

Bölüm Hedefleri

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

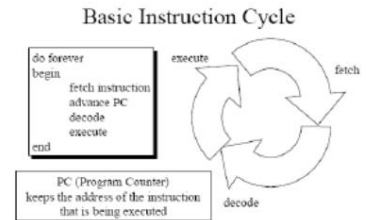
- Her veri taşıma talimatının işleyişini uygulanabilir adresleme modlarıyla açıklayın.
- Belirli bir veri taşıma görevini gerçekleştirmek için uygun derleme dili talimatını seçin.

CSE213 Mikrodenetleyici Programlama Adresleme Modları 59



CSE213 Mikrodenetleyici Programlama Adresleme Modları 63

Talimat Döngüsü



Veri Taşıma Talimatları

CSE213 Mikrodenetleyici Programlama Veri Taşıma Talimatları 2

CSE213 Mikrodenetleyici Programlama Veri Taşıma Talimatları 3

CSE213 Mikrodenetleyici Programlama Veri Taşıma Talimatları 4

- Yerel ikili kod mikroişlemcinin işlemlerini kontrol etmek için kullandığı talimatlardır. – talimatların uzunluğu 1 ila 13 bayt arasında değişir
- 100.000'den fazla makine dili çeşidi talimatlar.
 - Bu varyasyonların tam bir listesi yoktur • Bir makine dili talimatındaki bazı parçalar verilmiştir; kalan parçalar talimatın her varyasyonu için belirlenir.

MOD Alanı

- Adresleme modunu (MOD) belirtir ve seçili tipte bir yer değiştirmenin mevcut olup olmadığı .
 - MOD=11 ise, kayıt adresleme modu seçilir
 - Kayıt adreslemesi bunun yerine bir kayıt belirtir R/M alanını kullanarak bir bellek konumunun
- MOD alanı 00, 01 veya 10 içeriyorsa, R/M alanı veri belleği adresleme modlarından birini seçer .

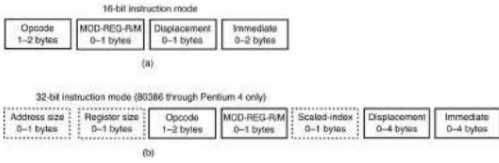
R/M Bellek Adresleme

- MOD alanı 00, 01 veya 10 içeriyorsa, R/M alanı yeni bir anlam kazanır. • Şekil 4–5, 16 bitlik MOV DL,[DI] talimatının veya talimatın (8A15H) makine dili sürümünü göstermektedir.
- Bu talimat 2 bayt uzunluğundadır ve 100010 işlem koduna sahiptir, D=1 (R/M'den REG'e), W=0 (bayt), MOD=00 (yer değiştirme yok), REG=010 (DL) ve R/M=101 ([DI]).

MOV Talimatı

- İkinci işleneni (kaynak) birinci işlenene (hedef) kopyalar .
- Kaynak işlenen, anlık bir değer, genel amaçlı bir kayıt veya bellek konumu olabilir.
- Hedef kayıt , genel amaçlı bir kayıt veya bellek konumu olabilir .
- Her iki işlenenin de aynı boyutta olması gerekir; bu boyut bir bayt veya bir sözcük olabilir.
- MOV yönergesi , CS ve IP kayıtlarının değeri .

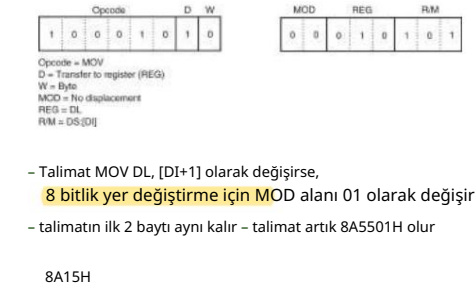
Talimat Formatı



Şekil 4-1 8086-Core2 talimatlarının biçimleri. (a) 16 bitlik form ve (b) 32 bitlik form.

- İşlemci talimatı yürüttüğünde tüm 8 bitlik yer değiştirmeler işaret genişletilerek 16 bitlik yer değiştirmelere dönüştürülür.
 - 8 bitlik yer değiştirme 00H–7FH (pozitif) ise, ofset adresine eklenmeden önce 0000H–007FH'ye işaret uzatılır
 - 8 bitlik yer değiştirme 80H–FFH (negatif) ise, FF80H–FFFFFFH'ye işaret uzatılmıştır

Şekil 4-5 Makine diline dönüştürülmüş bir MOV DL,[DI] talimatı.

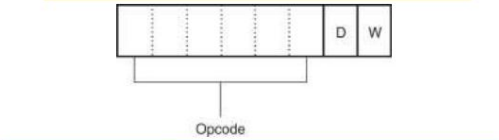


MOV'un İşlenenleri

- Bu tür işlenenler desteklenir: – MOV REG, bellek – MOV bellek, REG
 - MOV KAYIT, KAYIT
 - MOV bellek, anında
 - MOV REG, anında
- REG: AX, BX, CX, DX, AH, AL, BL, BH, CH, CL, DH, DL, DI, SI, SP,
- bellek: [BX], [BX+SI+7], değişken, vb.
- hemen: 5, -24, 3Fh, 10001101b, vb.

İşlem Kodu

- İşlemi (toplama, çıkarma vb.) seçer. mikroişlemci tarafından gerçekleştirilir. • Aşağıdaki şekil birçok talimatın ilk opkod baytının genel biçimini göstermektedir. – ilk baytın ilk 6 biti ikili opkoddur – kalan 2 bit veri akışının yönünü (D) belirtir ve verinin bir bayt mı yoksa bir kelime mi (W) olduğunu belirtir



Kayıt Ödevleri

- 2 baytlık bir talimat olan 8BECH'i varsayalım, bir makine dili programında görünür.
- 16 bit modunda, bu talimat ikiliye dönüştürülür ve Şekil 4–4'te gösterildiği gibi 1 ve 2 baytlık talimat biçimine yerleştirilir.

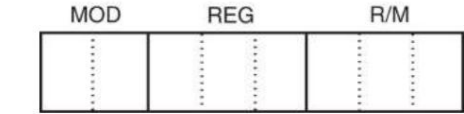
- MOD alanı 11 içerdiğinden R/M alanı da bir kaydı belirtir.
- R/M = 100(SP); bu nedenle, bu talimat verileri SP'den BP'ye taşır.
 - MOV BP,SP talimatı olarak sembolik biçimde yazılmıştır

- Derleyici program, kayıt ve adres boyutu örneklerini ve çalışma modunu izler .

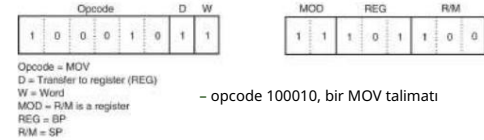
Segment Kayıt İşlenenleri

- Yalnızca segment kayıtları için bu MOV türleri desteklenir:
 - MOV SREG, bellek
 - MOV belleği, SREG
 - MOV KAYIT, SREG
 - MOV SREG, REG
- SREG: DS, ES, SS ve yalnızca ikinci işlenen olarak: CS.
- REG: AX, BX, CX, DX, AH, AL, BL, BH, CH, CL, DH, DL, DI, SI, BP, SP.
- bellek: [BX], [BX+SI+7], değişken, vb.

Şekil 4-3 Birçok makine dili talimatının 2. bayt, MOD, REG ve R/M alanlarının konumunu gösteriyor .



Şekil 4-4 Şekil 4-2 ve 4-3'teki 1 ve 2 bayt biçimlerine yerleştirilen 8BEC talimatı. Bu talimat bir MOV BP,SP'dir.



- D ve W bitleri mantıksal 1'dir, bu nedenle bir kelime REG alanında belirtilen hedef kayıt defterine taşınır
- REG alanı 101'i içerir ve BP kaydını gösterir, bu nedenle MOV talimatı verileri BP kaydına taşır

Veri Türleri

- Derleyiciye veri türünü bildirmek için şu önekler kullanılmalıdır: – BYTE PTR – bayt için.
 - WORD PTR - kelime için (iki bayt)
- Örnekler: – MOV AL, BYTE PTR [BX] ; bayt erişimi – MOV CX, WORD PTR [BX] ; kelime erişimi
- Derleyici daha kısa önekleri de destekler:
 - B. - BYTE PTR için
 - W. - WORD PTR için
- Bazı durumlarda montajcı verileri hesaplayabilir otomatik olarak yazın.

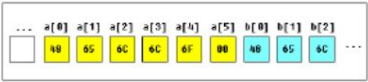
MOV Örneği

ORG 100h ; bu yönerge gerekiydi
MOV AX, 0B800h ; AX'ı B800h'nin onaltılık değerine ayarla.
MOV DS, AX ; AX değerini DS'ye kopyala.
MOV CL, 'A' ; CL'yi 'A'nın ASCII koduna ayarlayın, 41h olur.
MOV CH, 11011111b ; CH'yi ikili değere ayarla.
MOV BX, 15Eh ; BX'i 15Eh olarak ayarlayın.
MOV [BX], CX ; CX'in içeriklerini B800:015E'de belleğe kopyala
RET ; işletim sistemine geri dönün.

Değişkenler

- Değişken bildirimi için sözdizimi:
 - isim DB değeri _____
 - DW değeri adı _____
- DB - Define Byte deklare eder.
- DW - Define Word deklare eder.
- name herhangi bir harf veya rakam kombinasyonu olabilir, ancak bir harfle başlamalıdır. Adı belirtmeden adlandırılmamış değişkenleri bildirmek mümkündür (bu değişkenin bir adresi olacaktır ancak adı olmayacaktır).
- değer desteklenen herhangi bir sayısal değer olabilir numaralandırma sistemi (onaltılık, ikili veya ondalık) veya başlatılmamış değişkenler için "?" sembolü.

Dizi Elemanlarına Erişim



- Dizi içindeki herhangi bir elemanın değerine köşeli parantez kullanarak erişebilirsiniz , örneğin:
 - MOV AL, a[3]
- Ayrıca, örneğin BX, SI, DI, BP bellek dizin kayıtlarından herhangi birini de kullanabilirsiniz :
 - MOV SI, 3
 - MOV AL, a[SI]

Örnek

ORG 100h
 MOV AL, VAR1 ; VAR1 değerini AL'ye taşıyarak kontrol et.
 LEA BX, VAR1 ; BX'deki VAR1'in adresini al.
 MOV BYTE PTR [BX], 44h ; VAR1'in içeriğini değiştir.
 MOV AL, VAR1 ; VAR1 değerini AL'ye taşıyarak kontrol et.
 Geri dön
 VAR1 DB 22s
 SON

POP

- PUSH işleminin tersini gerçekleştirir.
- POP, verileri yığından kaldırır ve bunları hedef 16 bitlik bir kayıt defterine, segment kaydına veya 16 bitlik bir bellek konumuna yerleştirir.
 - anında POP olarak mevcut değil
- POPF (pop bayrakları) 16 bitlik bir sayıyı kaldırır yığından alır ve bayrak kayıt defterine yerleştirir;
 - POPF, yığından 32 bitlik bir sayıyı kaldırır ve onu genişletilmiş bayrak kaydına yerleştirir

Örnek

ORG 100h
 MOV AL, değişken1
 MOV BX, değişken2
 RET ; programı durdurur.
 var1 DB 7
 var2 DW 1234h

Büyük Dizileri Bildirme

- Eğer büyük bir dizi tanımlamanız gerekiyorsa DUP operatörünü kullanabilirsiniz.
- DUP için sözdizimi :
 - sayı DUP (değer(ler)) – sayı yapılacak çoğaltma sayısı (herhangi bir sabit değer).
 - değer - DUP'un kopyalayacağı ifade.
- Örnek:
 - c DB 5 ÇOĞALT(9)
 - şunu bildirmenin alternatif bir yoludur: c DB 9, 9, 9, 9, 9

Sabitler

- Sabitler tıpkı değişkenler gibidir, ancak yalnızca programınız derlenene (birleştirilene) kadar var olurlar.
- Bir sabitin tanımlı yapıldıktan sonra değeri değiştirilemez değişti.
- Sabitleri tanımlamak için EQU yönergesi kullanılır:
 - isim EQU < herhangi bir ifade >
- Örnek:
 - k EK 5
 - MOV AX, k
- Yukarıdaki örnek işlevsel olarak şu kodla aynıdır:
 - MOV AX, 5

- POPA (pop all), yığından 16 bayt veriyi kaldırır ve bunları gösterilen sırayla şu kayıtlara yerleştirir: DI, SI, BP, SP, BX, DX, CX ve AX. – PUSHA talimatıyla yığındaki yerleştirmenin ters sırası, aynı verilerin aynı kayıtlara geri dönmeye neden olur

ORG Yönergesi

- ORG 100h bir montajcı yönergesidir (montajcıya kaynak kodu nasıl işlenir).
- Derleyiciye çalıştırılabilir dosyanın 100h (256 bayt) ofsetinde yüklenmesini söyler , bu yüzden derleyici değişken adlarını ofsetleriyle değiştirirken tüm değişkenler için doğru adresi hesaplamalıdır .

- Yönergeler hiçbir zaman gerçek bir makineye dönüştürülmez kod.
- Yürütülebilir dosya neden 100h konumunda yükleniyor ?
- İşletim sistemi CS'nin (kod segmenti) ilk 256 baytında programla ilgili bazı verileri (kod satırı parametreleri vb.) tutar.

Büyük Dizileri Bildirme

- Bir örnek daha:
 - d DB 5 ÇOĞALT(1, 2)
 - şunu bildirmenin alternatif bir yoludur: d DB 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2
- Elbette, eğer DB yerine DW kullanabilirsiniz .
 - 255'ten büyük veya -128'den küçük değerlerin tutulması gerekir.
- DW dizeleri bildirmek için kullanılamaz.

İTMEK

- Yığına her zaman 2 bayt veri aktarır; – 80386 ve üzeri 2 veya 4 bayt aktarır
- PUSHA komutu, segment kayıtları hariç, dahili kayıt kümesinin içeriğini yığına kopyalar.
- PUSHA (tümünü it) talimatı, yığına şu sırayla kayıt yapar: AX, CX, DX, BX, SP, BP, SI ve DI.

4–4 DİZİ VERİ AKTARIMLARI

- Dize talimatları sunulmadan önce, D bayrak biti (yön), DI ve SI'nin işlemlerinin dize talimatlarına nasıl uygulandığı anlaşılmalıdır.

Diziler

- Diziler değişken zincirleri olarak görülebilir.
- Bir metin dizesi, bir bayt dizisinin örneğidir; her karakter bir ASCII kod değeri (0..255) olarak gösterilir.
- Örnekler: – DB
 - 48h, 65h, 6Ch, 6Ch, 6Fh, 00h
 - b DB 'Merhaba', 0
- b, a dizisinin tam bir kopyasıdır , derleyici tırnak işaretleri içinde bir dize gördüğünde onu otomatik olarak bayt kümesine dönüştürür.

Bir Değişkenin Adresini Alma

- LEA talimatı bir değişkenin ofset adresini almak için kullanılabilir .
- Değişkenin adresini almak bazı durumlarda çok faydalı olabilir, örneğin bir prosedüre parametre geçirmeniz gerektiğinde.

- PUSHF (push flags) komutu, flag register'ının içeriğini yığına kopyalar. • PUSHAD ve POPAD komutları, 80386 - Pentium 4'te ayarlanan 32-bit register'ın içeriğini iter ve çıkarır.
- PUSHA ve POPA talimatları çalışmıyor Pentium 4 için 64 bitlik çalışma modu

Yön Bayrağı

- Yön bayrağı (bayrak kaydında bulunan D), dize işlemleri sırasında DI ve SI kayıtları için otomatik artırma veya otomatik azaltma işlemini seçer . – yalnızca dize talimatlarıyla kullanılır
- CLD talimatı D bayrağını temizler ve STD komut setleri bunu belirler.
 - CLD talimatı otomatik artırma modunu seçer ve STD otomatik azaltma modunu seçer

- Dize talimatlarının yürütülmesi sırasında bellek erişimleri DI ve SI kayıtları aracılığıyla gerçekleşir.
 - DI ofset adresi, onu kullanan tüm dize talimatları için ekstra segmentteki verilere erişir
 - SI ofset adresi varsayılan olarak veri segmentindeki verilere erişir

Örnek

```
ORG 100h
LE DI, a1
MOV AL, 12s
MOV CX, 5
Temsilci
STOSB
Geri dön
a1 DB 5 dup(0)
```

- G/Ç aygıtı (port) adreslemesinin iki biçimi:
- Sabit port adresleme, 8 bitlik bir G/Ç port adresi kullanılarak AL, AX veya EAX arasında veri aktarımına olanak tanır . – port numarası, talimatın işlem kodunu takip eder
- Değişken port adresleme, AL, AX veya EAX ile 16 bitlik bir port adresi arasında veri transferine olanak tanır.
- G/Ç port numarası, bir programın yürütülmesi sırasında değiştirilebilen (değiştirilebilen) DX kayıt defterinde saklanır .

giriş

- Aritmetik ve mantığı inceliyoruz talimatlar. Aritmetik talimatlar toplama, çıkarma, çarpma, bölme, karşılaştırma, olumsuzlama, artırma ve azaltmayı içerir.
- Mantık talimatları arasında AND, OR bulunur. Özel-VEYA, DEĞİL, kaydırma, döndürme ve mantıksal karşılaştırma (TEST).

LODSB Talimatı

- DS:[SI]'daki baytı AL'ye yükle. SI'yi güncelle.

Algoritma:
AL = DS:[SI]

- eğer DF = 0 ise o zaman
 - SI = SI + 1
- başka
 - SI = SI - 1

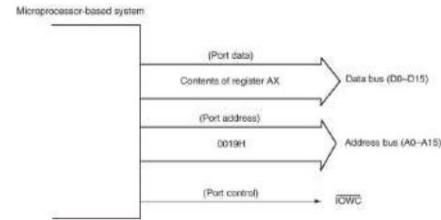
MOVSb Talimatı

- DS:[SI]'deki baytı ES:[DI]'ye kopyalayın. SI ve DI'yi güncelleyin.

Algoritma:
ES:[DI] = DS:[SI]

- eğer DF = 0 ise o zaman
 - SI = SI + 1
 - DI = DI + 1
- başka
 - SI = SI - 1
 - DI = DI - 1

Şekil 4-20 Mikroijlemlci tabanlı sistemde OUT 19H,AX komutu için bulunan sinyaller.



Bölüm Hedefleri

Bu bölümü tamamladıgınızda şunları yapabileceksiniz:

- Basit ikili, BCD ve ASCII aritmetiğini gerçekleştirmek için aritmetik talimatları kullanın .
- İkili bit işlemlerini gerçekleştirmek için mantık talimatlarını kullanın.
- Kaydırma ve döndürme talimatlarını kullanın.

Örnek

```
ORG 100h
LEAS SI, a1
MOV CX, 5
MOV AH, 0Eh
m: LODSB
INT 10s
DÖNGÜ m
Geri dön
a1 DB 'H', 'e', 'l', 'l', 'l', 'o'
```

Örnek

```
ORG 100h
ÇKD
LEAS SI, a1
LE DI, a2
MOV CX, 5
Temsilci
MOVSb
Geri dön
a1 DB 1,2,3,4,5 a2
DB 5 ÇOĞALT(0)
```

STOSB Talimatı

- AL'daki baytı ES:[DI]'de saklayın. DI'yi güncelleyin.

Algoritma:
Türkçe: [DI] = AL

- eğer DF = 0 ise o zaman
 - DI = DI + 1
- başka
 - DI = DI - 1

İÇERİ ve DIŞARI

- GİRİŞ ve ÇIKIŞ talimatları G/Ç işlemlerini gerçekleştirir. • AL, AX veya EAX içerikleri aktarılır. sadece G/Ç aygıtı ile mikroijlemlci arasında.
 - bir IN talimatı, verileri harici bir cihazdan aktarır G/Ç aygıtı AL, AX veya EAX'a
 - bir OUT, verileri AL, AX veya EAX'tan bir OUT'a aktarır harici G/Ç aygıtı

CSE213

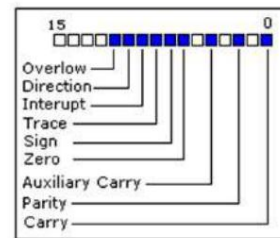
MİKRODENETLEYİCİ PROGRAMLAMA

Aritmetik ve Mantık Talimatları

PEARSON
Intel Mikroijlemler: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro İjlemlci, Pentium II, Pentium, 4 ve 64-bit Uzanlıları ile Core2 Minaris, Programlama ve Arayüzleme, Sekizinci Baskı
Barry B. Brey
Telif Hakkı ©2009 Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • Tüm hakları saklıdır.

Aritmetik ve Mantık Talimatları

- Aritmetik talimatların çoğu FLAGS'ı etkiler kayıt olmak.

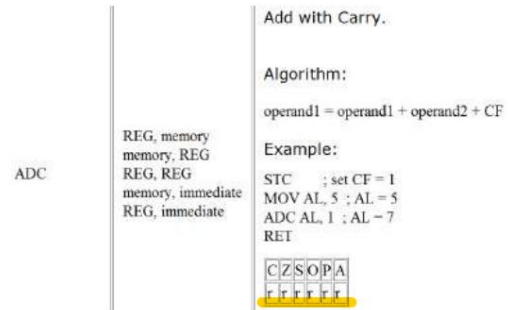
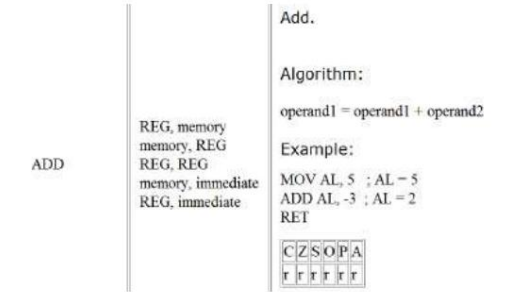


Aritmetik ve Mantık Talimatları

- Taşıma Bayrağı (C) - bu bayrak, işaretsiz bir taşıma olduğunda 1 olarak ayarlanır . Örneğin, 255 + 1 bayt eklediğinizde (sonuç 0...255 aralığında değildir). Taşıma olmadığına bu bayrak 0 olarak ayarlanır.
- Sıfır Bayrağı (Z) - sonuç sıfır olduğunda 1 olarak ayarlanır . Sıfır olmayan sonuç için bu bayrak 0 olarak ayarlanır .
- İşaret Bayrağı (S) - sonuç negatif olduğunda 1 olarak ayarlanır . Sonuç pozitif olduğunda 0'a ayarlanır. Aslında bu bayrak en anlamlı bitin değerini alır.
- Taşıma Bayrağı (O) - Taşıma olduğunda 1 olarak ayarlayın imzalı bir taşıma. Örneğin, 100 + 50 bayt eklediğinizde (sonuç -128...127 aralığında değildir).

- Parite Bayrağı (P) - bu bayrak , sonuçta çift sayıda bir bit olduğunda 1'e , tek sayıda bir bit olduğunda 0'a ayarlanır . Sonuç bir kelime olsa bile sadece 8 düşük bit analiz edilir!

- Yardımcı Bayrak (A) - 1 olarak ayarlandığında düşük bit için işaretsiz taşıma (4 bit).
- Kesinti etkinleştirme Bayrağı (I) - bu bayrak ayarlandığında 1 CPU harici cihazlardan gelen kesmelere tepki verir .
- Yön Bayrağı (D) - bu bayrak bazı kişiler tarafından kullanılır. veri zincirlerini işleme talimatları, bu bayrak 0 olarak ayarlandığında - işleme ileri doğru yapılır, bu bayrak 1 olarak ayarlandığında - işleme geri doğru yapılır.

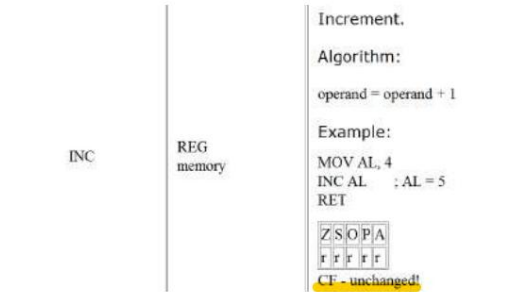


Çıkarma-Ödünç Alma

- Ödünç alma ile çıkarma (SBB) talimatı düzenli bir çıkarma işlemi gibi çalışır, ancak ödünç alma işlemini tutan taşıma bayrağı (C) aynı zamanda farktan da çıkarma yapar.
 - en yaygın kullanım 16'dan daha geniş çıkarmalardır 8086-80286 mikroişlemcilerde bitler veya 80386-Core2'de 32 bitten daha geniş.
 - geniş çıkarmalar, çıkarma işlemi boyunca yayılmak için ödünç almaları gerektirir , tıpkı geniş eklemelemlerin taşımayı yayması gibi

TOPLAMA, ÇIKARMA VE KARŞILAŞTIRMAK

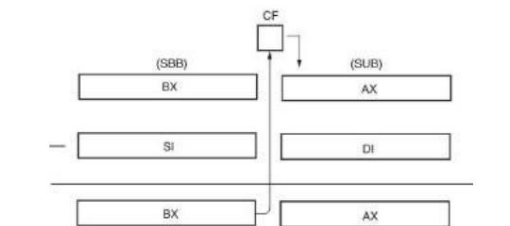
- Aritmetik talimatların büyük kısmı şu şekilde bulunur: herhangi bir mikroişlemci toplama, çıkarma ve karşılaştırmayı içerir. • Toplama, çıkarma ve karşılaştırma Talimatlar resimli olarak gösterilmiştir.
- Ayrıca kayıt ve bellek verilerinin işlenmesinde kullanımları da gösterilmektedir .



Çıkarma

- Talimat setinde çıkarma işleminin (SUB) birçok biçimi görülmektedir .
 - bunlar 8, 16 veya 32 bit veri içeren herhangi bir adresleme modunu kullanır
 - özel bir çıkarma biçimi (azaltma veya DEC), herhangi bir kayıt veya bellek konumundan 1 çıkarır
- Her çıkarma işleminden sonra, mikroişlemci bayrak kaydının içeriğini değiştirir. - bayraklar çoğu aritmetik/ mantık işlemi için değişir

Şekil 5-2, ödünç alma ile çıkarma, taşıma bayrağının (C) ödünç almayı nasıl yaydığını göstermektedir.

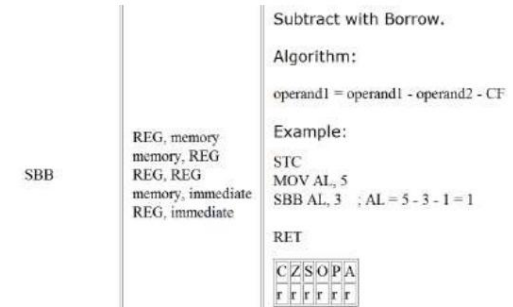
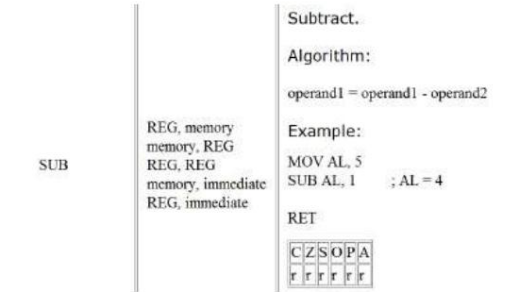


EK

- Ekleme (ADD) birçok biçimde ortaya çıkar mikroişlemci.
- Eklemenin ikinci bir biçimi, add-with-carry, ADC komutuyla birlikte tanıtılır.
- İzin verilmeyen tek ekleme türleri bellekten belleğe ve segment kaydırır. - segment kayıtları yalnızca taşınabilir, itilebilir, veya patladı.
- Artış talimatı (INC), özel bir tür talimattır Bir sayıya 1 ekleyen ekleme.

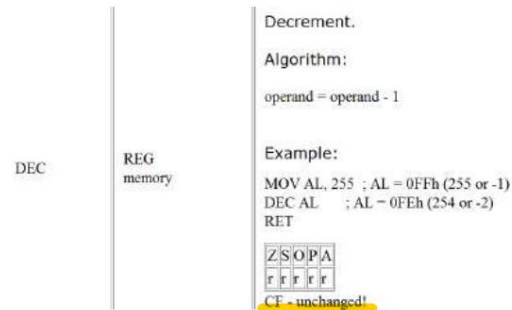
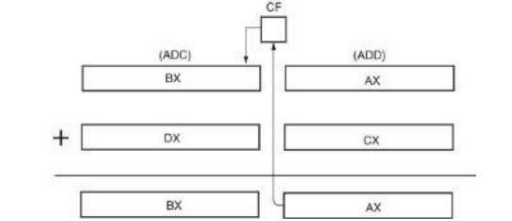
Taşımalı Ekleme

- ADC taşıma bayrağındaki (C) biti işlenen verilerine ekler. - esas olarak sayıları ekleyen yazılımlarda görülür 16 veya 32 bit'ten daha geniş.
 - ADD gibi, ADC de eklemekten sonra bayrakları etkiler
- Şekil 5-1 bunu göstermektedir, böylece taşıma bayrağının yerleşimi ve işlevi anlaşılabilir. - 8086-80286 yalnızca taşıma bayrağı biti eklenmeden kolayca gerçekleştirilemez.
 - 8 veya 16 bitlik sayıları toplar



- Herhangi bir ADD talimatı , işaret, sıfır, taşıma, yardımcı taşıma, eşlik ve taşıma bayraklarının içeriğini değiştirir .

Şekil 5-1 Taşımalı toplama, taşıma bayrağının (C) iki 16 bitlik toplamaya tek bir 32 bitlik toplamaya nasıl bağladığını göstermektedir.



Karşılaştırmak

- Karşılaştırma talimatı (CMP), bir yalnızca bayrak bitlerini değiştiren çıkarma. - hedef işlenen asla değişmez • Bir kayıt veya bellek konumunun içeriğini başka bir değere göre kontrol etmek için kullanılırdır. • Bir CMP'yi normalde bayrak bitlerinin durumunu test eden koşullu bir atlama talimatı izler.

CMP

REG, memory
memory, REG
REG, immediate
REG, immediate

Compare.

Algorithm:

operand1 - operand2

result is not stored anywhere, flags are set (OF, SF, ZF, AF, PF, CF) according to result.

Example:

```
MOV AL, 5
MOV BL, 5
CMP AL, BL ; AL = 5, ZF = 1 (so equal!)
RET
```

C Z S O P A
r r r r r r

IMUL

REG
memory

Signed multiply.

Algorithm:

when operand is a **byte**:
AX = AL * operand.

when operand is a **word**:
(DX AX) = AX * operand.

Example:

```
MOV AL, -2
MOV BL, -4
IMUL BL ; AX = 8
RET
```

C Z S O P A
r r r r r r

CF=OF=0 when result fits into operand of IMUL.

DIV

REG
memory

Unsigned divide.

Algorithm:

when operand is a **byte**:
AL = AX / operand
AH = remainder (modulus)

when operand is a **word**:
AX = (DX AX) / operand
DX = remainder (modulus)

Example:

```
MOV AX, 203 ; AX = 00CBh
MOV BL, 4
DIV BL ; AL = 50 (32h), AH = 3
RET
```

C Z S O P A
r r r r r r

DAA Talimatı

- DAA, ADD veya ADC talimatını takip eder
- Sonucu BCD sonucuna ayarlayın.
- BL ve DL kayıtları eklendikten sonra sonuç CL'ye kaydedilmeden önce bir DAA talimatı ile ayarlanır.

DAS Talimatı

- DAA talimatlarıyla aynı işlevi görür, ancak toplama yerine çıkarma işlemini takip eder.

ÇARPMALI VE BÖLMELİ

- Daha önceki 8-bit mikroişlemciler, bir dizi kaydırma ve toplama veya çıkarma kullanarak çarpan veya bölen bir program kullanılmadan çarpma veya bölme işlemini gerçekleştirmezdi. – üreticiler bu yetersizliğin farkındaydı,

çarpma ve bölmeyi daha yeni mikroişlemcilerin talimat setlerine dahil ettiler. • Pentium–Core2, çarpma işlemini

yalnızca bir saat periyodunda yapmak için özel devreler içerir. – daha önceki işlemcilerde 40'tan fazla saat periyodu

Bölüm

- Mikroişlemciye bağlı olarak 8 veya 16 bit ve 32 bit sayılarda meydana gelir. – işaretli (IDIV) veya işaretli (DIV) tam sayılar • Bölünen her

zaman çift genişlikte bir temettü olur, işlenene bölünür.

- Hemen bir bölme talimatı yok

Herhangi bir mikroişlemci için kullanılabilir.

IDIV

REG
memory

Signed divide.

Algorithm:

when operand is a **byte**:
AL = AX / operand
AH = remainder (modulus)

when operand is a **word**:
AX = (DX AX) / operand
DX = remainder (modulus)

Example:

```
MOV AX, -203 ; AX = 0FF35h
MOV BL, 4
IDIV BL ; AL = -50 (0CEh), AH = -3 (0FDh)
RET
```

C Z S O P A
r r r r r r

DAA

No operands

Decimal adjust After Addition.
Corrects the result of addition of two packed BCD values.

Algorithm:

if low nibble of AL > 9 or AF = 1 then:

- AL = AL + 6
- AF = 1

if AL > 9Fh or CF = 1 then:

- AL = AL + 60h
- CF = 1

Example:

```
MOV AL, 0Fh ; AL = 0Fh (15)
DAA ; AL = 15h
RET
```

C Z S O P A
r r r r r r

Çarpma

- Çarpma talimatı bir işlenen içerir çünkü her zaman işleneni AL/AX kaydının içeriğiyle çarpır .

- Baytlar, kelimeler veya çift kelimeler üzerinde gerçekleştirilir, – imzalı (IMUL) veya imzasız tamsayı (MUL) olabilir.

- Çarpmadan sonra elde edilen ürün her zaman çift genişlikte bir ürün olur. – iki

8 bitlik sayı çarpıldığında 16 bitlik bir ürün elde edilir;
iki 16 bitlik sayı çarpıldığında 32 bitlik bir ürün elde edilir;
iki 32 bitlik sayı 64 bitlik bir ürün üretir

- Bir bölünme iki tür hataya yol açabilir:
 - sıfıra bölme girişi
 - diğeri, küçük bir sayının büyük bir sayıya bölünmesiyle oluşan bir bölme taşmasıdır
- Her iki durumda da, bir bölme hatası olursa mikroişlemci bir kesinti üretir. • Çoğu sistemde, bir bölme hatası kesintisi videoda bir hata mesajı görüntüler.

ekran.

BCD ve ASCII Aritmetiği

- Mikroişlemci aritmetik işlemlerin yapılmasına olanak sağlar hem BCD (ikili kodlanmış ondalık) hem de ASCII (Bilgi Değişimi için Amerikan Standart Kodu) verilerinin işlenmesi . • BCD işlemleri, aşağıdaki gibi sistemlerde meydana gelir:
 - satış noktası terminaleri (örneğin, kasalar) ve nadiren karmaşık aritmetik gerektiren diğerleri.

DAS

No operands

Decimal adjust After Subtraction.
Corrects the result of subtraction of two packed BCD values.

Algorithm:

if low nibble of AL > 9 or AF = 1 then:

- AL = AL - 6
- AF = 1

if AL > 9Fh or CF = 1 then:

- AL = AL - 60h
- CF = 1

Example:

```
MOV AL, 0Fh ; AL = 0Fh (-1)
DAS ; AL = 99h, CF = 1
RET
```

C Z S O P A
r r r r r r

MUL

REG
memory

Unsigned multiply.

Algorithm:

when operand is a **byte**:
AX = AL * operand.

when operand is a **word**:
(DX AX) = AX * operand.

Example:

```
MOV AL, 200 ; AL = 0C8h
MOV BL, 4
MUL BL ; AX = 0320h (800)
RET
```

C Z S O P A
r r r r r r

CF=OF=0 when high section of the result is zero.

Geriye Kalan

- Bölümü yuvarlamak için kullanılabilir veya bölümü kesmek için bırakılabilir.
- Bölme işaretli ise, yuvarlama işlemi kalanın bölenin yansıya karşılaştırılmasını gerektirir Bölümün yuvarlanıp yuvarlanmayacağına karar vermek
- Kalan, kesirli kalana da dönüştürülebilir.

BCD Aritmetiği

- BCD ile iki aritmetik teknik çalışır
- Veriler: toplama ve çıkarma.
- DAA (eklemeden sonra ondalık ayarlama) talimatı BCD eklemesini takip eder,
- DAS (çıkarma işleminden sonra ondalık ayarlama) BCD çıkarma işlemini takip eder.
 - her ikisi de toplama veya çıkarma sonucunu düzeltir yani bu bir BCD sayıdır

ASCII Aritmetiği

- ASCII aritmetik talimatları işlevi
 - kodlanmış sayılar, 0–9 için 30H ila 39H değeri.
- ASCII aritmetiğinde dört talimat
 - işlemler: – AAA (eklemeden sonra ASCII ayarlaması)
 - AAD (Bölmeden önce ASCII ayarlaması)
 - AAM (Çarpmadan sonra ASCII ayarlaması)
 - AAS (Çıkarma işleminden sonra ASCII ayarlaması)
- Bu talimatlar kaynak ve hedef olarak AX kaydını kullanır.

- İki adet tek haneli ASCII kodlu sayının eklenmesi sayılar hiçbir yararlı veriye yol açmayacaktır.

AAD Talimatı

- Bir bölme işleminden önce görünür.
- AAD talimatı, yürütülmeden önce AX kaydının iki basamaklı paketlenmemiş bir BCD numarası (ASCII değil) içermesini gerektirir.

VE

- Doğruluk tablosuyla gösterilen mantıksal çarpma işlemini gerçekleştirir .
- Gereken hız çok büyük değilse VE ayrı VE kapılarının yerini alabilir – normalde gömülü kontrol uygulamaları için ayrılmıştır
- 8086'da, AND talimatı genellikle yürütülür yaklaşık bir mikrosaniye içinde.
 - daha yeni sürümlerle yürütme hızı büyük ölçüde artırıldı

AND

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Logical AND between all bits of two operands.
Result is stored in operand1.

These rules apply:
1 AND 1 = 1
1 AND 0 = 0
0 AND 1 = 0
0 AND 0 = 0

Example:
MOV AL, 'a' ; AL = 01100001b
AND AL, 11011111b ; AL = 01000001b ('A')
RET

CZSOP
011011

Şekil 5-6 Bir sayının bitlerinin nasıl bir ayarlandığını gösteren OR fonksiyonunun çalışması.

xxxx xxxx Unknown number
+ 0000 1111 Mask

xxxx 1111 Result
1ler gerekiz 0 nedene 1 ile maskelidi

MOV AL, 5 ;load data
MOV BL, 7
MUL BL
AAM ;adjust
OR AX, 3030H ;convert to ASCII

AAM Talimatı

- Çarpma talimatlarını takip eder iki adet tek basamaklı açılmamış BCD sayısının çarpılması.
- AAM, ikili sayıyı paketlenmemiş BCD'ye dönüştürür. • AX'te 0000H ile 0063H arasında bir ikili sayı görünüyorsa, AAM bunu BCD'ye dönüştürür.

AAS Talimatı

- AAS, bir ASCII çıkarmından sonra AX kaydını ayarlar.

Şekil 5-3 (a) VE işlemi için doğruluk tablosu ve (b) bir VE kapısının mantık sembolü.

A B T
0 0 0
0 1 0
1 0 0
1 1 1

A B T

Örnekler

TABLE 5-16 Example AND instructions.

Assembly Language Operation

AND AL,BL AL = AL and BL
AND CX,DX CX = CX and DX
AND ECX,EDI ECX = ECX and EDI
AND RDX,RBP RDX = RDX and RBP (64-bit mode)
AND CL,33H CL = CL and 33H
AND DI,4FFFH DI = DI and 4FFFH
AND ESI,34H ESI = ESI and 34H
AND RAX,1 RAX = RAX and 1 (64-bit mode)
AND AX,[DI] The word contents of the data segment memory location addressed by DI are ANDed with AX
AND ARRAY[SI],AL The byte contents of the data segment memory location addressed by ARRAY plus SI are ANDed with AL
AND [EAX],CL CL is ANDed with the byte contents of the data segment memory location addressed by ECX

OR

REG, memory
memory, REG
REG, REG
memory, immediate
REG, immediate

Logical OR between all bits of two operands.
Result is stored in first operand.

These rules apply:
1 OR 1 = 1
1 OR 0 = 1
0 OR 1 = 1
0 OR 0 = 0

Example:
MOV AL, 'A' ; AL = 01000001b
OR AL, 00100000b ; AL = 01100001b ('a')
RET

CZSOPA
011011

TEMEL MANTIK TALİMATLARI

- AND, OR, Exclusive-OR ve NOT'u ekleyin.
 - ayrıca AND komutunun özel bir biçimi olan TEST
 - NEG, NOT talimatına benzer
- Mantıksal işlemler, düşük seviyeli yazılımlarda ikili bit denetimi sağlar.
 - bitlerin ayarlanmasına, temizlenmesine veya tamamlanmasına izin verir
- Makinede düşük seviyeli yazılımlar belirli dil veya montaj dili bir sistemdeki G/Ç aygıtlarını oluşturur ve sıklıkla kontrol eder.

• VE, ikili bir sayının bitlerini temizler. – maskeleme olarak adlandırılır

• VE, bellekten-başlangıca hariç herhangi bir modu kullanır bellek ve segment kayıt adreslemesi. • Bir ASCII sayısı, en soldaki dört ikili bit konumunu maskelemek için VE kullanılarak BCD'ye dönüştürülebilir.

VEYA

- Mantıksal toplama işlemini gerçekleştir
 - genellikle Kapsayıcı-VEYA işlevi olarak adlandırılır
- OR fonksiyonu, herhangi bir giriş 1 ise mantıksal 1 çıktısı üretir. – 0, yalnızca tüm girişler 0 olduğunda çıktıda görünür
- Şekil 5-6, VEYA kapısının ikili sayının herhangi bir bitini (1) nasıl ayarladığını göstermektedir.
- VEYA talimatı segment kayıt adreslemesi haricindeki herhangi bir adresleme modunu kullanır.

Örnekler

TABLE 5-17 Example OR instructions.

Assembly Language Operation

OR AH,BL AL = AL or BL
OR SLDX SI = SI or DX
OR EAX,EBX EAX = EAX or EBX
OR R9,R10 R9 = R9 or R10 (64-bit mode)
OR DH,0A3H DH = DH or 0A3H
OR SP,990DH SP = SP or 990DH
OR EBP,10 EBP = EBP or 10
OR RBP,1000H RBP = RBP or 1000H (64-bit mode)
OR DX,[BX] DX is ORed with the word contents of data segment memory location addressed by BX
OR DATES[DI + 2],AL The byte contents of the data segment memory location addressed by DI plus 2 are ORed with AL

- Tüm mantık talimatları bayrak bitlerini etkiler. •

- Mantık işlemleri her zaman taşıma ve taşıma bayraklarını temizler.
- diğer bayraklar sonucu yansıtacak şekilde değişir
- İkili veriler bir veri tabanında işlendiğinde kayıt veya bir bellek konumu, en sağdaki bit konumu her zaman bit 0 olarak numaralandırılır. – konum numaraları bit 0'dan sola, bir bayt için bit 7'ye ve bir kelime için bit 15'e doğru artar
 - bir çift sözcük (32 bit) en soldaki biti olarak 31 bit konumunu ve bir dörtlü sözcük (64 bit) 63 konumunu kullanır

Şekil 5-4 Bir sayının bitlerinin nasıl temizlendiği gösteren VE işleminin çalışması.

xxxx xxxx Unknown number
• 0000 1111 Mask

0000 xxxx Result
Her Türlü Sıfır getirmeyi yüksek nibble maskelidi

MOV BX, 3135H ;load ASCII
AND BX, 0F0FH ;mask BX

Şekil 5-5 (a) VEYA işlemi için doğruluk tablosu ve (b) VEYA kapısının mantık sembolü.

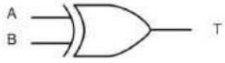
A B T
0 0 0
0 1 1
1 0 1
1 1 1

A B T

Özel-VEYA

- Inclusive-OR (OR)'dan farklıdır, çünkü Exclusive-OR'un 1,1 koşulu 0 üretir. – OR işlevinin 1,1 koşulu 1 üretir
- Exclusive-OR işlemi bu koşulu hariç tutar ; Inclusive-OR işlemi ise bunu içerir .
- Exclusive-OR fonksiyonunun girdileri hem 0 hem de 1 ise çıktı 0 olur; girdiler farklı ise çıktı 1 olur.
- Exclusive-OR bazen bir karşılaştırmacı.

A	B	T
0	0	0
0	1	1
1	0	1
1	1	0



(a)

(b)

- XOR, segment kayıt adreslemesi haricindeki herhangi bir adresleme modunu kullanır.
- Exclusive-OR, bir verinin bazı bitlerinin kayıt veya bellek konumu ters çevrilmelidir.
- Şekil 5–8, bir parçanın nasıl sadece bir kısmının bilinmeyen nicelik XOR ile tersine çevrilebilir. – X ile 1 Exclusive-OR yapıldığında sonuç X olur
 - eğer 0 X ile Özel-VEYA ise, sonuç X olur
- Exclusive-OR için yaygın bir kullanım talimat bir kaydı sıfırlamaktır

x x x x x x x x Unknown number
 ⊕ 0 0 0 0 1 1 1 1 Mask

 x x x x x x x x Result

Sıfırlama Yöntemi

```

OR  CX,0600H      ;set bits 9 and 10
AND CX,0FFFCH     ;clear bits 0 and 1
XOR  CX,1000H     ;invert bit 12
  
```

Logical XOR (Exclusive OR) between all bits of two operands. Result is stored in first operand.

These rules apply:

1 XOR 1 = 0
 1 XOR 0 = 1
 0 XOR 1 = 1
 0 XOR 0 = 0

Example:

```

MOV AL, 00000111b
XOR AL, 00000101b ; AL = 00000101b
RET
  
```

CZSOPA
0 1 1 0 1 1

Örnekler

TABLE 5–18 Example Exclusive-OR instructions.

Assembly Language	Operation
XOR CH,DL	CH = CH xor DL
XOR SI,BX	SI = SI xor BX
XOR EBX,EDI	EBX = EBX xor EDI
XOR RAX,RBX	RAX = RAX xor RBX (64-bit mode)
XOR AH,0EEH	AH = AH xor 0EEH
XOR DI,00DDH	DI = DI xor 00DDH
XOR ESI,100	ESI = ESI xor 100
XOR R12,20	R12 = R12 xor 20 (64-bit mode)
XOR DX,[SI]	DX is Exclusive-ORed with the word contents of the data segment memory location addressed by SI
XOR DEAL[BP+2],AH	AH is Exclusive-ORed with the byte contents of the stack segment memory location addressed by BP plus 2

Test ve Bit Testi Talimatları

- TEST, AND işlemini gerçekleştirir.
 - yalnızca bayrak sicilinin durumunu etkiler, testin sonucunu gösteren
 - CMP ile aynı şekilde çalışır
- Genellikle JZ (sıfır varsa atla) veya JNZ (sıfır değilse atla) talimatı takip eder. • Hedef işlenen normalde

test edilir

```

TEST AL,1      ;test right bit
JNZ RIGHT     ;if set
TEST AL,128    ;test left bit
JNZ LEFT      ;if set
  
```

Logical AND between all bits of two operands for flags only. These flags are effected: ZF, SF, PF. Result is not stored anywhere.

These rules apply:

1 AND 1 = 1
 1 AND 0 = 0
 0 AND 1 = 0
 0 AND 0 = 0

Example:

```

MOV AL, 00000101b
TEST AL,1      ;ZF = 0.
TEST AL,10b    ;ZF = 1.
RET
  
```

CZSOPA
0 1 1 0 1 1

Örnekler

TABLE 5–19 Example TEST instructions.

Assembly Language	Operation
TEST DL,DH	DL is ANDed with DH
TEST CX,BX	CX is ANDed with BX
TEST EDX,ECX	EDX is ANDed with ECX
TEST RDX,R15	RDX is ANDed with R15 (64-bit mode)
TEST AH,4	AH is ANDed with 4
TEST EAX,256	EAX is ANDed with 256

DEĞİL ve NEG

- NOT ve NEG herhangi bir adreslemeyi kullanabilir segment kayıt adreslemesi hariç mod.
- NOT komutu bir baytın, sözcüğün veya çift sözcüğün tüm bitlerini tersine çevirir.
- NEG iki, bir sayının tamamlayıcısıdır.
 - işaretli bir sayının aritmetik işareti pozitiften negatife veya negatiften pozitifte değişir
- NOT fonksiyonu mantıksal, NEG fonksiyonu ise aritmetik işlem olarak kabul edilir.

NOT

REG memory

Invert each bit of the operand.

Algorithm:

- If bit is 1 turn it to 0.
- If bit is 0 turn it to 1.

Example:

```

MOV AL, 00011011b
NOT AL ; AL = 11100100b
RET
  
```

CZSOPA
unchanged

NEG

REG memory

Negate. Makes operand negative (two's complement).

Algorithm:

- Invert all bits of the operand
- Add 1 to inverted operand

Example:

```

MOV AL, 5 ; AL = 05h
NEG AL ; AL = 0FBh (-5)
NEG AL ; AL = 05h (5)
RET
  
```

CZSOPA
0 1 1 0 1 1

Örnekler

TABLE 5–21 Example NOT and NEG instructions.

Assembly Language	Operation
NOT CH	CH is one's complemented
NEG CH	CH is two's complemented
NEG AX	AX is one's complemented
NOT EBX	EBX is one's complemented
NEG ECX	ECX is two's complemented
NOT RAX	RAX is one's complemented (64-bit mode)
NOT TEMP	The contents of data segment memory location TEMP is one's complemented
NOT BYTE PTR[BX]	The byte contents of the data segment memory location addressed by BX are one's complemented

Kayıdırma ve Döndürme

- Kaydırma ve döndürme talimatları ikili sayıları ikili bit düzeyinde işler. – AND, OR, Exclusive-OR ve NOT gibi
- G/Ç aygıtlarını kontrol etmek için kullanılan düşük seviyeli yazılımlardaki yaygın uygulamalar.
- Mikroişlemci tam bir
 - Herhangi bir bellek verisini veya kaydını kaydırmak veya döndürmek için kullanılan kaydırma ve döndürme talimatlarının tamamlayıcısı.

Vardiya

- Sayıları bir kayıt veya bellek konumu içinde sola veya sağa konumlandırın veya taşıyın . – ayrıca 2^n'nin kuvvetleriyle çarpma (sola kaydırma) ve 2'nin kuvvetleriyle bölme gibi basit aritmetik işlemleri gerçekleştirin
- Mikroişlemcinin talimat seti dört farklı kaydırma talimatı içerir:
 - ikisi mantıksal; ikisi aritmetik kaydırmalardır
- Dört kaydırma işleminin tamamı Şekil 5–9'da görülmektedir.

Şekil 5-9 Vites değişiminin işleyişini ve yönünü gösteren vites değiştirme talimatları.

Target register or memory

SHL C

SAL C

SHR 0

SAR Sign bit

– mantıksal kaydırmalar
 mantıksal sola kaydırma için en sağdaki bitte
 0 hareket eder; – mantıksal sağa kaydırma için en soldaki bit pozisyonuna 0 hareket eder
 – aritmetik sağa kaydırma işaret bitini sayı boyunca kopyalar
 – Mantıksal sağa kaydırma, sayının içinden 0'ı kopyalar.

Mantıksal kaydırmalar işaretsiz verileri çarpar veya böler; aritmetik kaydırmalar işaretli verileri çarpar veya böler. sola kaydırma, kaydırılan her bit konumu için her zaman 2 ile çarpar her zaman sağa doğru bir kayma her pozisyon için 2'ye böler iki basamak kaydırır, çarpar veya böler 4

SHL DX,14

OR

MOV CL,14

SHL DX,CL

Multiplies AX by 10 (1010)

```

SHL AX,1 ;AX times 2
MOV BX,AX
SHL AX,2 ;AX times 8
ADD AX,BX ;AX times 10
  
```

Multiplies AX by 18 (10010)

```

SHL AX,1 ;AX times 2
MOV BX,AX
SHL AX,1 ;AX times 16
ADD AX,BX ;AX times 18
  
```

Multiplies AX by 8 (1001)

```

MOV BX,AX
SHL AX,2 ;AX times 4
ADD AX,BX ;AX times 5
  
```

SHL

memory, immediate
REG, immediate

memory, CL
REG, CL

Shift operand1 Left. The number of shifts is set by operand2.

Algorithm:

- Shift all bits left, the bit that goes off is set to CF.
- Zero bit is inserted to the right-most position.

Example:
MOV AL, 11100000b
SHL AL, 1 ; AL = 11000000b, CF=L.

RET

CO

FF

OF=0 if first operand keeps original sign

6EM336-Mikroİşlemciler-İşletişim Programlama

Aritmetik ve Mantık Talimatları

70

- Bir döndürme sayımı anında yapılabilir veya şu konumda bulunabilir:
CL'yi kaydedin. – CL bir dönüş sayımı için kullanılırsa, değişmez
- Döndürme talimatları genellikle kaydırmak için kullanılır
sola veya sağa doğru geniş sayılar.

SHL AX, 1

RCL BX, 1

RCL DX, 1

Bu program, DX, BX ve AX kayıtlarındaki 48 bitlik sayıyı bir ikili yer sola kaydırır. En az önemli 16 bitin (AX) kaydırıldığını fark edin. İlk sola.

Bu, AX'in en soldaki bitini taşımaya taşır bayrak parçası.

Sonra, BX döndürme talimatı carry'yi BX'e döndürür ve en soldaki bit carry'e geçer. Son talimat carry'yi DX'e döndürür ve varidiya tamamlandı

6EM336-Mikroİşlemciler-İşletişim Programlama

Aritmetik ve Mantık Talimatları

74

Örnekler

TABLE 5-22 Example shift instructions.

Assembly Language	Operation
SHL AX,1	AX is logically shifted left 1 place
SHR BX,12	BX is logically shifted right 12 places
SHR ECX,10	ECX is logically shifted right 10 places
SHL RAX,50	RAX is logically shifted left 50 places (64-bit mode)
SAL DATA1,CL	The contents of data segment memory location DATA1 are arithmetically shifted left the number of spaces specified by CL
SHR RAX,CL	RAX is logically shifted right the number of spaces specified by CL (64-bit mode)
SAR SI,2	SI is arithmetically shifted right 2 places
SAR EDX,14	EDX is arithmetically shifted right 14 places

6EM336-Mikroİşlemciler-İşletişim Programlama

Aritmetik ve Mantık Talimatları

71

ROL

memory, immediate
REG, immediate

memory, CL
REG, CL

Rotate operand1 left. The number of rotates is set by operand2.

Algorithm:

- shift all bits left, the bit that goes off is set to CF and the same bit is inserted to the right-most position.

Example:
MOV AL, 1Ch ; AL = 00011100b
ROL AL, 1 ; AL = 00111000b, CF=0.

RET

CO

FF

OF=0 if first operand keeps original sign.

6EM336-Mikroİşlemciler-İşletişim Programlama

Aritmetik ve Mantık Talimatları

75

CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

Program Kontrol Talimatları

PEARSON

Intel Mikroİşlemciler: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro İşlemci, Pentium II, Pentium, 4 ve 64-bit Uzmanları ile Core2 Mimari, Programlama ve Arayüzleme, Sekizinci Baskı

Barry B. Brey

Telif Hakkı ©2009 Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • Tüm hakları saklıdır.

6EM336-Mikroİşlemciler-İşletişim Programlama

Aritmetik ve Mantık Talimatları

78

Program Akış Kontrolü

- Program akışını kontrol etmek çok önemlidir, programınızın kararlar alabileceği yer belirli koşullara dayalı.
- C programlama dilindeki if ifadesini hatırlayın .
- Assembly dilinde akış kontrolü çeşitli Atlama talimatları tarafından gerçekleştirilir şartlı ve şartsız atlayışlar dahil.

Döndürmek

- Bilgileri bir kayıt defterinde veya bellek konumunda bir uçtan diğerine veya taşıma bayrağı aracılığıyla döndürerek ikili verileri konumlandırır . – 16 bitten daha geniş sayıları kaydırmak/konumlandırmak için kullanılır • Her iki talimat türünde de programcı sola veya sağa döndürmeyi seçebilir.
- Rotate ile kullanılan adresleme modları shift ile kullanılanlarla aynıdır.
- Döndürme talimatları Şekil 5-10'da gösterilmektedir.

Örnekler

TABLE 5-23 Example rotate instructions.

Assembly Language	Operation
ROL SI,14	SI rotates left 14 places
RCL BL,6	BL rotates left through carry 6 places
ROL ECX,18	ECX rotates left 18 places
ROL RDX,40	RDX rotates left 40 places
RCR AH,CL	AH rotates right through carry the number of places specified by CL
ROR WORD PTR[BP],2	The word contents of the stack segment memory location addressed by BP rotate right 2 places

6EM336-Mikroİşlemciler-İşletişim Programlama

Aritmetik ve Mantık Talimatları

72

giriş

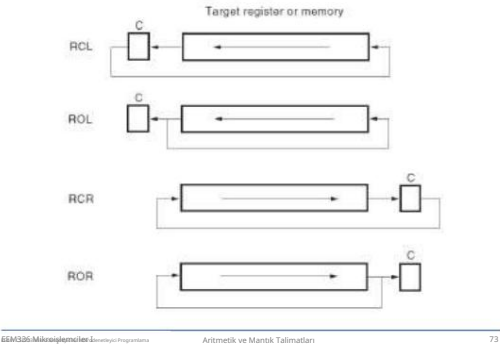
- Bu bölüm program kontrolünü açıkla
Atlamalar, döngüler, çağrılar ve dönüşler dahil olmak üzere talimatlar.

Etiketler Hakkında Daha Fazla Bilgi

- Etiket ayrı bir satırda veya herhangi bir diğer talimattan önce bildirilebilir
- Örnekler:
x1:
MOV AX, 1

x2: MOV AX, 2

Şekil 5-10 Her bir dönüş yönünü ve çalışmasını gösteren dönüş talimatları.



ÖZET

- Bayraklar
- Toplama, Çıkarma ve Karşılaştırma
- Çarpma ve Bölme
- BCD ve ASCII Aritmetiği
- Temel Mantık Talimatları
- Kaydırma ve Döndürme

Bölüm Hedefleri

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

- Bir programın akışını kontrol etmek için hem koşullu hem de koşulsuz atlama talimatlarını kullanın .
- Koşullu ve koşulsuz döngüleri kullanın
Bir programın akışını kontrol etmek.
- Arama ve geri dönüş talimatlarını kullanarak şunları ekleyin:
Program yapısındaki prosedürler.

JMP Örneği

```
ORG 100H
MOV AX, 5 ; AX'i 5'e ayarlayın.
MOV BX, 2 ; BX'i 2 olarak ayarlayın.
JMP hesaplaması ; 'calc'a git.
geri:
JMP durdurma ; 'dur'a git.
hesabi:
AX, BX ekle ; BX'i AX'a ekle.
JMP arka ; geri gitmek'.
durdurucu:
Geri dön ; işletim sistemine geri dön.
```

• JMP yönergesinin üç türü vardır:

1. **Kısa Atlama**
2. **Yakın Atlama**
3. **Uzak Atlama**

• **Kısa** ve **yakın** atlamalar **segment** içidir _____
zıplama ve **uzak** zıplama ise **segmentler arası** zıplamadır.

Yakın Atlama

- Yakın sıçrama, kısa sıçramaya benzer, ancak mesafe daha uzundur.
- Yakın **atlama**, kontrolü yakın atlama talimatından $\pm 32K$ bayt uzaklıkta bulunan geçerli kod segmentindeki bir talimata geçirir .
- Yakın sıçrama da bir **görelî** sıçramadır.

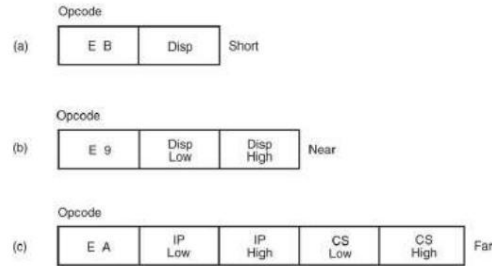
Koşullu Atlamalar

- Bir JMP talimatının aksine ,
koşulsuz atlama, koşullu atlamalar gerçekleştiren talimatlar vardır (sadece bazı koşullar doğru olduğunda atlama)
- Koşullu atlamalar 8086 - 80286 aralığında **her zaman kısa atlamalardır** .
- Bu talimatlar üç gruba ayrılır: - İlk grup sadece tek bayrağı test eder
- İkinci grup sayıları imzalandığı gibi karşılaştırır
- Üçüncü grup sayıları işaretlessiz olarak karşılaştırır.

İmzalı Sayılar İçin Atlama Talimatları

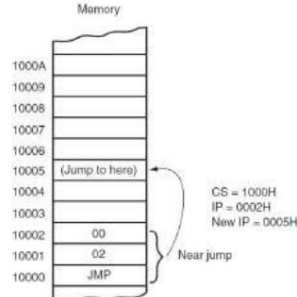
Talimat Açıklaması	Karşıt Durum	
JE, JZ	Eğitse atla (=) Sıfır ise atla	Z = 1 , JNE, JNZ
JNE, JNZ	Eğit Değilse Atla () Sıfır Değilse Atla	Z = 0 , JE, JZ
JG, JNLE	Daha Büyüğe Atla (>) Daha Az veya Egit Değilse Atla (<= değil)	Z = 0 , Ve S = 0 , JNG, JLE
JL, JNG	Daha Az (<) ise Atla Daha Büyük veya Egit Değilse Atla	S = 0 , JNL, JGE
JGE, JNL	Büyük veya Egitse Atla (>=) Daha Az Değilse Atla	S = 0 , JNG, JL
JLE, JNG	Daha Az veya Egitse Atla (<=) Daha Büyük Değilse Atla	Z = 1 , veya S = 0 , JNLE, JG

Atlama Türleri



Yakın Atlama Örneği

FIGURE 6-3 A near jump that adds the displacement (0002H) to the contents of IP.



Genel Sözdizimi

- Koşullu atlamalar için genel sözdizimi:
<Koşullu atlama talimatı> <etiket>
Örnek: **JC label1**
- Test altındaki koşul doğruysa, etikete bir dallanma meydana gelir.
- Koşul yanlış ise programdaki bir sonraki ardışık adım yürütülür.

İmzalanmamış Sayılar İçin Atlama Talimatları

Talimat	Tanımı	Koşul N	Zıt
JE, JZ	Eğitse atla (=) Sıfır ise atla	Z = 1 , JNE, JNZ	
JNE, JNZ	Eğit Değilse Atla () Sıfır Değilse Atla	Z = 0 , JE, JZ	
JA, JNBE	Yukarıdaki ise atla (>) Eğer Aşağı veya Egit Değilse Atla	C = 0 , Ve Ze = 0 , JNA, JBE	
JBE, JNA	Eğer Aşağıda veya Egitse Atla (<=) Üstünde Değilseniz Atlayın	C = 1 , veya Z = 1 , JNBE, JA	
JB, JNAE, JC	Aşağıdaysa Atla (<) Üstünde veya Egit Değilseniz Atlayın Taşıdığında Atla	C = 1 , JNB, JAE, JNC	
JAE, JNB, JNC	Üstte veya Egitse Atla (>=) Aşağıda Değilse Atla Taşımazsan Atla	C = 0 , JB, JNAE, JC	

Kısa Atlama

- Atlama adresi opcode'da saklanmaz.
- Bunun yerine, işlem kodunu bir mesafe veya yer değiştirme izler.
- Bu nedenle kısa sıçramalara aynı zamanda **bağıl** sıçramalar da denir. Atlamalar.
- Yer değiştirme, aralıkta bir değer alır
-128 ile +127 arasında .

Uzak Atlama

- Uzak **atlama** talimatı, atlamayı gerçekleştirmek için yeni bir segment ve ofset adresi alır.
- Uzak atlama talimatı bazen FAR PTR yönergesiyle birlikte görünür.
- Uzak bir sıçrama elde etmenin bir başka yolu, bir etiketi uzak etiket olarak tanımlamaktır . Bir etiket yalnızca geçerli kod parçasının veya prosedürün dışındaysa uzaktır (yani derleyici bunu otomatik olarak ayarlar).

Tek Bayrağı Test Eden Atlama Talimatları

Talimat	Tanım	Karşıt Durum
JZ, JE	Sıfır (Eşit) ise atla	Z = 1 , JNZ, JNE
JC, JB, JNAE	Taşıdığında Atla (Eğitin Altında, Üstünde Değil)	C = 1 , JNC, JNB, YAAA
J5	İşaret varsa atla	S = 1 , JNS
JO	Taşıma durumunda atla	O = 1 , JNO
JPE, JP	Parite Çift ise Atla	P = 1 , JPO, JNP
JNZ, JNE Sıfır Değilse Atla (Eşit Değil)		Z = 0 , JZ, JE
JNC, JNB, YAAA	Atla, Taşımazsan (Aşağıda Değil, Yukarıda Egitir) C = 0	JC, JB, JNAE
JNS	Atla Eğer İşaret Değilse	S = 0 , JS
JNO	Taşımazsa Atla	O = 0 , JO
JPO, JNP Parite Tek ise (Parite Yok) Atla		P = 0 , JPE, JP

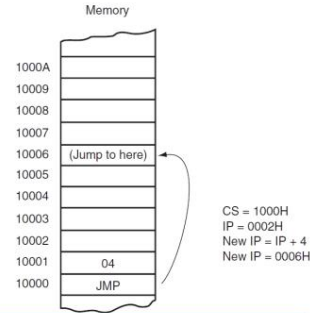
* Programların daha kolay anlaşılması için farklı isimler kullanılmıştır.

İmzalı mı İmzasız mı?

- Karşılaştırmadaki sayılar seçiminize göre işaretli veya işaretlessiz olabilir.
- İşaretli sayılar karşılaştırıldığında **"büyüktür"**, **"küçüktür"** vb. terimlerini içeren talimatları kullanın.
- İşaretsiz sayılar karşılaştırıldığında **"yukarıda"** ve **"aşağıda"** terimleriyle birlikte talimatları kullanın.

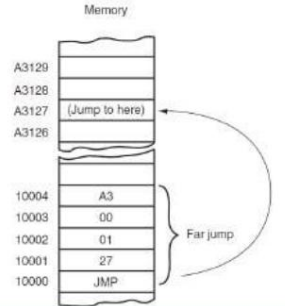
Kısa Atlama Örneği

FIGURE 6-2 A short jump to four memory locations beyond the address of the next instruction.



Uzak Atlama Örneği

FIGURE 6-4 A far jump instruction replaces the contents of both CS and IP with 4 bytes following the opcode.



Şekil 6-5 İşaretili ve işaretlessiz sayılar farklı sıraları takip eder.

Unsigned numbers	Signed numbers
255 FFH	+127 7FH
254 FEH	+126 7EH
132 84H	+2 02H
131 83H	+1 01H
130 82H	+0 00H
129 81H	-1 FFH
128 80H	-2 FEH
4 04H	-124 84H
3 03H	-125 83H
2 02H	-126 82H
1 01H	-127 81H
0 00H	-128 80H

Koşullu Atlama Örneği

ORG 100H
MOV AL, 25 ; **AL'yi 25 olarak ayarla.**
MOV BL, 10 ; **BL'yi 10 olarak ayarlayın.**
CMP AL, BL ; **AL - BL'yi karşılaştırın.**
JE eşittir ; **AL = BL (Z = 1) ise atla.**
MOV CX, 1 ; **eğer buraya gelirse, o zaman AL <> BL**
JMP stop ; **CX'i ayarlayıp stop'a geçiyoruz.**
eşit: MOV ; **eğer buraya gelirse,**
CX, 0 ; **o zaman AL = BL, dolayısıyla CX'i temizle.**
durmak:
Geri dön ; **ne olursa olsun buraya gelir.**

- Tüm koşullu sıçramaların, JMP'nin aksine, büyük bir sınırlaması vardır
talimat sadece 127 bayt ileri atlayabilirler ve 128 bayt geriye doğru.

- Bu sınırlamayı sevimli bir numarayla kolayca aşabiliriz:
 - Yukarıdaki tablodan zıt koşullu bir atlama talimatı alın, bunu label_x'e atlat .
 - İstenilen yere atlamak için JMP komutunu kullanın .
 - JMP talimatından hemen sonra label_x tanımlayın . - label_x:
 - herhangi bir geçerli etiket adı olabilir, ancak aynı ada sahip iki veya daha fazla etiket olmamalıdır.

Döngüler

Talimat	İşlem ve Atlama Durumu	Kayıt Yolu
DÖNGÜ	CX'i azaltın, CX sıfır değişse etikete atlayın	DEC CX ve JCKZ
DÖNGÜ	CX'i azaltın, CX sıfır değişse ve (Z = 1)'e eşitse etikete atlayın	DÖNGÜ
DÖNGÜ	CX'i azaltın, CX sıfır değişse ve eşit değişse (Z = 0) etikete atlayın	DÖNGÜ
DÖNGÜ	CX'i azalt, CX sıfır değişse ve Z = 0 ise etikete atla LOOPZ	
DÖNGÜ	CX'i azalt, CX sıfır değişse ve Z = 1 ise etikete atla LOOPNZ	
JCKZ	CX sıfırda etikete atla	VEYA CX, CX ve JNZ

İç İçe Döngü Örneği

```
org 100h
mov bx, 0 ; total step counter.

k1: mov cx, 5
    add bx, 1
    mov al, 1
    mov dx, 0eh
    jmp 10h
    push cx
    mov cx, 5
    k2: add bx, 1
        mov al, 0eh
        mov dx, 0eh
        int 10h
        push cx
        mov cx, 5
        k3: add bx, 1
            mov al, 0eh
            mov dx, 0eh
            int 10h
            loop k3 ; internal in internal loop.
        pop cx
        mov dx, k2
        loop k2 ; internal loop.
    pop cx
    mov dx, k1
    loop k1 ; external loop.

ret
```

Bir Prosedürü Çağırma

- CALL komutu bir prosedürü çağırarak için kullanılır.
- Örnek:

ORG 100H

m1'i arayın

MOV AX, 2

RET ; işletim sistemine geri dön.

m1 İŞLEM

MOV BX, 5

RET ; arayana geri dön.

m1 SON

SON

Örnek

```
include "emu8086.inc"
org 100h
mov al, 0
cmp al, bl ; compare al = bl.
; je equal ; there is only 1 byte

jne not_equal ; jump if al <> bl (zf = 0).
jmp equal
not_equal:

add bl, 01
sub al, 10
xor al, bl

jmp skip_data
db 256 dup(0) ; 256 bytes
skip_data:

putc 'n' stop ; if it gets here, then al <> bl,
; so print 'n', and jump to stop.
jmp equal:
putc 'y' ; if gets here,
; then al = bl, so print 'y'.
stop:
ret
```

Döngü Örneği: C Kodu

- İki dizinin içeriklerini toplayan ve sonucu ikinci diziye depolayan bir program yazın.

kisa arr1[100];

kisa arr2[100];

int sayim = 100;

int idx = 0;

(sayim > 0) iken {

arr2[idx] = arr1[idx] + arr2[idx];

idx++;

sayim--;

}

İŞLEMLER

- Bir prosedür, genellikle tek bir görevi gerçekleştiren bir talimat grubudur . - alt rutin, yöntem veya işlev bir herhangi bir sistemin mimarisinin önemli bir parçası
- Bir prosedür, bir kez bellekte saklanan ve yeniden kullanılabilen bir yazılım bölümüdür.
- gerektiği kadar sık. - bellek
- alanından tasarruf sağlar ve yazılım geliştirmeyi kolaylaştırır

ARAMA

- Program akışını prosedüre aktarır . • CALL talimatı atlama talimatından farklıdır. çünkü bir CALL yığında bir dönüş adresi kaydeder.
- İade adresi kontrolü geri verir hemen ardından gelen talimat Bir programda, prosedürün sonunda bir RET talimatı yürütüldüğünde CALL.

DÖNGÜ

- LOOP, JMP'ye benzer.
- CX'i azaltma ve JNZ koşullu atlamanın birleşimidir . • LOOP, CX'i azaltır. - CX != 0 ise, belirtilen adrese atlar.

etikete göre -
CX 0 olursa, bir sonraki ardışık talimat yürütür

Döngü Örneği: Montaj Kodu

```
org 100h
mov cx, 100 ; Bloklardaki eleman sayısı
hareketli bx, 0 ; Dizin başlat
L1:
mov ax, BLOCK1[bx] ; BLOCK1'den sonraki sayıyı oku
ax, BLOCK2[bx] ekle; BLOCK2'den sonraki sayıyı ekle
mov BLOCK2[bx], ax ; sonucu depola
bx'i ekle, 2 ; bir sonraki ögeye geç
döngü L1
Geri dön
BLOCK1 DW 100 ÇOĞALT (1)
BLOCK2 DW 100 ÇIFT (2)
```

Prosedür Sözdizimi

- isim PROC ; işte kod ; prosedürün ...
Geri dön
isim ENDP
- name - prosedürün adıdır
- aynı isim en üstte ve en altta olmalı alt kısımda prosedürlerin doğru bir şekilde kapatılıp kapatılmadığını kontrol etmek için kullanılır.

ARAMA

(devamı)

- CALL yapısı PUSH ve JMP komutlarının birleşimidir .
- CALL yürütüldüğünde, dönüş adresini yığına iter ve ardından prosedüre atlar. • Yakın CALL, IP'nin içeriğini yığına yerleştirir ve uzak CALL, hem IP'yi hem de CS'yi yığına yerleştirir.

Koşullu DÖNGÜLER

- LOOP komutunun da koşullu biçimleri vardır: DÖNGÜ ve DÖNGÜ
- LOOPE (eşitken döngü) talimatı, eşit bir koşul mevcutken CX != 0 ise atlar. - koşul eşit değilse veya CX kaydı 0'a düşer
- LOOPNE (eşit olmayan döngü) CX != 0 ise ve eşit olmayan bir koşul mevcutsa atlar . - koşul eşitse veya CX döngüden çıkar. kayıt 0'a düşer

İç içe döngüler

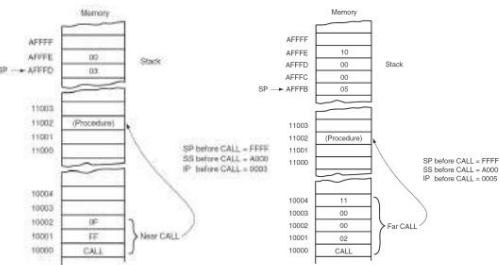
- Tüm döngü talimatları adımları saymak için CX kaydını kullanır , bildiğiniz gibi CX kaydının 16 bitli vardır ve tutabileceği maksimum değer 65535 veya FFFF'dir.
- Ancak biraz çeviklikle bir döngüyü diğerine yerleştirmek ve 65535 x 65535 x gibi güzel bir değer elde etmek mümkündür. 65535sonsuz kadar....veya ram'in veya yığın belleğinin sonuna kadar.

- PUSH kullanarak CX kaydının orijinal değerini depolamak mümkündür CX talimatını çalıştırın ve dahili döngü POP CX ile sona erdiğinde orijinaline döndürün

Prosedür Örneği

TOPLAM İŞLEMLERİ	
	AX, BX'i ekleyin
	AX, CX'i ekleyin
	AX, DX'i ekleyin
Geri dön	
TOPLAMLAR SONDP	

Yakın ve Uzak ÇAĞRI Örnekleri



- RET talimatı bir Yığından dönüş adresini kaldırarak ve onu IP'ye (near return) veya IP ve CS'ye (far return) yerleştirerek yapılan bir işlem .

Örnek Hakkında

- Program 24'ü hesaplar
- Prosedür parametrelerini AL'den **alır** ve **BL**; bunları çarpar ve sonucu **AX'e** depolar
- C eşdeğeri aşağıdaki şekilde düşünülebilir:

```
kısa m2(char al, char bl)
{
    kısa balta = al * bl;
    baltaı geri getir;
}
```

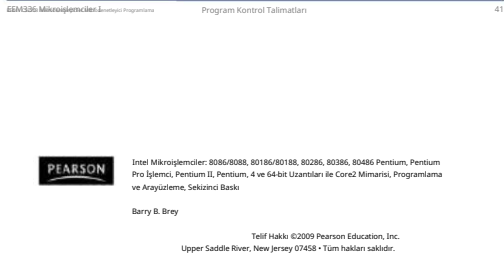
Bölüm Hedefleri

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

- Bellek adresini çözün ve kod çözücünün çıktılarını kullanarak çeşitli bellek bileşenlerini seçin.
- Bellek adreslerini çözmek için programlanabilir mantık aygıtlarını (PLD'ler) kullanın.
- Hata düzeltme kodunun (ECC) nasıl çalıştığını açıklayın hafıza ile kullanılır.

- Bir bellek aygıtındaki adres pinlerinin sayısı, içinde bulunan bellek konumlarının sayısına göre belirlenir.
- Günümüzde bellek aygıtları 1K'dan başlayıp G'lara kadar uzanan bellek konumlarına sahiptir.
- 1K bellek aygıtında 10 adres pimi vardır. – bu nedenle, 10 adres girişi gereklidir. 1024 bellek konumundan herhangi birini seçin

- Bir prosedüre parametre geçirmenin birkaç yolu vardır
- En kolay yol kayıt defterlerini kullanmaktır
 - Kayıtlara parametreler koyun ve prosedürü çağırın
 - Prosedür, bu araçları kullanacak şekilde tasarlanmalıdır. kayıtlar
- Başka bir yol da yığın kullanmaktır
 - Parametreleri yığına itin ve prosedürü çağırın
 - Prosedür, yığından parametre alacak şekilde tasarlanmalıdır
- Her iki durumda da programcı, prosedürün yapısı



BELLEK AYGITLARI

- Belleği mikroişlemciye bağlamaya çalışmadan önce , bellek bileşenlerinin işleyişini anlamak önemlidir . • Bu bölümde, bellek bileşenlerinin işlevlerini açıklıyoruz.

dört yaygın bellek türü: – Salt Okunur Bellek (ROM)
– Flaş Bellek (EEPROM)
– Statik Rasgele Erişimli Bellek (SRAM)
– Dinamik Rastgele Erişimli Bellek (DRAM)

- 1024 konumlu bir aygıtta herhangi bir tek konumu seçmek için 10 bitlik bir ikili sayı gerekir.
 - 1024 farklı kombinasyon
 - eğer bir cihazın 11 adres bağlantısı varsa, 2048 (2K) dahili bellek konumuna sahiptir
- Bellek lokasyonlarının sayısı pin sayısından tahmin edilebilir.

Bir Prosedüre Parametre Geçirme

- Bir prosedüre parametre geçirmenin birkaç yolu vardır
- En kolay yol kayıt defterlerini kullanmaktır
 - Kayıtlara parametreler koyun ve prosedürü çağırın
 - Prosedür, bu araçları kullanacak şekilde tasarlanmalıdır. kayıtlar
- Başka bir yol da yığın kullanmaktır
 - Parametreleri yığına itin ve prosedürü çağırın
 - Prosedür, yığından parametre alacak şekilde tasarlanmalıdır
- Her iki durumda da programcı, prosedürün yapısı

CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

Bellek Arayüzü

Bellek Pin Bağlantıları



Şekil. Adres, veri ve kontrol bağlantılarını gösteren bir sözde bellek bileşeni.

Veri Bağlantıları

- Tüm bellek aygıtlarının bir veri kümesi vardır çıktılar veya giriş/çıkışlar. – bugün, birçok cihazda çift yönlü ortak G/Ç pinleri
 - veri bağlantıları, verilerin depolama için girildiği veya okuma için çıkarıldığı noktalardır
- Bellek aygıtlarındaki veri pinleri D0 olarak etiketlenir 8 bit genişliğinde bir bellek aygıtı için D7'ye kadar .

Örnek

```
ORG 100h
MOV AL, 1
MOV BL, 2
m2'yi arayın
m2'yi arayın
m2'yi arayın
m2'yi arayın
RET ; işletim sistemine geri dön.

m2 İŞLEM
MUL BL ; AX = AL * BL.
RET ; arayaana geri dön.

m2 SON
SON
```

giriş

- Basit veya karmaşık, her mikroişlemci tabanlı sistemde bir bellek sistemi vardır. • Hemen hemen tüm sistemler iki ana bellek türü içerir: salt okunur bellek (ROM) ve rasgele erişimli bellek (RAM) veya okuma/yazma belleği.

- Bu bölümde her iki bellek türünün Intel mikroişlemci ailesine nasıl bağlanacağı açıklanmaktadır.

Adres Bağlantıları

- Bellek aygıtlarının adres girişleri vardır
 - Aygıt içinde bir bellek konumu seçin. • Neredeyse her zaman en az önemli adres girişi olan A0'dan An'a kadar etiketlenir – burada n herhangi bir değer olabilir
 - her zaman toplam sayıdan bir eksik olarak etiketlenir adres pinlerinin
- 10 adres pinine sahip bir bellek aygıtı Adres pinleri A0'dan A9'a kadar etiketlenmiştir .

- 8 bit genişliğindeki bir bellek aygıtına genellikle bayt genişliğinde bellek denir.
 - çoğu aygıt şu anda 8 bit genişliğindedir, – bazıları 16 bit, 4 bit veya sadece 1 bit genişliğindedir
- Bellek aygıtlarının katalog listeleri genellikle bellek konumlarını konum başına bit sayısıyla ifade eder. – 1K bellek konumuna sahip bir bellek aygıtı ve her konumdaki 8 bit genellikle şu şekilde listelenir: üretici tarafından 1K 8
- Bellek aygıtları genellikle toplam bit kapasitesine göre sınıflandırılır.

- Her bellek aygıtının, bellek aygıtını seçen veya etkinleştiren bir giriş vardır. - bazen birden fazla
- Bu tür girdilere çoğunlukla çip denir
(CS) çipi etkinleştirir (CE) seçeneğini seçin veya sadece seçin (S) girişi.
- RAM belleği genellikle en az bir CS veya S girişine sahiptir ve ROM en az bir CE'ye sahiptir
- Birden fazla CE bağlantısı mevcutsa, Veri okumak veya yazmak için hepsinin aktif edilmesi gerekir.

Veri sayfaları

- IC'ler hakkında detaylı bilgi alabilirsiniz
sadece veri sayfalarına bakarak hafıza birimlerini de dahil ederek.
- Her üretici, ürünlerinin operasyonel bilgilerini içeren veri sayfalarını yayınlar.

- Daha yeni bir tür çoğunlukla okunan bellek (RMM) flaş belleğe denir . - ayrıca sıklıkla EEPROM (elektriksel olarak silinebilir programlanabilir ROM) olarak da adlandırılır
 - EARAM (elektriksel olarak değiştirilebilir ROM) - veya NOVRAM (kalıcı olmayan RAM) • Sistemde elektriksel olarak silinebilir, ancak bunlar normal RAM'den daha fazla zaman gerektirir.
- Flash bellek aygıtı, bilgisayardaki ekran kartı gibi sistemlere ait kurulum bilgilerinin saklanması için kullanılır .

Dinamik RAM (DRAM) Bellek

- 256M 8'e kadar (2G bit) mevcuttur. • DRAM esasen SRAM ile aynıdır, ancak verileri yalnızca 2 veya 4 ms boyunca saklar entegre bir kondansatör üzerinde.
- 2 veya 4 ms sonra, DRAM'ın içeriği tamamen yeniden yazılmalıdır (yenilenmelidir). – çünkü bir mantık 1 depolayan kapasitörler veya mantık 0, yüklerini kaybederler

Kontrol Bağlantıları

- Tüm bellek aygıtlarının bir tür kontrol girişi veya girişleri vardır.
 - ROM genellikle bir kontrol girişine sahipken, RAM genellikle bir veya iki kontrol girişine sahiptir
- ROM üzerinde sıklıkla bulunan kontrol girişi, çıkış etkinleştirme veya kapı bağlantısıdır ve çıkış verileri pinlerinden veri akışına izin verir.
- OE bağlantısı bir OE bağlantısını etkinleştirir ve devre dışı bırakır. Cihazda bulunan ve veri okumak için aktif olması gereken üç durumlu tampon kümesi.

ROM Belleği

- Salt okunur bellek (ROM) kalıcı olarak
- Sistemde yerleşik programları/verileri depolar. – ve güç kesildiğinde değişmemelidir. • Genellikle kalıcı bellek olarak adlandırılır, çünkü güç kesildiğinde bile içeriği değişmez .

- ROM adını verdiğimiz bir cihaz, üreticiden toplu miktarda satın alınır.
 - fabrikada üretim sırasında programlanır

- Flash, çoğu bilgisayar sisteminde BIOS için EPROM'un yerini almıştır. – bazı sistemlerde saklanan bir parola vardır
- flaş bellek aygıtında
- Flash belleğin en büyük etkisi dijital fotoğraf makinelerindeki hafıza kartlarında ve MP3 çalarlardaki hafızada görülmektedir.

- DRAM'da tüm içerik 2 veya 4 ms'lik aralıklarla 256 okumayla yenilenir.
 - ayrıca bir yazma, okuma veya özel bir yenileme döngüsü sırasında da meydana gelir
- DRAM, üreticilerin çok sayıda adres pini gerektirmesi nedeniyle çoklu adres girişleri üretir. • DRAM genellikle SIMM (Tek Sıralı Bellek Modülleri) veya DIMM (Çift Sıralı Bellek Modülleri) adı verilen küçük kartlara yerleştirilir.

- RAM'de bir veya iki kontrol girişi vardır. – bir kontrol girişi varsa, genellikle RW olarak adlandırılır. • RAM'de iki kontrol girişi varsa, genellikle WE (veya W) ve OE (veya G) olarak etiketlenir.
- bellek yazımı gerçekleştirmek için yazma etkinleştirmesi etkin olmalı ve bellek okuması gerçekleştirmek için OE etkin olmalıdır.

girişi varsa, genellikle WE (veya W) ve OE (veya G) olarak etiketlenir.

- bellek yazımı gerçekleştirmek için yazma etkinleştirilmesi etkin olmalı ve bellek okuması gerçekleştirmek için OE etkin olmalıdır.
- iki kontrol mevcut olduğunda, her ikisi de aynı anda aktif olmamalıdır

- Her iki girdi de etkin değilse, veriler hiçbirini ne yazıldı ne de okundu.
 - bağlantılar yüksek empedans durumundadır

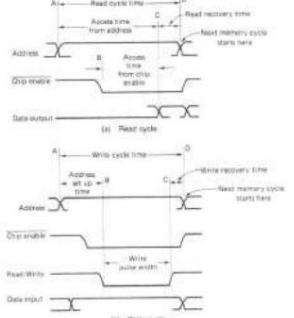
- EPROM (silinebilir programlanabilir salt okunur bellek), yazılımların sık sık değiştirilmesi gerektiğinde yaygın olarak kullanılır.
 - veya düşük talep ROM'u ekonomik olmaktan çıkardığında
 - ROM'un pratik olabilmesi için en az 10.000 cihaza ihtiyaç vardır fabrika masraflarını karşılamak için satılması gerekiyor
- EPROM, EPROM programlayıcı adı verilen bir cihaz üzerinde sahada programlanır.
- Ayrıca yüksek yoğunluklu ultraviyole ışığa maruz kaldığında silinebilir. – EPROM türüne bağlı olarak

Statik RAM (SRAM) Aygıtları

- Statik RAM bellek aygıtları verileri tutar
DC güç uygulandığı sürece.
- Verileri saklamak için özel bir işlem gerekmediğinden, bu aygıtlara statik bellek denir. – ayrıca geçici bellek olarak da adlandırılır çünkü bunlar güç olmadan veriyi saklama
- ROM ve RAM arasındaki temel fark
RAM'ın normal çalışma sırasında yazıldığı, ROM'un ise bilgisayar dışında programlandığı ve normalde sadece okunabildiğidir.

- Bir diğer tür ise RAMBUS Corporation'ın RIMM bellek modülüdür; bu bellek türü piyasadan çekilmiştir.
- En son DRAM, DDR (çift veri hızı) bellek aygıtı ve DDR2'dir.
 - DDR, saatin her iki ucunda veri aktarımı yapar, SDRAM'ın iki katı hızda çalışmasını sağlıyor.

Örnek Bellek Okuma/Yazma Zamanlama Döngüleri



- Günümüzde çok yaygın olmasa da PROM bellek aygıtları da mevcuttur.
- PROM (programlanabilir salt okunur) (hafıza) da sahada açıkta bulunan küçük Ni-krom veya silisyum oksit sigortaların yakılmasıyla programlanır.
- Bir kez programlandıktan sonra silinemez.

- Tipik bir 4016 SRAM'deki erişim süresi 250 ns'dir; bu, bekleme durumları olmadan 5 MHz'de doğrudan bir 8088/8086'ya bağlanmak için yeterince hızlıdır .
 - Bellek bileşenlerinin uyumluluğunu belirlemek için erişim süresi her zaman kontrol edilmelidir
 - Erişim süreleri 1,0 ns kadar düşük olabilir
- Bilgisayar ön belleğinde kullanılan SRAM

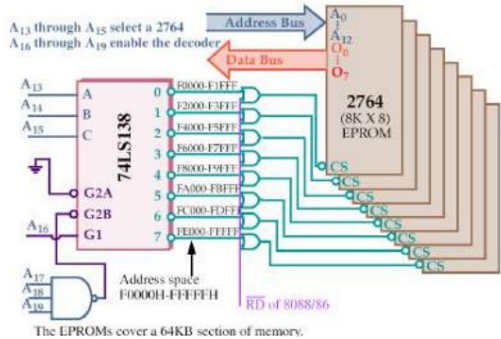
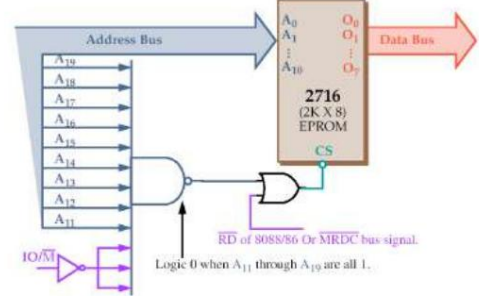


Ortak DRAM paketleri

Yukarıdan aşağıya: DIP,
SIPP, SIMM (30-pin),
SIMM (72-pin), DIMM
(168-pin), DDR DIMM
(184-pin).

- SRAM statiktir, DRAM ise dinamiklidir
- SRAM, DRAM'a kıyasla daha hızlıdır
- SRAM, DRAM'den daha az güç tüketir
- SRAM, DRAM'a kıyasla bellek biti başına daha fazla transistör kullanır
- SRAM, DRAM'dan daha pahalıdır
- Ana bellekte daha ucuz DRAM kullanılırken SRAM genellikle önbellek belleğinde kullanılır

Şekil. FF800H- FFFFFH bellek konumu için 2716 EPROM'u seçen basit bir NAND kapısı kod çözücüsü



Hata Tespiti ve Düzeltme

- Parite, BCC (Blok Kontrol Karakteri) ve CRC (Döngüsel Artıklik Denetimi) yalnızca hata tespiti için kullanılan mekanizmalardır.
- Bellekte bir hata bulunması durumunda sistem durdurulur.
- Yeni sistemlerde hata düzeltme özelliği görülmeye başlanıyor.
- SDRAM'de ECC (Hata Düzeltme Kodu) bulunur .
- Düzeltme, sistemin çalışmaya devam etmesini sağlayacaktır .
- İki hata meydana gelirse, bunlar tespit edilebilir ancak düzeltilir.
- Hata düzeltilmenin maliyeti elbette daha fazla olacaktır.
ekstra parçalar.

ADRES KOD ÇÖZME

- Mikroişlemciye bir bellek aygıtı bağlamak için , kod çözme işlemi yapılması gerekir.
mikroişlemciden gönderilen adres.
- Kod çözme, hafızanın işlevini hızlandırır
bellek haritasının zensersiz bölümü veya bölümü.
- Adres kod çözücüsü olmadan yalnızca bir
bellek aygıtı bir mikroişlemciye bağlanabilir, bu
da onu
neredeyse işe yaramaz.

- 20 bitlik ikili adres,
NAND kapısı, en soldaki 9 bit 1'ler, en sağdaki 11 bit ise
önemsiz (X) olacak şekilde yazıldığından, EPROM'un gerçek
adres aralığı belirlenebilir.
- umursamamak mantıksal 1 veya mantıksal 0'dir, hangisi olursa olsun
uygundur

PLD Programlanabilir Kod Çözücüler

- Üç SPLD (basit PLD) cihazı aynı şekilde çalışır ancak farklı isimlere sahiptir:
 - PLA (programlanabilir mantık dizisi)
 - PAL (programlanabilir dizi mantığı)
 - GAL (kapalı dizi mantığı)
- 1970'li yılların ortalarından beri varlığını sürdüren bu teknolojiler, 1990'lı yılların başından itibaren bellek sistemlerinde ve dijital tasarımlarda yer almaya başlamıştır .

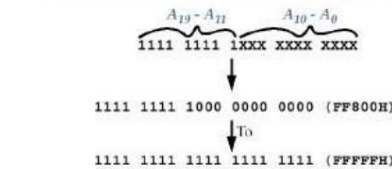
Hata Düzeltme

- Hata düzeltme Hamming kodlarına dayanmaktadır
- Hata tespiti ve düzeltme
- Büyük olasılıkla bir bit hatası varsayılır
- Hata ve pozisyonu algılar
bu nedenle düzeltmeye izin verir

Neden Hafızayı Çözmeliyiz?

- 8088'in 20 adres bağlantısı vardır ve 2716 EPROM'un 11 bağlantısı vardır.
- 8088 20 bitlik bir bellek gönderir
Veri okurken veya yazarken her zaman adresi belirtir.
 - çünkü 2716'nın sadece 11 adres pini var, düzeltilmesi gereken bir uyumsuzluk var
- Kod çözücü, uyumsuzluğu şu şekilde düzeltir: bağlanmayan adres pinlerini kod çözme bellek bileşenine.

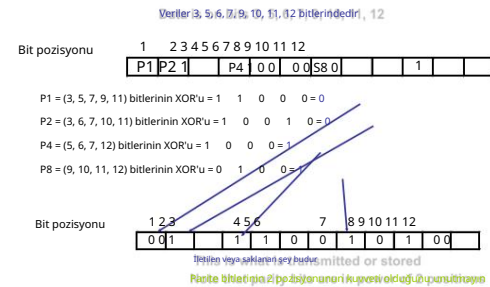
To determine the address range that a device is mapped into:



- NAND gate decoders are not often used
 - Large fan-in NAND gates are not efficient
 - Multiple NAND gate IC's might be required to perform such decoding
 - Rather the 3-to-8 Line Decoder (74LS138) is more common.

- PAL ve PLA sigorta programlı, bazı PLD aygıtları ise silinebilir aygıtlardır.
 - hepsi programlanabilir mantık elemanlarının dizileridir • Mevcut diğer PLD'ler:
 - CPLD'ler (karmaşık programlanabilir mantık aygıtları)
 - FPGA'lar (sahada programlanabilir kapı dizileri)
 - FPIC'ler (sahada programlanabilir ara bağlantı) • Bu
- PLD'ler,
- Tasarımda daha yaygın olarak kullanılan SPLD'ler tam bir sistem.

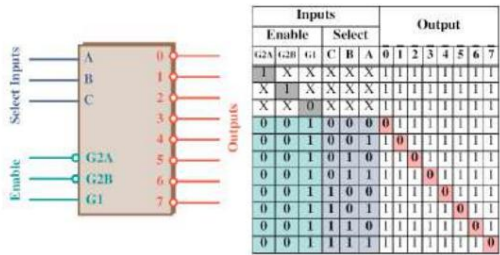
Hamming Kod Örneği



Basit NAND Kapısı Kod Çözücü

- 2K 8 EPROM kullanıldığında, 8088'in A10-A0 adres bağlantıları bağlanır
EPROM'un A10-A0 girişlerini adreslemek için . - kalan dokuz adres pimi (A19-A11)
bir NAND kapısı kod çözücüsüne bağlanır
- Kod çözücü EPROM'u aşağıdakilerden birinden seçer:
8088 mikroişlemcisindeki 1M-bayt bellek sisteminin 2K-bayt bölümleri .
- Bu devrede, şekilde görüldüğü gibi, bir NAND kapısı bellek adresini çözer.

3 ila 8 Satırlı Kod Çözücü (74LS138)



Note that all *three* Enables (G2A, G2B, and G1) must be active, e.g. low, low and high, respectively.
Each output of the decoder can be attached to an 2764 EPROM (8K X 8).

- Eğer odak noktamız adresleri çözmekse, SPLD kullanılır.
- Eğer yoğunlaşma komple bir sistem üzerinde ise, tasarımı uygulanmasında CPLD, FPGA veya FPIC kullanılır.
- Bu cihazlara ASIC (Uygulamaya Özel Entegre Devre) adı da verilir.

Hamming Kod Örneği



Bellekten alınan veya okunan veriler doğru değilse correct

Bit pozisyonu 2 3 4 5 6 7 8 9 10 11 12

0	0	1	1	1	0	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---

Yanlış okunursa (bit 3'ten 0'a değiştirin) 1 to 0)

Bit pozisyonu

1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	1	1	1	0	0	1	0	1	0

4 kontrol biti aşağıdaki gibi değeri takip eder: follows:

C1 = (1, 3, 5, 7, 9, 11) bitlerinin XOR'u = 0 0 1 0 0 0 = 1

C2 = (2, 3, 6, 7, 10, 11) bitlerinin XOR'u = 0 0 0 0 1 0 = 1

C4 = (4, 5, 6, 7, 12) bitlerinin XOR'u = 1 1 0 0 0 0 = 0

C8 = (8, 9, 10, 11, 12) bitlerinin XOR'u = 1 0 1 0 0 0 = 0

0 0 1 1
Error
in bit 3

Bölüm Hedefleri

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

- Temel giriş ve çıkış arayüzlerinin çalışmasını açıklayın.
- G/Ç talimatlarını kullanın.
- El sıkışmayı tanımlayın ve nasıl kullanılacağını açıklayın I/O aygıtlarıyla.

G/Ç Talimatları

- Bir talimat türü bilgiyi aktarır
 - Bir G/Ç aygıtına (ÇIKIŞ) • Bir
- diğeri bir G/Ç aygıtından (GİRİŞ) okur.
- Ayrıca bellek ile G/Ç arasında veri dizilerinin aktarılmasına yönelik talimatlar da sağlanır.
 - 8086/8088 hariç, GİRİŞLER ve ÇIKIŞLAR bulund

- 16 bitlik değişken port numarası (DX)
 - A15–A0 adres bağlantılarında görünür . • İlk 256 G/Ç bağlantı noktası adresine (00H–FFH) hem sabit hem de değişken G/Ç talimatları tarafından erişilir. – 0100H'den FFFFH'ye kadar herhangi bir G/Ç adresi
- yalnızca değişken G/Ç adresi (aracılığıyla) tarafından erişilir (DX)
- Bir PC bilgisayarında, 16 adres veri yolu bitinin tamamı 0000H–03FFH konumlarıyla kod çözülür.
 - ISA üzerindeki PC içindeki G/Ç için kullanılır (endüstri standardı mimarisi) otobüs

PEARSON

Intel Mikroİşlemciler: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro İşlemci, Pentium II, Pentium, 4 ve 64-bit Uzunları ile Core2 Mimarisi, Programlama ve Arayüzleme, Sekizinci Baskı

Barry B. Brey

Telif Hakkı ©2009 Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • Tüm hakları saklıdır.

Bölüm Hedefleri

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

- Seri ve seri olmayan arasındaki farkları öğrenin paralel iletişim.
- Harici cihazlara nasıl arayüz oluşturulacağını anlayın I/O portları aracılığıyla mikroİşlemciye.
- Emülatörle birlikte verilen sanal aygıtları kullanın.

- Bir G/Ç aygıtı ile mikroİşlemcinin akümülatörü (**AL, AX veya EAX**) arasında veri aktarımı yapan talimatlara GİRİŞ ve ÇIKIŞ adı verilir. • G/Ç adresi, DX kayıt defterinde 16 bitlik bir adres olarak veya işlem kodunun hemen ardından gelen baytta (p8) 8 bitlik bir adres olarak saklanır.
- 16 bitlik adrese değişken denir Adres , DX'te depolandığı ve daha sonra G/Ç aygıtını adreslemek için kullanıldığı için.

- INS ve OUTS talimatları bir G/Ç'yi ele alır DX kaydını kullanan aygıt.
 - ancak akümülatör arasında veri aktarımı yapmayı ve G/Ç aygıtı IN/OUT talimatlarında olduğu gibi
 - Bunun yerine, verileri bellek ile diğer bellekler arasında aktarırlar ve G/Ç aygıtı
- 64-bit modunda çalışan Pentium 4 ve Core2 aynı G/Ç talimatlarına sahiptir.
- 64 bit modunda 64 bit G/Ç talimatları yoktur .
 - G/Ç'nin çoğu hala 8 bittir ve büyük ihtimalle öyle kalacaktır

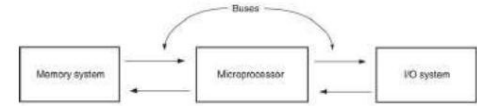
CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

G/Ç Arayüzü

G/Ç ARAYÜZÜNE GİRİŞ

- Mikroİşlemciler sorunları çözmeye harikadır.
- Ancak dışarıyla iletişim kuramıyorsa dünyada pek kıymeti yoktur.
- Bu nedenle I/O arayüzü büyük önem taşımaktadır.



- G/Ç'yi ele almak için DX kullanan diğer talimatlar INS ve OUTS talimatlarıdır.
- G/Ç portları 8 bit genişliğindedir.
 - 16 bitlik bir port aslında iki ardışık 8 bitlik porttur adreslenen limanlar
 - 32 bitlik bir G/Ç portu aslında dört adet 8 bitlik porttur

G/Ç talimatlarının özeti

- AL'DE, s8
- AX'TE, s8
- AL'DE, DX
- AX'TE, DX
- DIŞARI p8, AL
- DIŞARI p8, AX
- DIŞ DX, AL
- DIŞ DX, AX

p8: 0 ile 255 arasında bir port numarası
Diğer port numaraları için DX kayıt defterini kullanın.

giriş

- Temel G/Ç arayüzü
- El sıkışma süreci
- Seri ve Paralel iletişim
- G/Ç arayüzü örnekleri

G/Ç ARAYÜZÜNE GİRİŞ

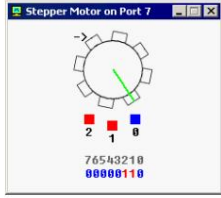
- G/Ç talimatları (IN, INS, OUT ve OUTS) Bu derste anlatılmaktadır.
- Ayrıca izole edilmiş (doğrudan veya G/Ç eşlemeli G/Ç) ve bellek eşlemeli G/Ç, temel giriş ve çıkış arayüzleri ve el sıkışma.
- Bu konuların bilinmesi, programlanabilir arayüz bileşenlerinin bağlantısını ve çalışmasını anlamayı kolaylaştırır
- ve G/Ç teknikleri.

- Veriler GİRİŞ veya ÇIKIŞ kullanılarak aktarıldığında, G/Ç adresi (port numarası veya basitçe port) adres veri yolunda görünür. • Harici G/Ç arabirimi, portu kod çözer bellek adresiyle aynı şekilde numara. – 8 bitlik sabit port numarası (p8), bitlerle A7–A0 adres veri yolu bağlantılarında görünür
- A15–A8 00000002'ye eşittir – A15'in üzerindeki bağlantılar tanımsızdır G/Ç talimatı

İzole (doğrudan) ve Bellek Eşlenen G/Ç

- G/Ç'yi arayüzlemenin iki farklı yöntemi: izole edilmiş G/Ç ve belleğe eşlenmiş G/Ç. • İzole edilmiş G/Ç'de, GİRİŞ, GİRİŞLER, ÇIKIŞ ve ÇIKIŞLAR mikroİşlemcinin akümülatörü veya belleği ile G/Ç aygıtı arasında veri aktarımı yapar. • Belleğe eşlenmiş G/Ç'de, belleğe başvuran herhangi bir talimat aktarımı gerçekleştirilebilir .
- Bilgisayar bellek eşlemeli G/Ç kullanmaz.

Adım motoru



- Adım motoru, I/O portu 7'ye veri gönderilerek kontrol edilir.
- Step motor, bilgisayardan gelen sinyallerle hassas bir şekilde kontrol edilebilen elektrik motorudur.
- Motor, bir sinyal aldığı anda her seferinde belirli bir açıyla döner.

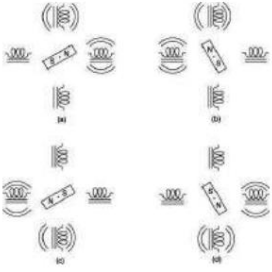
Adım motoru

Dört Bobin

Bir Step Motoru kullanan

Tek Kutuplu

Armatür



Robot

- Üçüncü bayt (port 11) bir durum kayıdır. • Durumu belirlemek için bu porttan değerleri okuyun. robot.

- Her bitin belirli bir özelliği vardır:

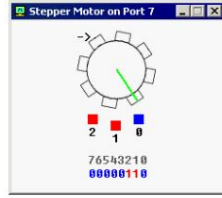
bit number	description
bit #0	zero when there is no new data in data register, one when there is new data in data register
bit #1	zero when robot is ready for next command, one when robot is busy doing some task.
bit #2	zero when there is no error on last command execution, one when there is an error on command execution (when robot cannot complete the task: move, turn, examine, switch on/off lamp).

CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

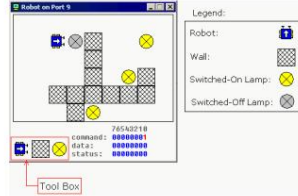
Kesintiler

Adım motoru



- Oranı değiştirerek sinyal darbeleri üretilir, motor farklı hızlarda çalıştırılabilir hızların veya belirli bir açıyla döner ve sonra durur.
- Bu temel bir 3 fazlı adım motorudur, 0, 1 ve 2 bitleri tarafından kontrol edilen 3 mknatısı vardır. Diğer bitler (3..7) kullanılmaz.

Robot



Robot, I/O portu 9'a veri gönderilerek kontrol ediliyor.

Robot

- Örnek:

```
MOV AL, 1 ; ilerlemek.
OUT 9, AL ;
MOV AL, 3 ; sağa dön.
OUT 9, AL ;
MOV AL, 1 ; ilerlemek.
OUT 9, AL ;
MOV AL, 2 ; sola dön.
OUT 9, AL ;
MOV AL, 1 ; ilerlemek.
MOV AL, 1 ; ilerlemek.
ÇIKIŞ 9, AL ;
```

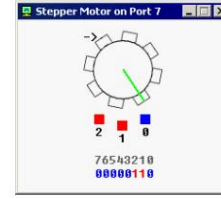
giriş

- Bu bölümde, kesintili işlenmiş G/Ç adı verilen bir teknik incelenerek temel G/Ç ve programlanabilir çevre birimleri arayüzlerinin kapsamı genişletilmektedir.

- Bir kesme, şu anda yürütülmekte olan herhangi bir programı kesen donanım tarafından başlatılan bir prosedürdür. • Bu bölüm,

Intel mikroişlemci ailesinin kesme yapısına ilişkin örnekler ve ayrıntılı bir açıklama sağlar.

Adım motoru



- Mknatıs çalıştığında kırmızı renk alır.
- Sol üstteki ok köşe son motor hareketinin yönünü gösterir.
- Yeşil çizginin burada olmasının sebebi, gerçekten döndüğünü görmektir.

Robot

- İlk bayt (port 9) bir komut kayıdır. • Robotun bir şey yapmasını sağlamak için bu porta veri gönderin.

decimal value	binary value	action
0	00000000	do nothing.
1	00000001	move forward.
2	00000010	turn left.
3	00000011	turn right.
4	00000100	examine. examines an object in front using sensor. when robot completes the task, result is set to data register and bit #0 of status register is set to 1.
5	00000101	switch on a lamp.
6	00000110	switch off a lamp.

Robot

- Robotun mekanik bir yaratık olduğunu ve bir görevi tamamlamasının biraz zaman aldığını unutmayın. • Durum kaydının bit#1'ini her zaman kontrol etmelisiniz. 9 numaralı porta veri göndermeden önce.
- Aksi takdirde robot komutunuzu reddedecek ve "meşgul!" mesajı gösterilecektir. • El sıkışma sürecini hatırlayın!

Bölüm Hedefleri

Bu bölümü tamamladıysanız şunları yapabileceksiniz:

- Intel mikroişlemci ailesinin kesme yapısını açıklayın.
- INT, INTO, IRET yazılım kesme talimatlarının işleyişini açıklayın
- Kesme etkinleştirme bayrağı bitinin (IF) kesme yapısını nasıl değiştirdiğini açıklayın.
- Emülatörün desteklediği kesmeleri öğrenin.

Adım motoru

- Mknatıslarla oynadıysanız nasıl çalıştığını anlamışsınızdır.
- Mknatısları belirli bir sıraya göre aktive etmeniz yeterli.
- Örneğin, aşağıdaki kod saat yönünde üç yarım adım yapacaktır:


```
MOV AL, 001b ; başlat
7, AL DIŞARI
MOV AL, 011b ; yarım adım 1
7, AL DIŞARI
MOV AL, 010b ; yarım adım 2
7, AL DIŞARI
MOV AL, 110b ; yarım adım 3
7, AL DIŞARI
```

Robot

- İkinci bayt (port 10) bir veri kayıdır. • Bu kayıt, robot incelemeyi tamamladıktan sonra ayarların emretmek:

decimal value	binary value	meaning
255	11111111	wall
0	00000000	nothing
7	00000111	switched-on lamp
8	00001000	switched-off lamp



Intel Mikroişlemciler: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro İşlemci, Pentium II, Pentium, 4 ve 64-bit Uzunları ile Core2 Minari, Programlama ve Arayüzleme, Sekizinci Baskı

Barry B. Brey

Telif Hakkı ©2009 Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • Tüm hakları saklıdır.

KESİNTİ KAVRAMI

Örnek 1: Ofisteki adam

- Ofisinde çalışan bir adamı düşünün. Çalışması sırasında çeşitli kesintilere maruz kalabilir.
- Bununla başa çıkmayı, ertelemeyi veya görmezden gelmeyi seçebilir. kesinti.
- Örneğin:
 - Bir arkadaş gelir ve bir iyilik ister. O, meşgul, arkadaşına daha sonra gelmesini söyler.
 - Telefon çalıyor, ama o telefonu açmıyor.
 - Bazı kesintileri göz ardı etmemek gerekir - yangın alarmı!
- Kesintilere doğru şekilde cevap verebilmemiz gerekiyor.

Örnek 2: Otobüs şoförü

- Otobüs şoförü normalde otobüsü sürer. •

Yolculardan herhangi biri tarafından kesintiye uğrarsa, "DUR" butonuna basacak;

- Bir sonraki otobüs durağında otobüsü durdurun
- Açık kapı
- Yolcunun/yolcuların ayrılmasını bekleyin
- Kapıyı kapat
- Sürüşe devam edin

- Özetle, sabit bir dizi başlatacaktır. kesintiye uğradığında tepki verir, ardından sürüşe devam eder.

- İşlemcinin en önemli görevlerinden biri **kesmeleri yönetmektir**.

Hangi yöntem iyidir?

- Yani, bilgisayarın bir girdi beklerken "donması" gerekmez. Kapatılabilir ve diğer programları çalıştırabilir, vb. • Aslında bir girdi veya çıktı bekleyen bir programın tamamlanması CPU'dan çok az dikkat gerektirir. CPU'nun tutumu "Bir şeyin olduğunda bana geri dön" olarak özetlenebilir.

- Bu, çoklu görevli sistemler için çok önemlidir. Bir görev G/C'nin tamamlanmasını beklerken, diğeri CPU döngülerini kullanabilir. Ve çoğu program zamanının çoğunu G/C'yi bekleyerek geçirir.

- Örneğin, bir Word işlemciden bir dosya yazdırırsanız, karakterlerin yazıcıya aktarımı kesmeler kullanılarak arka planda gerçekleşir. Hala yazabilirsiniz (ve diğer programları çalıştırabilirsiniz).

Intel'deki kesintiler

- Intel işlemcilerde kesme isteğinde bulunan iki adet donanım pini (INTR ve NMI) bulunur .
- Ve INTR aracılığıyla talep edilen kesintiye onaylamak için bir donanım pimi (INTA). • İşlemcinin ayrıca yazılım kesintileri vardır

INT, INTO gibi

- Bayrak biti IF (kesme bayrağı) ayrıca kesme yapısı ve özel dönüş talimatı IRET ile birlikte kullanılır

– 80386, 80486 veya Pentium'daki IRETD

Yazılım Kesinti Talimatları

- Mikroişlemciye ortak yazılım kesme talimatları sunulur:

- INT koşulsuzdur
- INTO koşulludur.

- IRET özel bir kesme dönüş talimatıdır.

KESİNTİLER ve G/C

- Görevlerin senkronizasyonu, bir şirkette çok önemli bir işlevdir. bilgisayar.
- Tipik bir bilgisayardaki tüm bileşenlerin işlemciyle iletişim kurmaya çalıştığını ve işlemcinin de onlarla iletişim kurmaya çalıştığını hayal edin.

- Hızlı bir bilgisayar, genellikle nispeten yavaş G/C aygıtlarıyla etkileşime girmek zorundadır; yavaş olmalarının nedeni mekanik olmaları veya insan dostu hızlarda çalışmaları gerektiridir.

- Etkileşimin iki yolu
 - Anket (EI sıkışma)
 - Yarıda kesmek

Neyin kesintiye uğraması gerekiyor?

- Harici Aygıtlar

- Klavye, Fare, Sensörler, vb.

- Dahili Etkinlikler

- Resetleme, Yasadışı İşlem, Kullanıcı Kesintisi, vb.

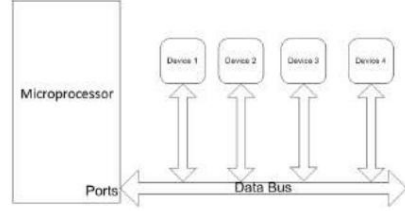
Kesinti Vektörleri

- Kesinti vektörleri ve vektör tablosu, donanımın anlaşılması için çok önemlidir ve yazılım kesintileri.
- Kesinti vektör tablosu şurada bulunur: 000000H-0003FFH adreslerindeki belleğin ilk 1024 baytı.
 - 256 farklı dört baytlık kesme vektörü içerir
- Bir kesinti vektörü adresi içerir (kesinti hizmet rutini/prosedürünün (ISR) segmenti ve ofseti).

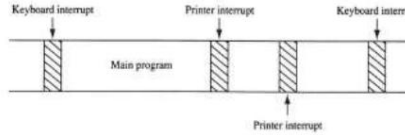
- INTO taşma bayrağını (O) kontrol eder veya test eder.
 - Eğer O = 1 ise, INTO adresi kesme vektörü türü 4'te depolanan prosedürü çağırır – Eğer O = 0 ise, INTO hiçbir işlem yapmaz ve sonraki ardışık program talimatı yürütülür
- INT n talimatı, adresteki kesme hizmeti prosedürünün çağırır n vektör numarasıyla temsil edilir .
- IRET talimatı özel bir geri dönüş talimatıdır Hem yazılım hem de donanım kesintileri için geri dönmek amacıyla kullanılan talimat . – uzak bir RET'e çok benzer şekilde, geri dönüşü alır Yığından adres

Anket

- Her cihaza sırayla ihtiyacı olup olmadığını "sorun" servis. Not: Anket sırasında hiçbir cihazın servise ihtiyacı olmayabilir.



Şekil 12-1 Tipik bir sistemdeki kesinti kullanımı gösteren zaman çizelgesi.



- bir zaman çizelgesi klavyede yazmayı gösteriyor, bellekten veri kaldıran bir yazıcı ve çalıştırılan bir program
- klavye kesintisi tarafından çağrılan klavye kesintisi hizmet prosedürü ve yazıcı kesintisi hizmet prosedürü her biri uygulanması çok az zaman alır

Kesinti Hizmeti Rutini (ISR)

- ISR, tasarlanmış özel bir alt programdır kesintiye "hizmet etmek" için
 - “Kesinti işleyicisi” olarak da adlandırılır
- Başka bir deyişle, ISR şunları içerir: Belirli bir kesme gerçekleştiğinde yürütülecek alt rutin.

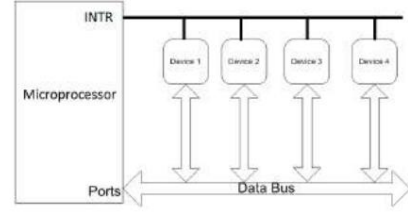
Gerçek Mod Kesintisinin Çalışması

- İşlemci geçerli talimatı yürütmeyi tamamladığında , aşağıdakileri kontrol ederek bir kesintinin etkin olup olmadığını belirler : – (1) talimat yürütmeleri – (2) tek adımı

- (3) NMI –
- (4) yardımcı işlemci segmenti taşıması
- (5) GİRİŞ
- (6) INT talimatları, sunulduğu sırayla

Yarıda kesmek

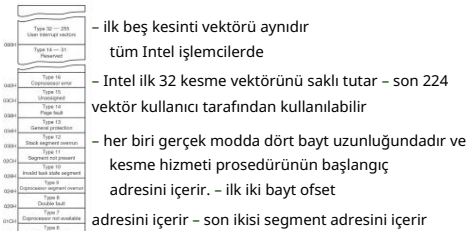
- Aygıt, bakıma ihtiyacı olduğunu belirtmek için CPU'yu "keser".



Kesinti Türleri

- Yazılım kesintisi , bir kesintinin oluşturulmasıdır Bir işlemci içerisinde bir talimatı yürüterek. Yazılım kesmeleri genellikle sistem çağrılarını uygulamak için kullanılır.
- Donanım kesintisi , G/C aygıtları veya çevre birimleri gibi donanımlar tarafından oluşturulan bir kesintidir
- Maskelenebilir kesinti: bir donanım kesintisidir. göz ardı edilebilir.
- Maskelenemeyen kesinti (NMI), asla göz ardı edilemeyen bir donanım kesintisidir. NMI'ler, zamanlayıcılar, sıfırlama vb. gibi en yüksek öncelikli görevler için kullanılır.

Şekil 12-2 (a) Mikroişlemci için kesme vektörü tablosu ve (b) bir kesme vektörünün içeriği .



adresini içerir – son ikisi segment adresini içerir

- Bir veya daha fazlası mevcutsa: – 1. Bayrak kayıt defterinin içerikleri yığına itilir.
 - 2. Kesme bayrağı temizlenir ve INTR pini devre dışı bırakılır.
 - 3. Kod segmenti kaydının (CS) içeriği yığına itilir.
 - 4. Talimat işaretçisinin (IP) içeriği yığına itilir.
- 5. Kesinti vektörünün içerikleri getirilir ve IP ve CS'ye yerleştirilir, böylece bir sonraki talimat vektör tarafından adreslenen kesme hizmet prosedüründe yürütülür .
- 6. ISR tamamlandığında, yığından önceki duruma geri dönlür.

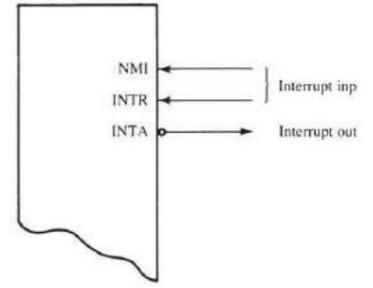
- Kesme bayrağı (IF), işlem tamamlandıktan sonra temizlenir. Bir kesme sırasında bayrak kaydının içeriği yığılır .

- IF ayarlandığında, INTR pininin neden olmasına izin verir bir kesinti
- IF temizlendiğinde, INTR pininin kesintiye neden olmaktan

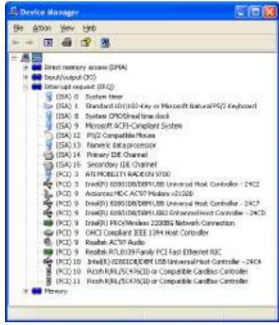
Donanım Kesintileri

- İki donanım kesinti girişi: – maskelenemeyen kesinti (NMI) – kesinti isteği (INTR) • NMI girişi etkinleştirildiğinde, tip 2 kesinti meydana gelir
 - çünkü NMI dahili olarak kodlanır
- INTR girişinin harici olarak kodlanması gerekir Bir vektör seçmek için.

- INTR pimi için herhangi bir kesme vektörü seçilebilir, ancak genellikle bir kesme kullanırız 20H ile FFH arasında tip numarası. • Intel, 00H - 1FH kesintilerini ayırdı dahili ve gelecekteki genişleme. • INTA aynı zamanda işlemcide bir kesme pimidir. – INTR girişine yanıt olarak kullanılan bir çıkıştır D7-D0 veri yolu bağlantılarına bir vektör tipi numarası uygulamak için
- Şekil 12-5, mikroişlemcideki üç kullanıcı kesme bağlantısını göstermektedir.



Kişisel Bilgisayarda Kesintiler



Emülatörde Kesintiler

- Birçok yararlı yazılım kesintisi vardır. Emülatör ile kullanılabilir.
- **INT değeri** talimatı , burada **değer** 0 ile 255 (veya 0 ile 0FFh) arasında bir sayı olabilir kullanılır.
- Desteklenen kesmelerin listesi ve bunların Açıklamalar emülatör belgelerinde mevcuttur.

Örnek: INT 10h

- **INT 10h** alt fonksiyonu **0Eh** ekrana tek bir karakter yazdırmak için kullanılır.
- **AL** kayıt defterinden yazdırılacak karakteri alır .
- **AH** register'ının değerini değiştirmez .

```
ORG 1000H ; instruct compiler to make single single segment .com file.

; The sub-function that we are using
; does not modify the AH register on
; return, so we may set it only once.
MOV AH, 0EH ; select sub-function.

; INT 10h / 0EH sub-function
; receives an ASCII code of the
; character that will be printed
; in AL register.

MOV AL, 'H' ; ASCII code: 72
INT 10H ; print int

MOV AL, 'a' ; ASCII code: 97
INT 10H ; print int

MOV AL, '1' ; ASCII code: 49
INT 10H ; print int

MOV AL, '!' ; ASCII code: 33
INT 10H ; print int

MOV AL, '*' ; ASCII code: 42
INT 10H ; print int

RET ; returns to operating system.
```

İmleç Pozisyonunu Ayarla

- **INT 10s / AH = 2**
- Girişler: – DH = satır – DL = sütun – BH = sayfa numarası (0..7)
- Örnek: **h**areketli **dh**, 10 **h**areketli **dl**, 20 **h**areketli **bh**, 0 **mov ah**, 2 **int 10h**

İmleç Konumunu ve Boyutunu Al

- **INT 10s / AH = 03s**
- Girişler: – BH = sayfa numarası.
- İfade: – DH = satır. – DL = sütun. – CH = imleç başlangıç satırı. – CL = imleç alt satırı.

Karakter ve Nitelik Oku

- **INT 10h / AH = 08h** - imleç konumundaki karakteri ve niteliği oku.
- Giriş: – BH = sayfa numarası.
- İfade: – AH = öznitelik. – AL = karakter.

Karakter ve Nitelik Yaz

- **INT 10h / AH = 09h** - imleç konumuna karakter ve öznitelik yaz.
- Girişler: – AL = görüntülenecek karakter. – BH = sayfa numarası. – BL = öznitelik. – CX = karakterin yazılacağı zaman sayısı.

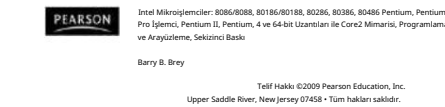
Nitelikler

- Karakter niteliği 8 bit değerindedir, düşük 4 bit rengi, yüksek 4 bit arka plan rengini belirler.

HEX Bili Rengi	HEX 816 Rengi	
0 0000 Siyah	8 1000 Koyu Gri	
1 0001 Mavi	9 1001 Açık Mavi	
2 0010 Yeşil	10 1010 Açık Yeşil	
3 0011 Camgöbeği	B 1011 Açık Camgöbeği	
4 0100 Kırmızı	C 1100 Açık Kırmızı	
5 0101 Macenta	D 1101 Açık Macenta	
6 0110 Kahverengi	E 1110 Sarı	
7 0111 Açık Gri F 1111 Beyaz		

Karakter Yaz

- INT 10h / AH = 0Ah - yalnızca karakteri yaz imleç konumu.
- Girişler: – AL = görüntülenecek karakter. – BH = sayfa numarası. – CX = karakterin yazılacağı zaman sayısı.



CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

Kayan Nokta Sayıları

- Bu bölümde gerçek sayıların gösterimi işlemcilerde anlatılmaktadır.

- Kayan nokta (FP) gösterimi tanıtıldı.

- Yaygın olarak tercih edilen kayan nokta sayı standardı IEEE 754 açıklanmaktadır.

- FP aritmetiği kısaca açıklanmıştır.

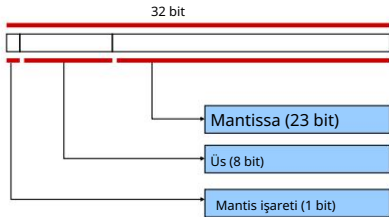
Kayan Nokta

- Farklı kayan nokta değerleri olmasına rağmen gösterimler arasında en sık karşılaşılan gösterim IEEE 754 Standardıdır.

- Kayan nokta gösteriminin sabit nokta (ve tam sayı) gösterimine göre avantajı, çok daha geniş bir değer aralığını destekleyebilmesidir. • Kayan nokta

işlemlerinin hızı, birçok uygulama alanındaki bilgisayarlar için önemli bir performans ölçüsüdür . FLOPS cinsinden ölçülür.

Tek Hassasiyet Formatı



Özel Değerler

- Durum

– örn. = 111...1

- Vakalar

– örn. = 111...1, kesir = 000...0

- Değeri temsil eder (sonsuz)

- Taşın işlem

- Hem olumlu hem olumsuz

- Örn., 1,0/0,0 = 1,0/ 0,0 = + , 1,0/ 0,0 =

– örn. = 111...1, frak 000...0

- Sayı Değil (NaN)

- Sayısal bir değer belirlenemediği durumu temsil eder

- Örneğin, sqrt(-1),

Bölüm Hedefleri

Bu bölümü tamamladığınızda şunları yapabileceksiniz:

- Kayan noktanın nedenini açıklayın

sayılara ihtiyaç var.

- Kayan nokta ve gerçek sayı gösterimleri arasında dönüşüm yapın.

- FP aritmetiğini gerçekleştirin.

Üstel Notasyon

- Aşağıdakiler 1.234'ün eşdeğer gösterimleridir

123.400,0	x 10 ⁻²
12.340,0	x 10 ⁻¹
1.234,0	x 10 ⁰
123,4	x 10 ¹
12,34 x 10 ²	
1,234 x 10 ³	
0,1234 x 10 ⁴	

Gösterimler ondalık basamağının farklı olması nedeniyle farklılık gösterir – "nokta" -- (üsse uygun ayarlama ile) sola veya sağa "yüzer".

Normalleştirme

- Mantis normalize edildi

- Sol tarafta ima edilen bir ondalık basamak vardır

- Ondalık basamağın solunda ima edilen bir "1" vardır

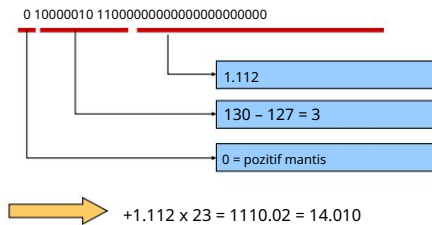
- Örneğin,

– Mantis 101000000000000000000000

- Temsil etmek... 1.1012 = 1.62510

Örnek

- Tek hassasiyet



Kayan Nokta

- Üst düzey dillerde, değişken türlerini şu şekilde tanımlayabilirsiniz:

– int

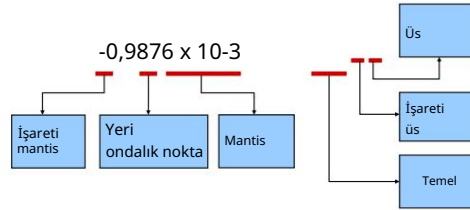
– batmadan yazmak

– çift – vb.

- Ancak bu, assembly dilinde doğrudan mümkün değildir.

- İkili sayılar, kullandığınız gösterime bağlı olarak farklı anlamlara gelebilir.

Kayan Nokta Sayılarının Bölümleri



Fazla Notasyon

- Üsleri dahil etmek için "fazlalık" gösterimi kullanılır • Tek hassasiyet: fazlalık 127

- Çift hassasiyet: fazlalık 1023

- Depolanan üssün değeri gerçek üsten daha büyüktür

- Örn., fazlalık 127, –

Üs 10000111

- Temsil etmek... 135 – 127 = 8

Onaltılık

- Orijinal kayan nokta sayısını onaltılık sayı sisteminde temsil etmek uygun ve yaygındır

- Önceki örnek...

0 10000010 1100000000000000000000
4 1 6 0 0 0 0 0

Kayan Nokta

- Kayan nokta gösterimi gerçek değerleri temsil etmenin bir yoludur ikili biçimdeki sayılar.

- Sayılar genellikle belirli sayıda anlamlı basamağa kadar temsil edilir ve bir üs kullanılarak ölçeklenir.

- Ölçeklemenin tabanı normalde 2, 10 veya 16'dır. • Tam olarak temsil edilebilen tipik sayı, biçim:

anlamlı rakamlar x taban üs

- Kayan nokta terimi, taban noktasının (ondalık nokta veya bilgisayarlı nokta veya bilgisayarlı nokta veya bilgisayarlı nokta) "yüzer" olabilir; yani sayının anlamlı basamaklarına göre herhangi bir yere yerleştirilebilir.

IEEE 754 Standard

- Kayan nokta sayılarını temsil etmek için en yaygın standart

- Tek hassasiyet: 32 bit, şunlardan oluşur:

- İşaret biti (1 bit)
- Üs (8 bit)
- Mantissa (veya kesir) (23 bit) • Çift

hassasiyet: 64 bit, şunlardan oluşur:

- İşaret biti (1 bit)
- Üs (11 bit)
- Mantissa (veya kesir) (52 bit)

Denormalize Edilmiş Değerler

- Durum

– örn. = 000...0

- Değer

– Üs değeri E = –Bias + 1 – Önemli değer

m = 0.xxx...x2

• xxx...x: frac parçaları

- Vakalar

– exp = 0 = 000...0, kesir = 000...0

değerini temsil eder

• +0 ve -0'ın farklı değerlere sahip olduğunu unutmayın

– örn. = 000...0, kesir 000...0

• 0,0'a çok yakın sayılar

• Küçülürken hassasiyeti kaybedersiniz

• "Yavaş yavaş azalma"

Kayan Noktadan Dönüştürme

- Örn., aşağıdaki 32 bitlik kayan noktalı sayı hangi ondalık değeri temsil eder?

C17B000016

- Adım 1
 - İkili olarak ifade edin ve S, E ve M'yi bulun

C17B000016 =

1 10000010 111101100000000000000002

S E M

1 = olumsuz
0 = pozitif

Kayan Nokta Dönüştürme

- Örn. 36.562510'u 32 bitlik kayan nokta sayısı (onaltılık) olarak ifade edin

36.562510 =

100100.10012

S.M.

100100.10012

+1.0010010012 x 25

S.M.

N

- Adım 4
 - 32 bitlik ikiliyi oluşturmak için S, E ve M'yi bir araya getirin sonuç

0 10000100 001001001000000000000002

Güney Doğu M

FP Ekleme

İşlenenler

- (-1)s1 M1 2 E1
- (-1)s2 M2 2 E2

Kesin Sonuç

- (-1)sn M 2 E

İşaret s, anlamlı M: İşaretili hizalama ve eklemenin sonucu

Üs E: E1

Düzeltilme

- Eğer M 2 ise, M'yi sağa kaydır, E'yi artır
- eğer M < 1 ise, M'yi k pozisyon sola kaydır, E'yi k kadar azalt
- E aralık dışındaysa taşıma
- Kirik hassasiyetine uyması için yuvarlak M

E1-E2

(-1)s1m1

(-1)s2m2

(-1)sn m

C17B000016 =

1 10000010 111101100000000000000002

Güney Doğu M

- Adım 2
 - "Gerçek" üssü bulun, n - n = E
 - 127
 - = 10000102 - 127
 - = 130 - 127
 - = 3

- Adım 1
 - Orijinal değeri ikili olarak ifade edin

36.562510 =

100100.10012

S.M.

100100.10012

+1.0010010012 x 25

S.M.

N

- Adım 5
 - Onaltılık sayı sisteminde ifade edin

0 10000100 001001001000000000000002 =

0100 0010 0001 0010 0100 0000 0000 00002 =

4 2 1 2 4 0 0 016

Cevap: 4212400016

Intel Mikroişlemcide FP İşlemleri

- Intel 80X87 ailesi yardımcı işlemci desteği kayan nokta sayıları.
- 80X87 harici bir entegre devredir mikroişlemci ile eş zamanlı çalışacak şekilde tasarlanmıştır.
 - 80486DX-Core2'de 80387'nin dahili sürümleri bulunur.
- İşlemci tüm normal talimatları yürütür ve 80X87 yardımcı işlemci talimatlarını yürütür.
 - 80X87 68 farklı talimatı yürütür

C17B000016 =

1 10000010 111101100000000000000002

S E M

- Adım 3
 - S, M ve n'yi bir araya koyup ikili sonucu oluştur - (Mantisin solundaki "1"i unutmayın.)
 - 1.11110112 x 2n =
 - 1.11110112 x 23 =
 - 1111.10112

- Adım 2
 - Normalleştirmek

100100.10012 =

1.0010010012 x 25

Kayan Nokta İşlemleri

- Kavramsal Görünüm
 - İlk önce kesin sonucu hesaplayın
 - İstenilen hassasiyete uymasını sağlayın
 - Üs çok büyükse taşıma olasılığı vardır
 - Muhtemelen fra'ı uysak pekilde yuvarlak
- Yuvarlama Modları (yuvarlama ile gösterin)

1,40 \$ 1,60 \$ 1,50 \$ 2,50 - Sıfır 1,00 \$ 1,00 -1,50 dolar

\$ 1,00 \$ 2,00 - 1,00 \$ - Aşağı yuvarla (-) 1,00 \$ 1,00 \$ 1,00 \$ 2,00 - 2,00 \$
- Yuvarla (+) \$2,00 \$2,00 \$2,00 \$3,00 - \$1,00
- En Yakın Çift (varsayılan) \$1.00 \$2.00 \$2.00 \$2.00 - \$2.00
- Not:
 - Aşağı yuvarlama: Yuvarlanan sonuç gerçek sonuca yakındır ancak gerçek sonuçtan büyük değildir.
 - Yuvarlama: Yuvarlama sonucu gerçek sonuca yakın, ancak gerçek sonuçtan daha az değildir.

ÖZET

- İşlemcilerde reel sayıların gösterimi anlatılmaktadır.

- FP temsiliyeti tanıtıldı.
- IEEE 754 standardı anlatılmaktadır.
- FP aritmetiği kısaca açıklanmıştır.

C17B000016 =

1 10000010 111101100000000000000002

S E M

- Adım 4
 - Sonucu ondalık olarak ifade edin

-1111.10112

-15

2 -1 = 0,5
2 -3 = 0,125
2 -4 = 0,0625
0,6875

Cevap: -15.6875

- Adım 3
 - S, E ve M'yi belirleyin

+1.0010010012 x 25

S.M.

N

E = n + 127
= 5 + 127
= 132
= 100001002

S = 0 (çünkü değer pozitifdir)

FP Çarpımı

- İşlenenler
 - (-1)s1 M1 2 E1
 - (-1)s2 M2 2 E2
- Kesin Sonuç
 - (-1)sn M 2 E
- İşaret s: s1 ^ s2
- Önemli M: M1 * M2
- Üs E: Düzeltme E1 + E2
- Eğer M 2 ise, M'yi sağa kaydır, E'yi artır
- E aralık dışındaysa, taşıma
- Kirik hassasiyetine uyması için yuvarlak M
- Uygulama
 - En büyük iş, anlamları çoğaltmaktır

Ek Referanslar

- Ders Notları, "Bilgi Teknolojilerine Giriş", York Üniversitesi
- Ders Notları, CS 213 Dersi.



Intel Mikroİşlemciler: 8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro İşlemci, Pentium II, Pentium, 4 ve 64-bit Uzantıları ile Core2 Mimarisi, Programlama ve Arayüzleme, Sekizinci Baskı

Barry B. Brey

Telif Hakkı ©2009 Pearson Education, Inc.
Upper Saddle River, New Jersey 07458 • Tüm hakları saklıdır.

CSE213

MİKRODENETLEYİCİ PROGRAMLAMA

Mikroİşlemci Performansı

Soru

- Ayrı görevler için ayrı işlemciler kullanan bir sisteme ek bir işlemci eklediğimizi varsayalım.

- Bu, tepki süresini iyileştiriyor mu?
- Bu, verimi artırıyor mu?

CPU Performansı

- CPU saat döngüleri
= talimat sayısı x CPI (Talimat başına döngü)
- CPU zamanı
= talimat sayısı x CPI x Saat çevrim süresi



Bilgisayar Saati Zamanları

- Bilgisayarlar, çalışan bir saate göre çalışır. sabit bir oranda
- Zaman aralığına saat döngüsü denir (örneğin, 10ns).
- Saat hızı, saat döngüsünün tersidir - bir frekans, saniyedeki döngü sayısı (örneğin, 100MHz). – 10 ns = 1/100.000.000 (saat döngüsü), şuna benzer:- – 1/10ns = 100.000.000 = 100MHz (saat hızı).

Talimat Başına Döngü (Verim)

"Talimat Başına Ortalama Döngüler"

$$CPI = (CPU \text{ Süresi} \times \text{Saat Hızı}) / \text{Talimat Sayısı} = \text{Döngüler} / \text{Talimat Sayısı}$$

$$CPU \text{ zamanı} = \sum_{j=1}^N \text{Döngü Süresi}_{CPIj} \quad I_j$$

$$TÜFE \text{ } \frac{N}{TÜFE} = \frac{1}{\sum_{j=1}^N \frac{1}{CPIj}} \quad F \text{ nerede} = \frac{\text{Saat Hızı}}{\text{Talimat Sayısı}} \quad \text{"Talimat Sıklığı"}$$

Amdahl Yasası

Sistemde bir iyileştirme yapıldıktan sonra yürütme süresi aşağıdaki şekilde verilir:

$$\text{İyileştirmeden sonra yürütme süresi} = I / A + E$$

I = iyileştirmeden etkilenen yürütme süresi
A = iyileştirme miktarı
E = yürütme süresi etkilenmez

Performans

- Tepki süresi:** Görevin başlangıcı ile bitişi arasındaki süre (örneğin, yürütme süresi)
- Verim:** belirli bir sürede yapılan toplam iş miktarı

Satın Alma Kararı

- Bilgisayar A'nın 100MHz işlemcisi var
- Bilgisayar B'nin 300MHz işlemcisi var
- Yani B daha hızlı, değil mi?
- EMİN DEĞİLİM!**
- Şimdi, doğrusunu yapalım.....

Örnek: TÜFE'nin hesaplanması

Temel Makine (Reg / Reg)				
İşlem	Frekans	Döngüleri	CPI(i)	(% Zaman)
ALÜ	%50	1.5	(33%)	
Yük	%20	2	%10	.4 (27%)
Mağaza	2	%20	2	.2 (%13)
Dal			.4	(27%)
			1.5	

Tipik Talimat Türlerinin Karışımı programda

Soru

- Bilgisayardaki işlemciyi daha hızlı bir modelle değiştirdiğimizi varsayalım . Bu tepki süresini iyileştirir mi?
- Peki verim nasıl?

CPU Performansı

- Bir program için CPU Yürütme süresi

$$= \text{Bir program için CPU Saat Döngüleri} \times \text{Saat Döngüsü süresi}$$

$$= \text{Bir program için CPU Saat Döngüleri} / \text{Saat hızı}$$

Performans

(Mutlak) Performans

$$\text{Performance}_X = \frac{1}{\text{Execution time}_X}$$

Göreceli Performans

"X, Y'den n kat daha hızlıdır" demek

$$n = \frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X}$$

Referanslar

- Performans Ölçümü, Chris Clack, B261 Sistem Mimarisi.
- CSCE 350 Bilgisayar Mimarisi, Rabi Mahapatra